Chapter 1: Introduction to Computer Networks

Welcome to the world of computer networks! In this chapter, we’ ll learn what computer networks are, why we use them, how to judge their quality, and the different shapes and styles they come in.

1.1 Definition and Goals of Computer Networks

Definition:
A computer network is a group of two or more computers (or devices like printers, smartphones, game consoles) connected together so they can share information and resources.

Think of it like a neighborhood of houses connected by roads – the roads let people visit each other, exchange news, or share tools. Similarly, a network lets computers send data back and forth.

Main goals of a computer network:

1. **Resource sharing** – Share hardware (printers, scanners), software (applications), and files (documents, photos) across many computers.
   Example: In an office, 10 people can use one networked printer instead of buying 10 printers.

2. **Communication** – People can send emails, chat, make video calls, or work together on documents.
   Example: Zoom calls or WhatsApp messages travel over networks.

3. **Cost reduction** – Sharing expensive devices and centralizing data storage saves money.
   Example: A school stores all student records on one central server instead of every teacher’ s PC.

4. **Reliability** – If one computer fails, others can take over (with backup systems).
   Example: Large websites like Google use thousands of servers so that if a few break, the site still works.

5. **Scalability** – You can add more computers or devices without breaking the existing network.
   Example: Your home Wi-Fi works with 2 phones today and 5 phones next month – just connect them.

1.2 Network Criteria – How to Judge a Network

Not all networks are equally good. We judge them by three main criteria: **Performance**, **Reliability**, and **Security**.

1.2.1 Performance

- **Throughput** – How much data can be sent per second (measured in Mbps or Gbps).
- **Delay (Latency)** – The time it takes for a piece of data to travel from source to destination.
- **Jitter** – Variation in delay. For voice/video calls, low jitter is important.
- **Number of users** – More users sharing the network usually slows it down.

1.2.2 Reliability

- **Fault tolerance** – The network keeps working even if some parts fail.
- **Error rate** – How often data gets corrupted during transmission.
- **Uptime** – The percentage of time the network is available.

1.2.3 Security

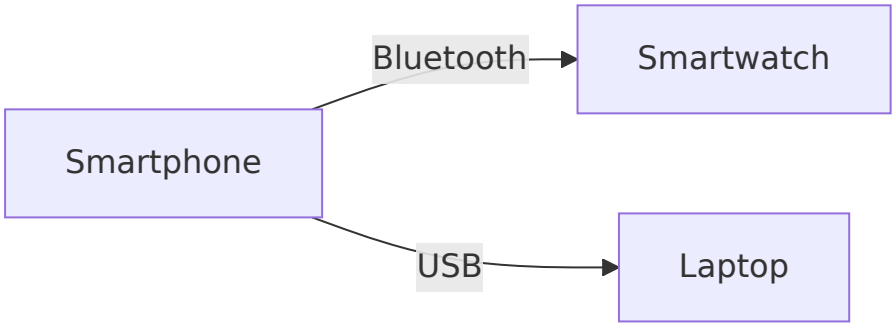
- **Confidentiality** – Only authorized people can see the data.
- **Integrity** – Data is not changed by accident or on purpose.
- **Availability** – Authorized people can access the network when they need to.
- **Authentication** – Verify that a user or device is who they claim to be.

1.3 Types of Networks

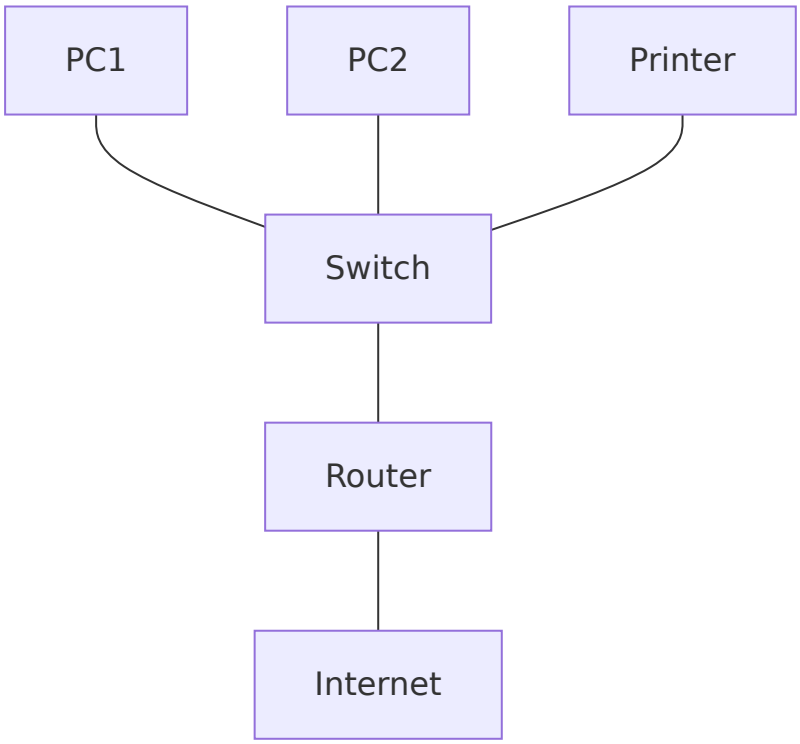
Type	Full Name	Range	Typical Use
PAN	Personal Area Network	A few meters	Connecting phone, laptop, smartwatch

LAN	Local Area Network	A building or campus	Sharing files, printers, internet in an office
MAN	Metropolitan Area Network	A city	Connecting multiple buildings of a university across a city
WAN	Wide Area Network	Countries or continents	The Internet, or a company with global offices
WLAN	Wireless Local Area Network	Same as LAN, but wireless	Home Wi-Fi, airport Wi-Fi

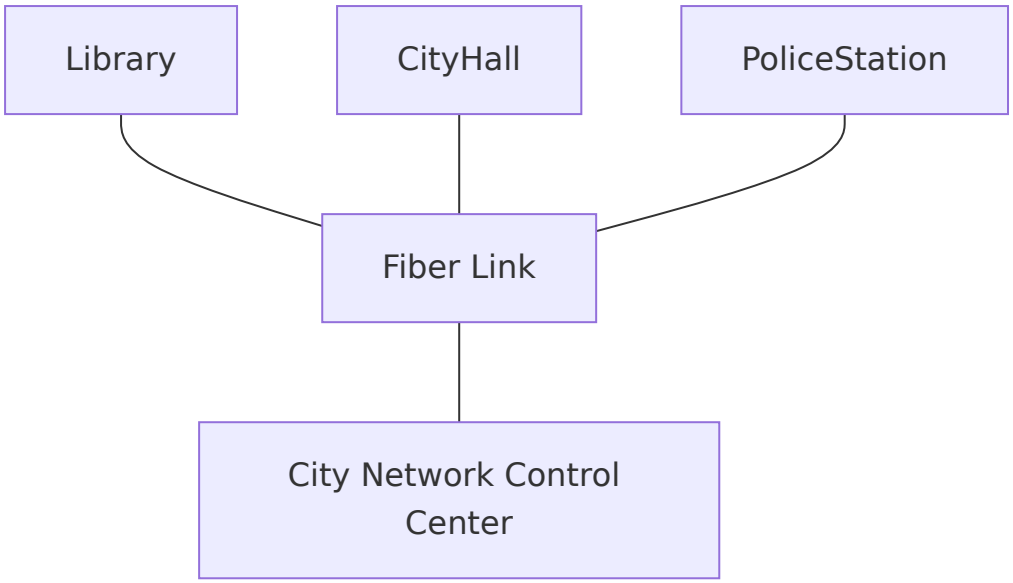
1.3.1 PAN (Personal Area Network)



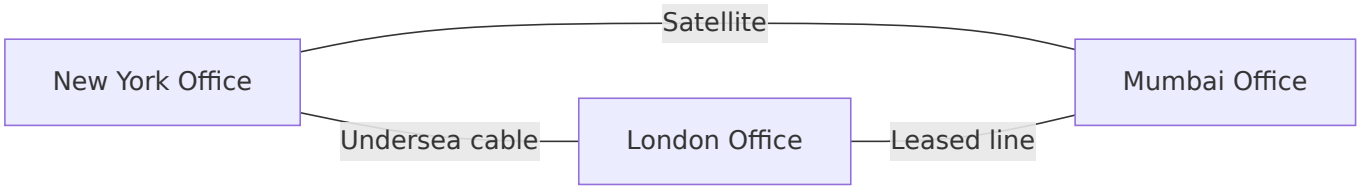
1.3.2 LAN (Local Area Network)



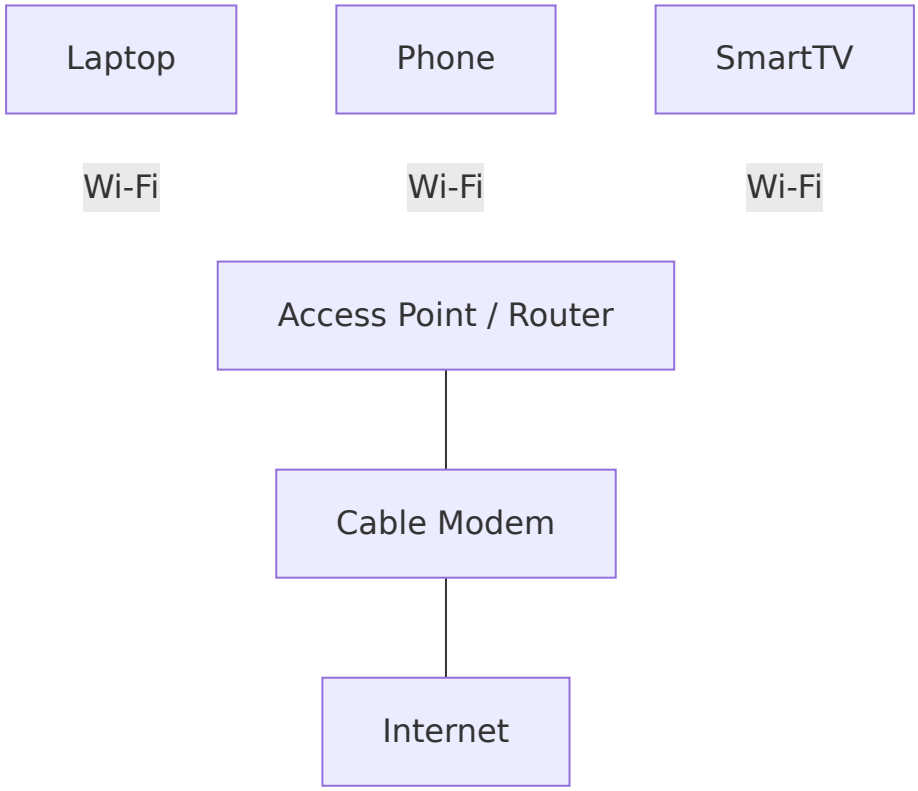
1.3.3 MAN (Metropolitan Area Network)



1.3.4 WAN (Wide Area Network)



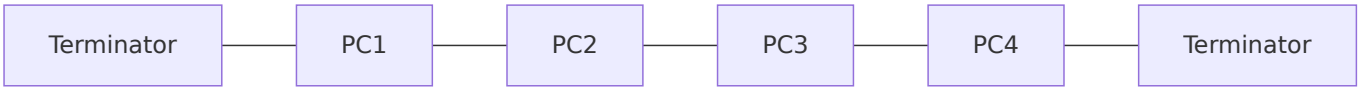
1.3.5 WLAN (Wireless Local Area Network)



1.4 Network Topologies

1.4.1 Bus Topology

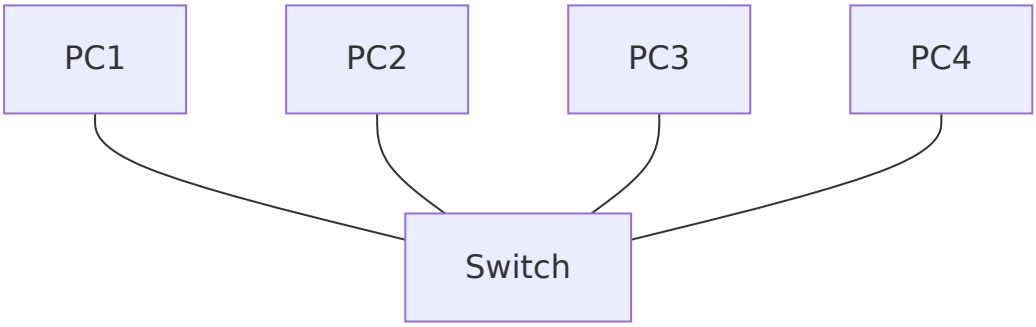
All devices share a single central cable (the bus). Data travels both ways; every device sees the data, but only the intended recipient accepts it.



- **Advantages:** Cheap, easy to install, uses little cable.
- **Disadvantages:** A break in the bus brings down the whole network. Only one device can send at a time.
- **Example:** Very old Ethernet networks – rarely used today.

1.4.2 Star Topology

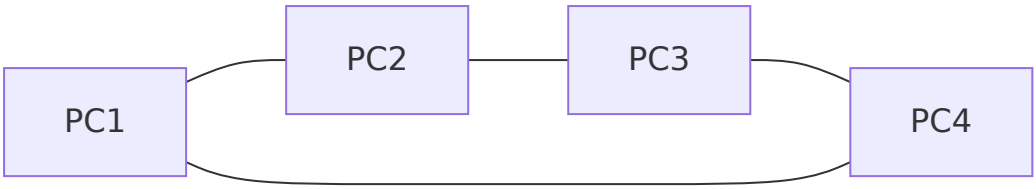
All devices connect to a central switch or hub. Every message goes through the center.



- **Advantages:** If one cable fails, only that device is affected. Easy to add devices. Easy to find faults.
- **Disadvantages:** If the central switch fails, the whole network goes down. Requires more cable than bus.
- **Example:** Most home and office networks today (Ethernet switches, Wi-Fi access points).

1.4.3 Ring Topology

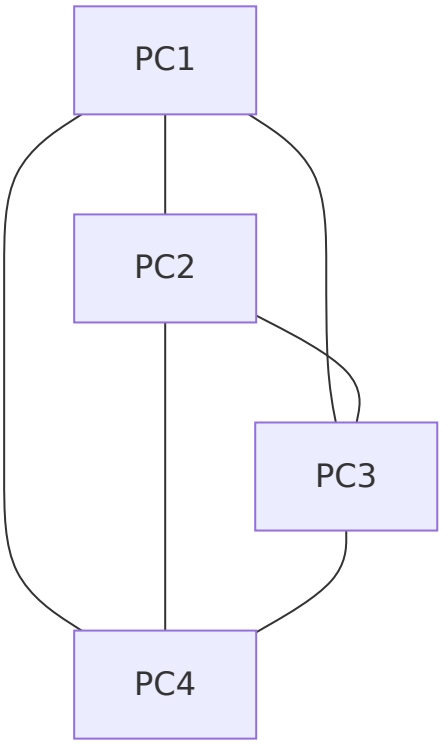
Devices are connected in a closed loop. Data travels in one direction (or both directions in some rings) from device to device.



- **Advantages:** Orderly data flow (no collisions). Each device can repeat the signal, allowing longer distances.
- **Disadvantages:** A break in the ring or a failed device can bring down the whole network (unless it’ s a dual ring).
- **Example:** Older IBM Token Ring networks, some fiber optic rings (SONET).

1.4.4 Mesh Topology

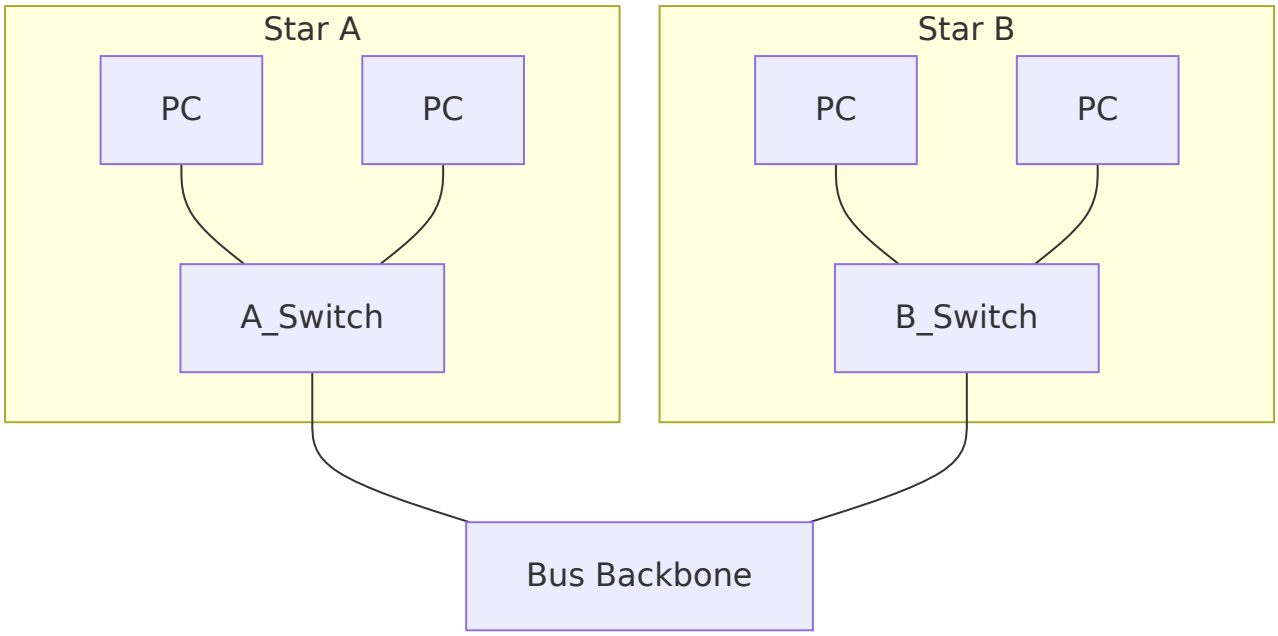
Every device is connected to every other device. This gives many redundant paths.



- **Advantages:** Very reliable – many alternative paths. No single point of failure.
- **Disadvantages:** Expensive – needs many cables and ports. Complex to set up.
- **Example:** Backbone of the Internet, military networks, large data centers.

1.4.5 Hybrid Topology

Combines two or more basic topologies (e.g., stars connected by a bus backbone).

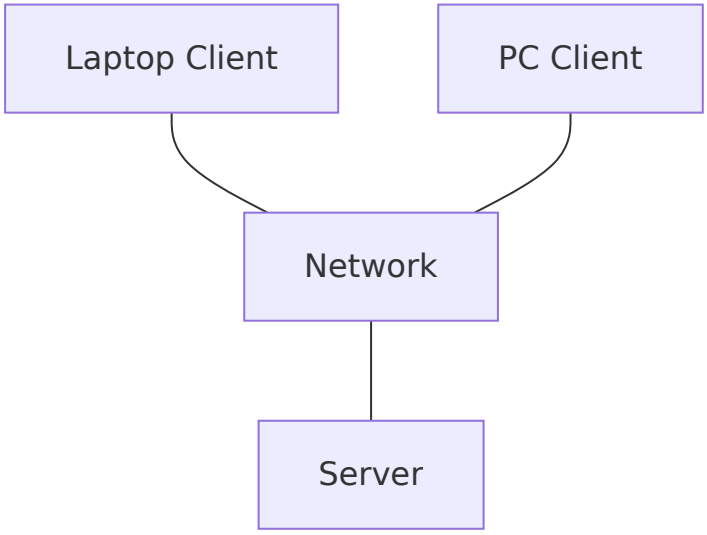


- **Advantages:** Flexible, can be designed to fit specific needs.
- **Disadvantages:** Complex design, more expensive, harder to troubleshoot.
- **Example:** Large corporate networks (each department is a star, linked via a ring or bus backbone).

1.5 Network Models

1.5.1 Client-Server Model

- **Server** – A powerful computer (or software) that provides services.
- **Client** – Any device that asks the server for something.



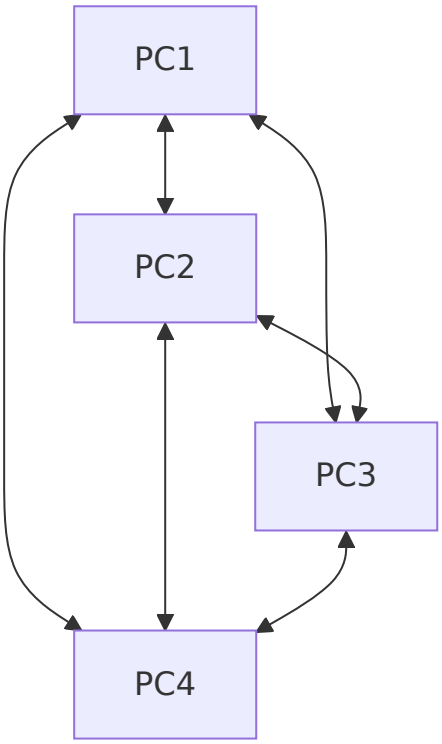
How it works:
Client sends a request (“give me file.pdf”). Server processes it and sends back the answer.

Examples: Web browsing, email, online banking.

Advantages: Centralized management, easy access control, powerful servers.
Disadvantages: Server is a single point of failure; expensive; needs an administrator.

1.5.2 Peer-to-Peer (P2P) Model

Every computer (peer) is equal. Each peer can act as both a client (asking) and a server (sharing). No central server.



(Arrows show that any two peers can communicate directly.)

Examples: BitTorrent, old file-sharing apps, small home or office workgroups.

Advantages: No expensive server, easy to set up, very scalable.
Disadvantages: Security risks, no central backup, performance depends on the slowest peer.

Comparison Table

Feature	Client-Server	Peer-to-Peer
Central control	Yes (server)	No

Cost	Higher (server needed)	Lower
Security	Better (central policies)	Weaker
Reliability	Single point of failure	No single failure point (but peers can leave)
Example	Web, email, databases	BitTorrent, Windows Workgroup

Summary

- A **computer network** connects devices to share resources and communicate.
- Three key **criteria** to judge a network: performance, reliability, security.
- **Types of networks** range from small (PAN) to huge (WAN); LAN and WLAN are most common.
- **Topologies** describe the layout: Bus, Star, Ring, Mesh, Hybrid. Star is most popular today.
- **Network models** are either Client-Server (centralized) or Peer-to-Peer (decentralized). Each has its own strengths.

Now you have a solid foundation to understand how networks are built. In the next chapter, we’ ll explore how data is actually sent across these networks using layers, protocols, and addressing.

Chapter 2: Network Models (OSI and TCP/IP)

In this chapter, we’ ll learn about two important models that help us understand how networks work: the **OSI Model** and the **TCP/IP Model**. These models break network communication into smaller, simpler layers. Each layer has a specific job. We’ ll also see how data is wrapped (encapsulated) and unwrapped (decapsulated) as it travels.

2.1 Why Do We Need Network Models?

Imagine sending a letter. You don’ t just throw the paper – you put it in an envelope, write an address, add a stamp, drop it in a mailbox, and the postal service handles the rest. Network models work the same way: they divide the work into steps (layers). Each layer talks only to the layers above and below it. This makes network design easier, and changes in one layer don’ t affect the others.

2.2 The OSI Model (Open Systems Interconnection)

The OSI model has **7 layers**. It’ s a theoretical model created by the International Organization for Standardization (ISO) to help vendors and developers create interoperable network products.

2.2.1 The 7 Layers – From Bottom to Top

We’ ll start from the bottom (closest to the physical hardware) and move up to the application (closest to the user).

Layer #	Layer Name	Function (in simple words)
1	Physical	Transmits raw bits (0s and 1s) over a physical medium (cable, radio, fiber).
2	Data Link	Organizes bits into frames; handles physical addressing (MAC addresses); error detection.

3	Network	Routes packets from source to destination across multiple networks; uses logical addressing (IP addresses).
4	Transport	Provides reliable or unreliable data delivery; error recovery; flow control; uses port numbers.
5	Session	Manages sessions (dialogues) between applications; establishes, maintains, and terminates connections.
6	Presentation	Translates data formats (encryption, compression, character encoding).
7	Application	Provides network services directly to user applications (e.g., web browser, email client).

2.2.2 Detailed Functions of Each Layer with Examples

Layer 1 – Physical Layer

- **What it does:** Converts bits into electrical signals, light pulses, or radio waves. Defines cables, connectors, voltage levels, and data rates.
- **Examples of protocols/technologies:** Ethernet (physical part), USB, Bluetooth radio, fiber optics, DSL.
- **Real-life analogy:** The actual road or train track on which vehicles (bits) travel.

Layer 2 – Data Link Layer

- **What it does:** Groups bits into **frames**. Adds a **MAC address** (hardware address) of the source and destination. Detects (and sometimes corrects) transmission errors. Controls access to the physical medium (e.g., CSMA/CD for Ethernet).
- **Two sublayers:** LLC (Logical Link Control) and MAC (Media Access Control).
- **Examples:** Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), PPP (Point-to-Point Protocol).
- **Real-life analogy:** The postal service within a single city – it knows how to deliver a letter to a specific house (MAC address) on that street.

Layer 3 – Network Layer

- **What it does:** Routes **packets** across different networks. Uses **IP addresses** (logical addresses) to find the best path. Fragments large packets if needed.
- **Examples:** IP (Internet Protocol – IPv4 and IPv6), ICMP (ping), RIP, OSPF (routing protocols).

- **Real-life analogy:** The interstate highway system and GPS – it figures out the best route from New York to Los Angeles, not just within one city.

Layer 4 – Transport Layer

- **What it does:** Provides end-to-end communication between applications. Uses **port numbers** (e.g., 80 for web, 443 for secure web). Two main protocols:
  - **TCP** (Transmission Control Protocol) – reliable, connection-oriented, error-checked, ordered delivery.
  - **UDP** (User Datagram Protocol) – fast, connectionless, no guarantee (used for streaming, gaming).
- **Examples:** TCP, UDP, SCTP.
- **Real-life analogy:** A registered mail service that tracks your package, resends it if lost (TCP) vs. a regular postcard that might get lost but is fast (UDP).

Layer 5 – Session Layer

- **What it does:** Sets up, manages, and ends **sessions** (conversations) between applications. Controls who can talk when (dialog control). Adds checkpoints for large data transfers (so if a transfer fails, it can resume from a checkpoint).
- **Examples:** NetBIOS, RPC (Remote Procedure Call), PPTP (VPN tunnel control).
- **Real-life analogy:** A meeting moderator who decides when each person speaks and when the meeting starts/ends.

Layer 6 – Presentation Layer

- **What it does:** Translates data between the application format and the network format. Handles **encryption/decryption**, **compression**, and **character encoding** (e.g., ASCII, UTF-8, EBCDIC).
- **Examples:** SSL/TLS (cryptography – though often placed in session/transport in practice), JPEG, GIF, ASCII, EBCDIC.
- **Real-life analogy:** A translator who converts English to French and also encrypts the message with a secret code.

Layer 7 – Application Layer

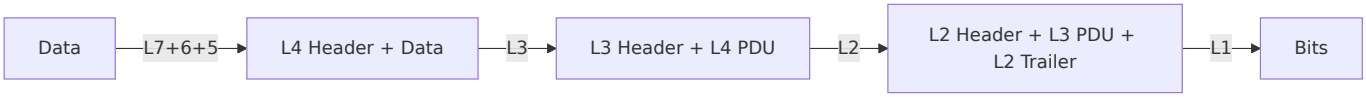
- **What it does:** Provides network services directly to the user’ s software (e.g., web browser, email client, file transfer). This is the layer the user interacts with.
- **Examples:** HTTP (web), HTTPS, FTP (file transfer), SMTP (email sending), POP3/IMAP (email receiving), DNS (domain name resolution), Telnet, SSH.

- **Real-life analogy:** The letter you write – the actual content of the communication.

2.3 Encapsulation and Decapsulation

When data travels from a sender to a receiver, each layer adds its own **header** (and sometimes a trailer). This process is called **encapsulation**. At the receiving side, each layer removes its header – **decapsulation**.

Simplified view:



Example: Sending “Hello” over HTTP

1. **Application layer (HTTP)** – “Hello” is the user data.
2. **Presentation layer** – optionally compresses/encrypts (still “Hello”).
3. **Session layer** – adds session ID (e.g., “session 123”).
4. **Transport layer (TCP)** – adds source port (e.g., 5000) and destination port (80). Creates a TCP segment.
5. **Network layer (IP)** – adds source IP (192.168.1.2) and destination IP (8.8.8.8). Creates an IP packet.
6. **Data Link layer (Ethernet)** – adds source MAC (AA:BB:CC:DD:EE:FF) and destination MAC (next hop router). Adds a trailer with error check (FCS). Creates an Ethernet frame.
7. **Physical layer** – sends bits as electrical pulses.

At the receiver, each layer strips its header and passes the rest upward.

2.4 Protocols at Each Layer of the OSI Model

Here’s a table of common protocols (not every protocol fits perfectly – some span layers, but this is typical):

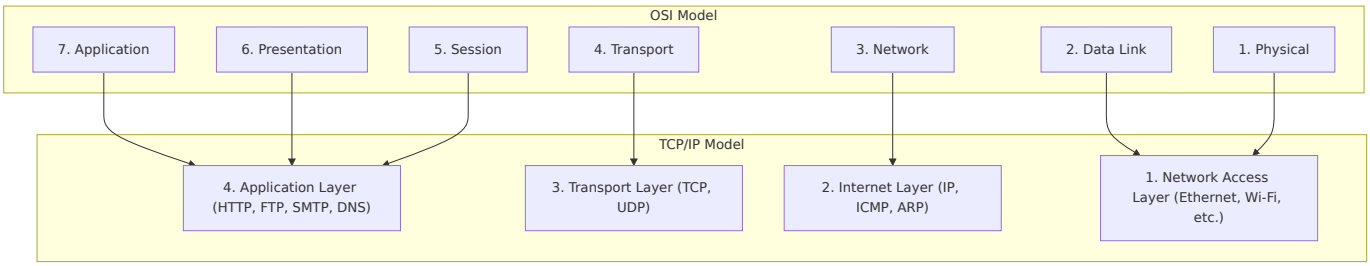
OSI Layer	Example Protocols / Technologies
7 – Application	HTTP, HTTPS, FTP, SMTP, POP3, IMAP, DNS, SSH, Telnet, SNMP
6 – Presentation	SSL/TLS (often here), JPEG, MPEG, ASCII, EBCDIC

5 – Session	NetBIOS, RPC, PPTP, SMB (partly session)
4 – Transport	TCP, UDP, SCTP, DCCP
3 – Network	IPv4, IPv6, ICMP, IGMP, ARP (often placed in L2/L3), OSPF, RIP
2 – Data Link	Ethernet (802.3), Wi-Fi (802.11), PPP, HDLC, Frame Relay, Token Ring
1 – Physical	Ethernet (cables, repeaters), DSL, SONET/SDH, Bluetooth radio, USB

2.5 The TCP/IP Model

The TCP/IP model is a practical, simpler model used on the real Internet. It has **4 layers** (sometimes 5, depending on how you count). It was developed by the U.S. Department of Defense (ARPANET).

TCP/IP Layers vs OSI Layers



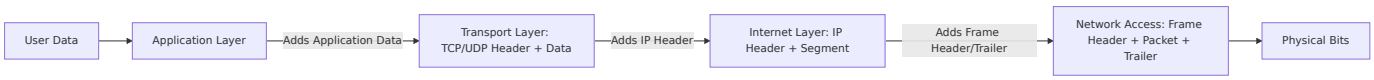
2.5.1 Functions of Each TCP/IP Layer

TCP/IP Layer	Function	Equivalent OSI Layers	Example Protocols
Application	Provides user applications with network services. Combines OSI layers 5,6,7.	Application, Presentation, Session	HTTP, HTTPS, FTP, SMTP, DNS, SSH
Transport	End-to-end reliability, flow control, multiplexing (port numbers).	Transport	TCP, UDP
Internet	Routing packets across networks, logical addressing (IP).	Network	IPv4, IPv6, ICMP, ARP (sometimes)

Network Access (also called Link Layer)	Sending frames over a physical medium, hardware addressing.	Data Link + Physical	Ethernet, Wi-Fi, PPP, DSL

2.5.2 Encapsulation in TCP/IP

The process is similar to OSI, but with only 4 layers:



2.5.3 Comparison Table: OSI vs TCP/IP

Feature	OSI Model	TCP/IP Model
Number of layers	7	4 (or 5 if separating physical)
Developed by	ISO (International Org)	DARPA (U.S. DoD)
Theoretical / Practical	Theoretical (reference model)	Practical (used on the Internet)
Session & Presentation	Separate layers	Included in Application layer
Physical & Data Link	Separate layers	Combined into Network Access layer
Protocol dependencies	Independent of protocols	Built around TCP and IP
Popularity	Used for teaching & documentation	Used in real devices and networks

---

2.6 Why Both Models Matter Today

- **OSI model** helps you **understand** network functions layer by layer. It’ s a great tool for learning, troubleshooting, and designing networks.
- **TCP/IP model** is what actually runs the **Internet**. When you browse a website, send an email, or stream a video, you are using TCP/IP.

When network engineers talk about “Layer 2 switches” or “Layer 3 routers”, they are referring to OSI layers (Data Link and Network). Similarly, “TCP port 80” comes from the Transport layer of TCP/IP.

Chapter Summary

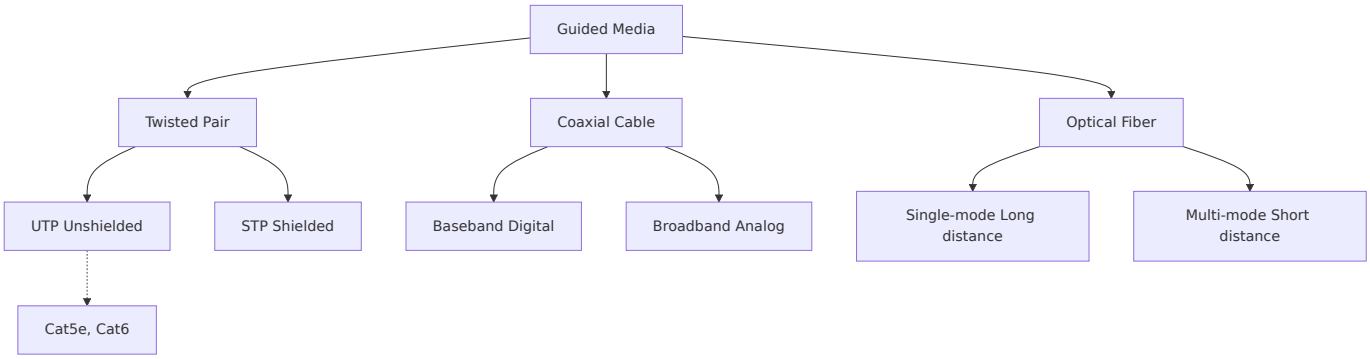
- The **OSI model** has 7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application.
- Each layer has a specific job, and protocols operate at each layer.
- **Encapsulation** adds headers at each layer on the sender side; **decapsulation** removes them at the receiver.
- The **TCP/IP model** has 4 layers: Network Access, Internet, Transport, Application. It’s the real model used on the Internet.
- Both models help us design, understand, and troubleshoot networks.

In the next chapter, we’ll dive into **Physical Layer** details: cables, connectors, signaling, and how bits actually travel.

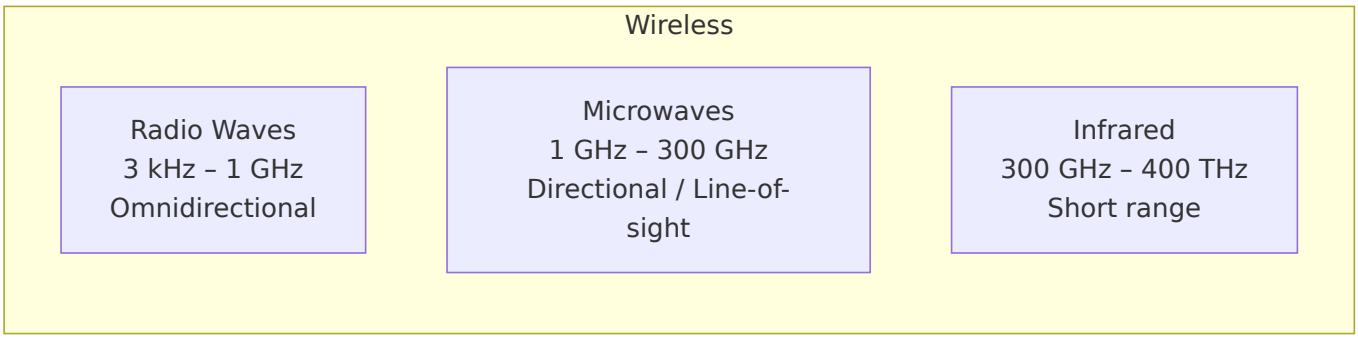
Chapter 3: Physical Layer

1. Transmission Media

1.1 Guided Media (Wired)



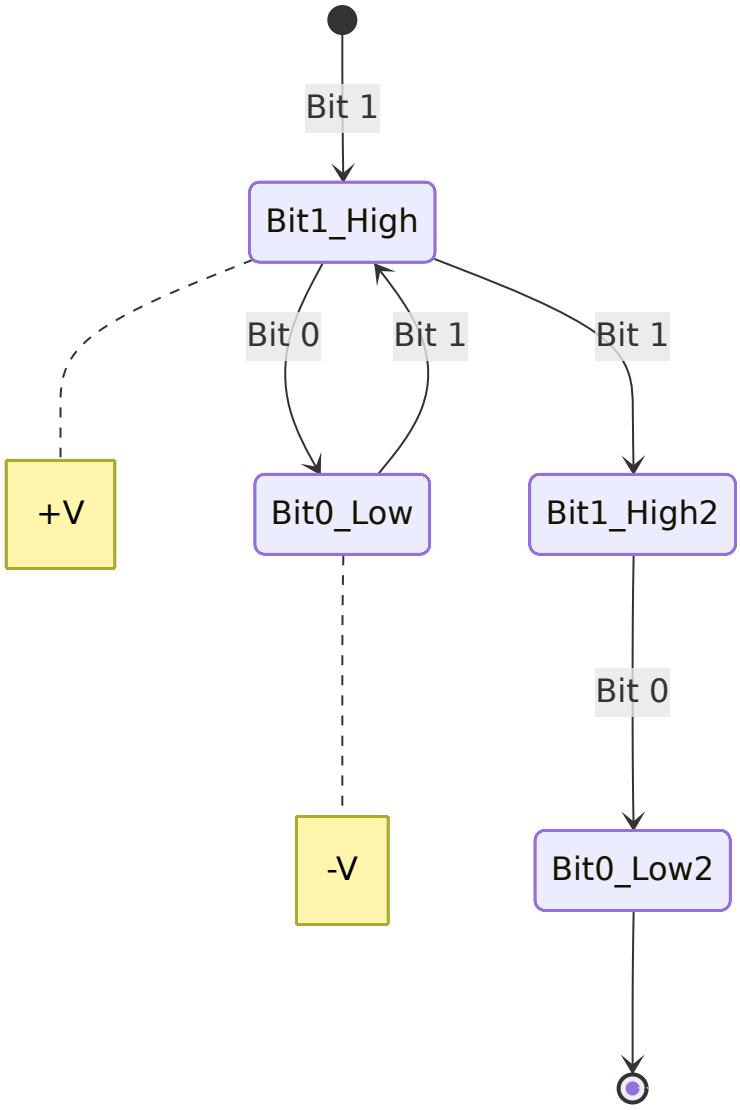
1.2 Unguided Media (Wireless)



---

2. Signal Types: Analog vs Digital

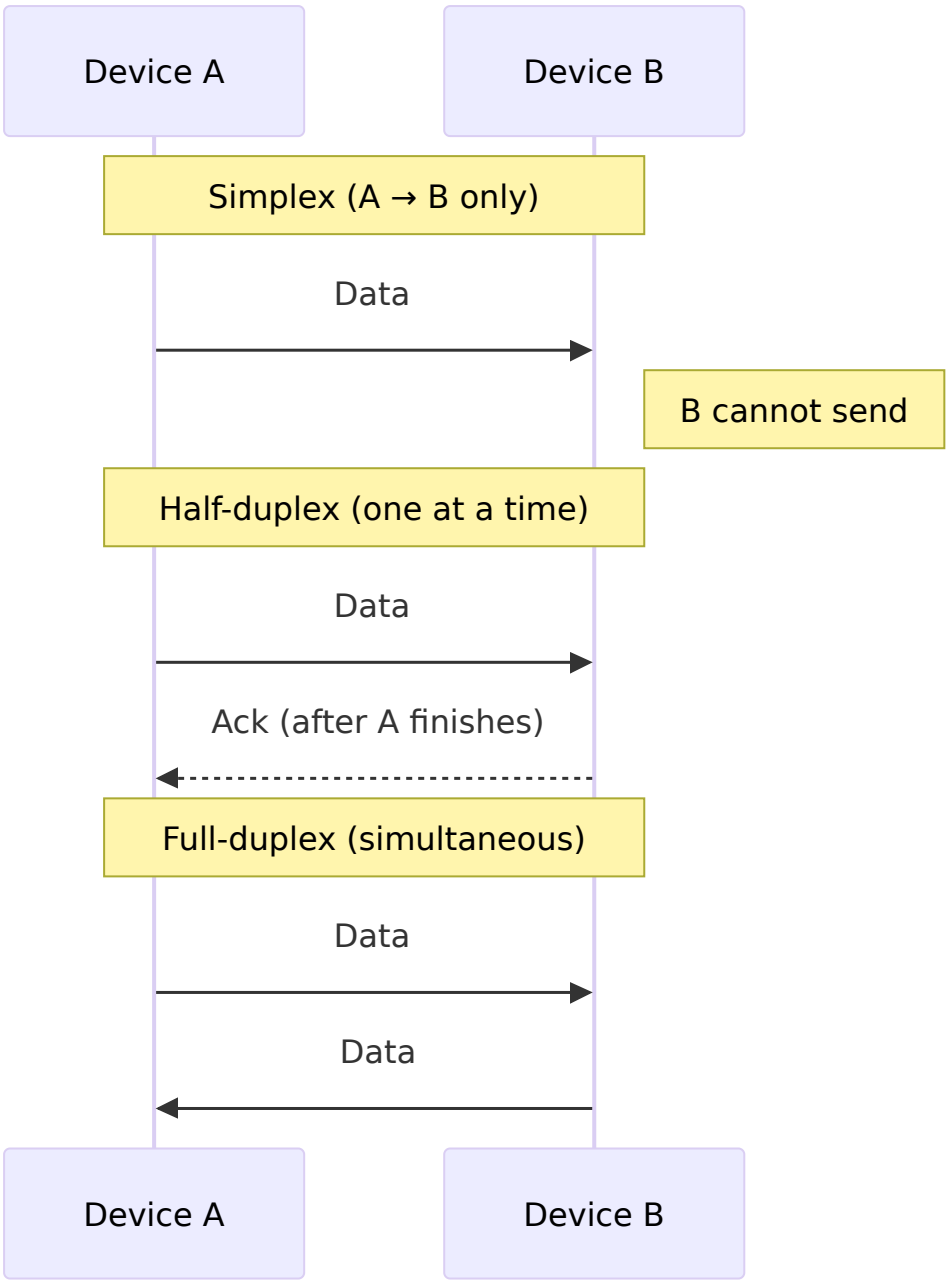
Digital Signal: A digital signal is a discrete signal that represents data using specific values, usually in binary form (0 and 1). It changes in steps rather than continuously. Digital signals are less affected by noise, easier to store, process, and transmit, and are widely used in computers and modern communication systems.
Digital signal representation using state diagram – each state is a voltage level over a bit interval:



Analog Signal: An analog signal is a continuous signal that varies smoothly with time. It can take an infinite number of values within a given range. These signals represent real-world physical quantities such as sound, temperature, and light. However, analog signals are more prone to noise and distortion during transmission.

---

3. Transmission Modes



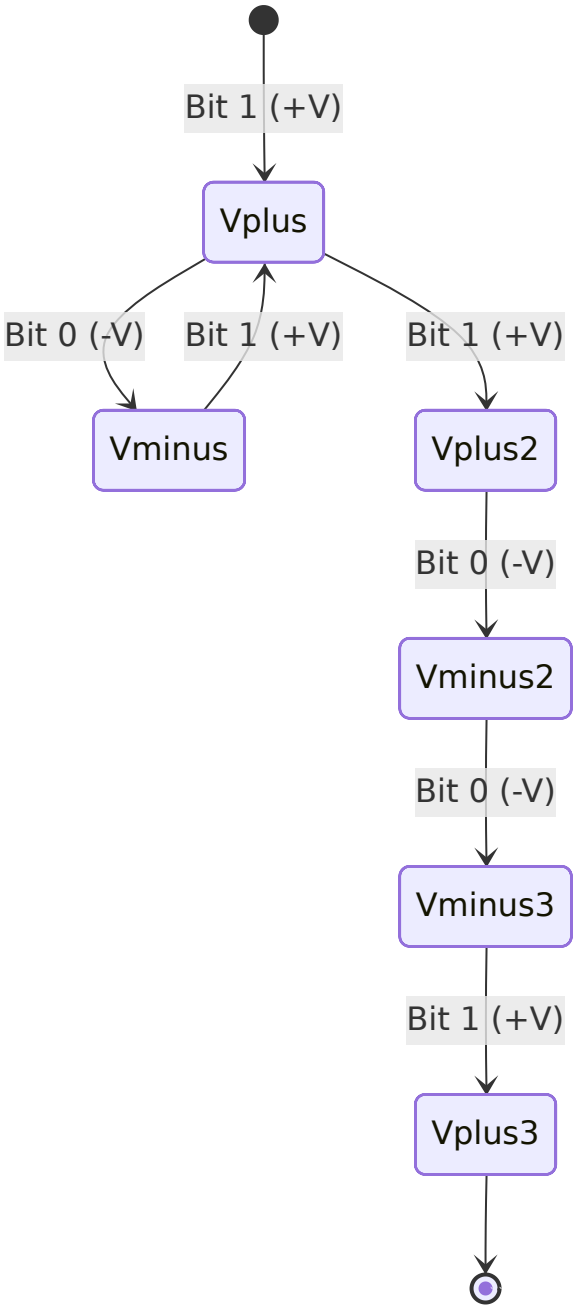
---

4. Line Coding Techniques

Instead of `gantt`, I use **state diagrams** to show voltage levels over time.

4.1 NRZ-L (Non-Return to Zero, Level)

Bits: `1 0 1 1 0 0 1`
Voltage: `+V` for 1, `-V` for 0.

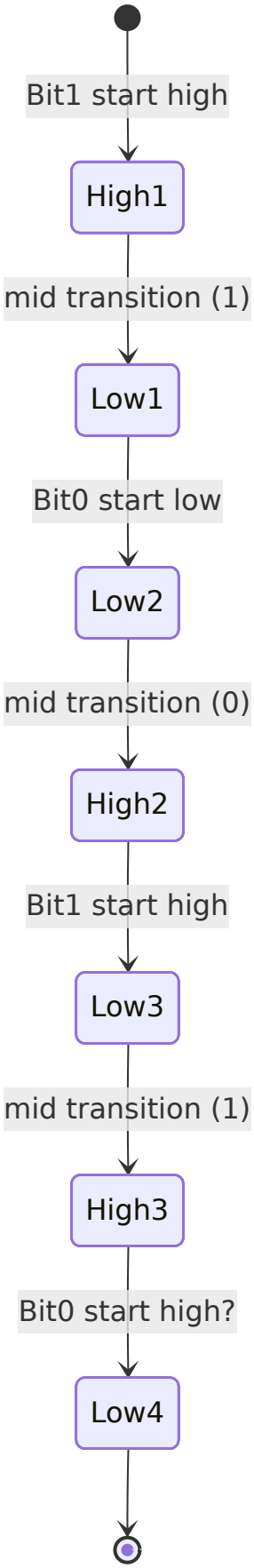


4.2 Manchester Coding (IEEE 802.3)

Each bit has a mid-bit transition:

- `1` = high→low
- `0` = low→high

Bits: `1 0 1 0`



Simpler view – use a table:

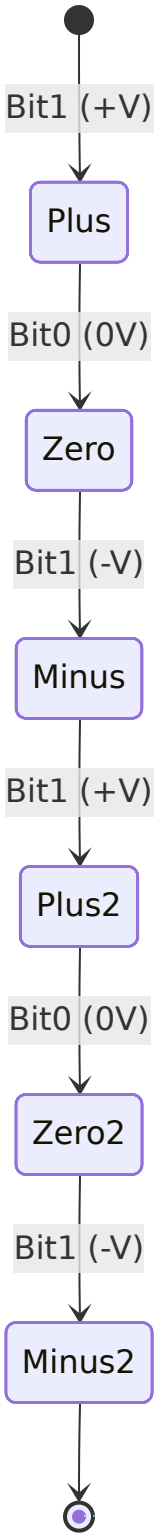
Bit	Start level	Mid transition	End level

1	High	→ Low	Low
0	Low	→ High	High
1	High	→ Low	Low
0	Low	→ High	High

4.3 Bipolar AMI (Alternate Mark Inversion)

- `0` → 0V
- `1` → alternating +V and -V

Bits: `1 0 1 1 0 1`



---

5. Data Rate Concepts

5.1 Nyquist Theorem (Noiseless)

Formula:

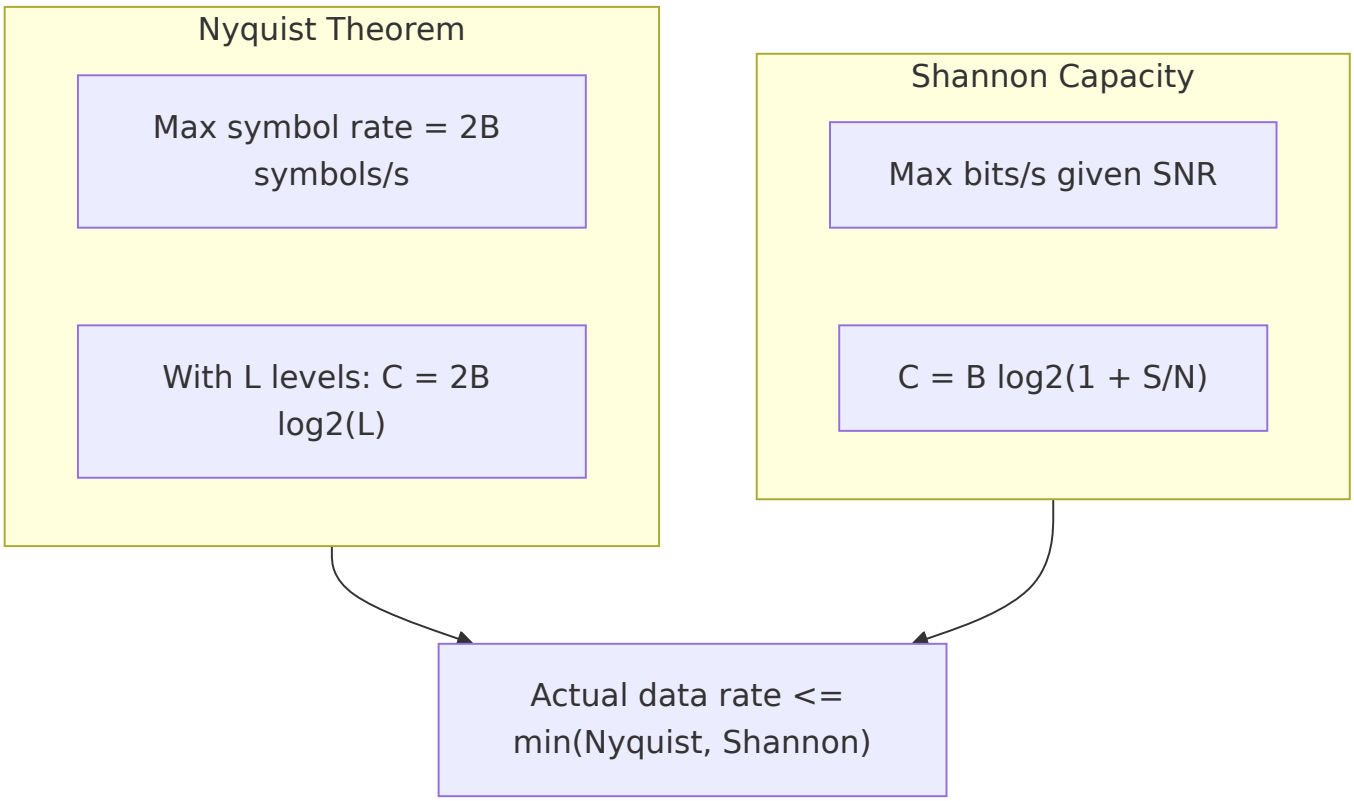
```
C = 2B log2(L)  | B = Bandwidth, L = number of levels,C = Capacity
```

5.2 Shannon Capacity (Noisy)

Formula:

```
C = B log2(1 + S/N) | C = Capacity, B = Bandwidth, S/N = Signal to Noise Ratio
```

Relationship diagram – using a flowchart:



Key insight:

- Nyquist tells how fast you can send symbols (no noise).
- Shannon tells how many bits per symbol you can actually use (with noise).

Example: Telephone line with B = 3.1 kHz, SNR = 30 dB (S/N = 1000)

- Shannon: $C \approx 3100 \times \log_2(1001) \approx 30.9$ kbps
- Nyquist with L=4 (2 bits/symbol): $C = 2 \times 3100 \times 2 = 12.4$ kbps
  → The channel is limited by Nyquist if you only use 4 levels. With better SNR you could increase L, but noise limits L to $\sqrt{(1+S/N)} \approx 31.6$, giving Nyquist $C \approx 2 \times 3100 \times \log_2(31.6) \approx 30.9$ kbps – same as Shannon.

Summary Table

Concept	Formula	Limiting Factor
Nyquist	C = 2B log2(L)	Bandwidth & number of levels
Shannon	C = B log2(1+S/N)	Bandwidth & SNR

Chapter 4: Data Link Layer

The Data Link Layer is Layer 2 of the OSI model. Its job is to transfer data between two directly connected nodes reliably. This chapter covers all its essential functions, error detection techniques, flow and error control protocols, and the MAC sub-layer.

4.1 Functions of the Data Link Layer

The Data Link Layer performs four main tasks:

1. **Framing** – It takes raw bits from the physical layer and groups them into frames (like putting letters into envelopes). Each frame has a header, payload, and trailer.
2. **Error detection and correction** – It checks if bits got flipped during transmission. It can detect errors and sometimes fix them without retransmission.
3. **Flow control** – It prevents a fast sender from overwhelming a slow receiver. The receiver tells the sender when it is ready for more data.
4. **Access control** – When multiple devices share the same medium (like Wi-Fi or Ethernet cable), this decides who gets to transmit at a given time.

Diagram – Data Link Layer in context:

4.2 Error Detection

When data travels, noise can flip bits. These methods help detect errors. If an error is found, the receiver asks the sender to retransmit.

4.2.1 Parity Check

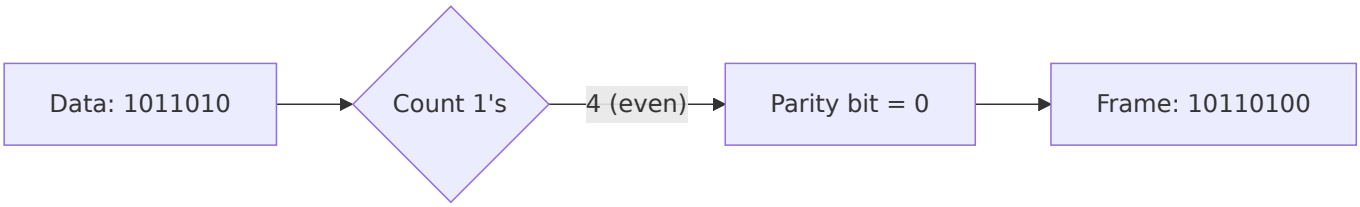
A single extra bit (parity bit) is added to make the total number of 1s even (even parity) or odd (odd parity).

Example: Send `1011010` with even parity.
Count of 1s = 4 (already even) → parity bit = 0 → transmitted frame = `10110100`.

If a single bit flips, the receiver will detect an odd number of 1s and know an error occurred.

Limitation: Cannot detect two bit flips (error cancels out).

Diagram:

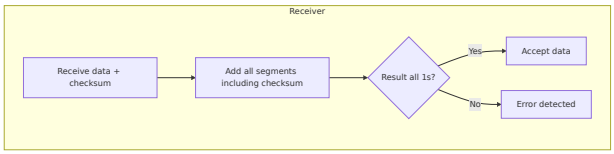


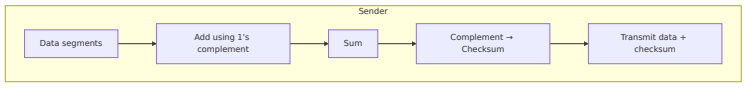
4.2.2 Checksum

Used in protocols like TCP/IP and UDP. The data is divided into fixed-size segments. All segments are added using one’ s complement arithmetic. The final sum is complemented and sent as the checksum.

Example (simplified): Data = `01001100 01101010` (two 8-bit segments).
Add: `01001100 + 01101010 = 10110110`. One’ s complement of the sum = `01001001` (checksum).
Receiver adds all segments including checksum. If the result is all 1s (`11111111`), data is correct.

Diagram:





4.2.3 Cyclic Redundancy Check (CRC)

CRC is the most powerful error detection method used in Ethernet, Wi-Fi, and many storage systems. It treats the data as a large binary number and divides it by a fixed divisor (generator polynomial). The remainder is the CRC code.

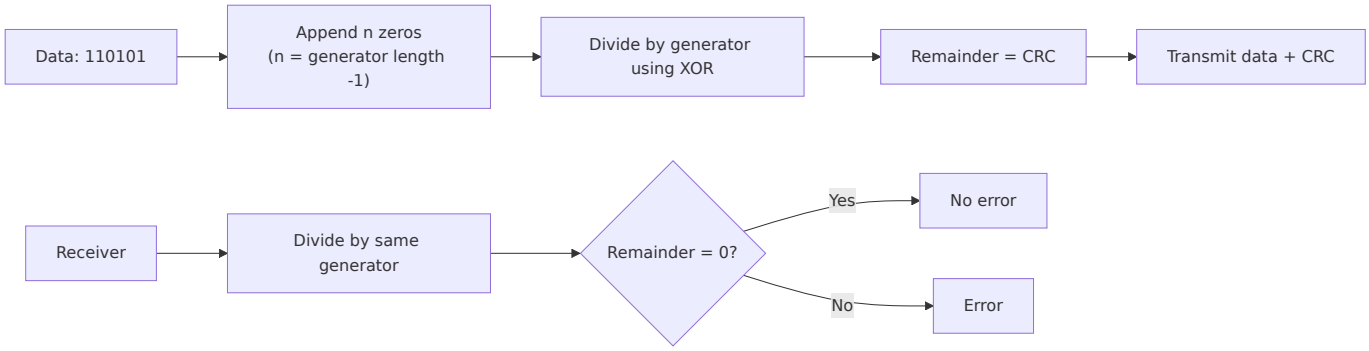
Example: Data = `110101` , Generator = `1011` (4 bits → CRC of 3 bits).
Append 3 zeros to data → `110101000` . Divide by `1011` using XOR. Remainder = `001` → transmitted frame = `110101001` .
Receiver divides the received frame by the same generator. If remainder is zero, no error.

Common generators:

- CRC-16: `10001000000100001` (used in Bluetooth)

- CRC-32: used in Ethernet (32-bit check)

Diagram:



4.2.4 Hamming Code

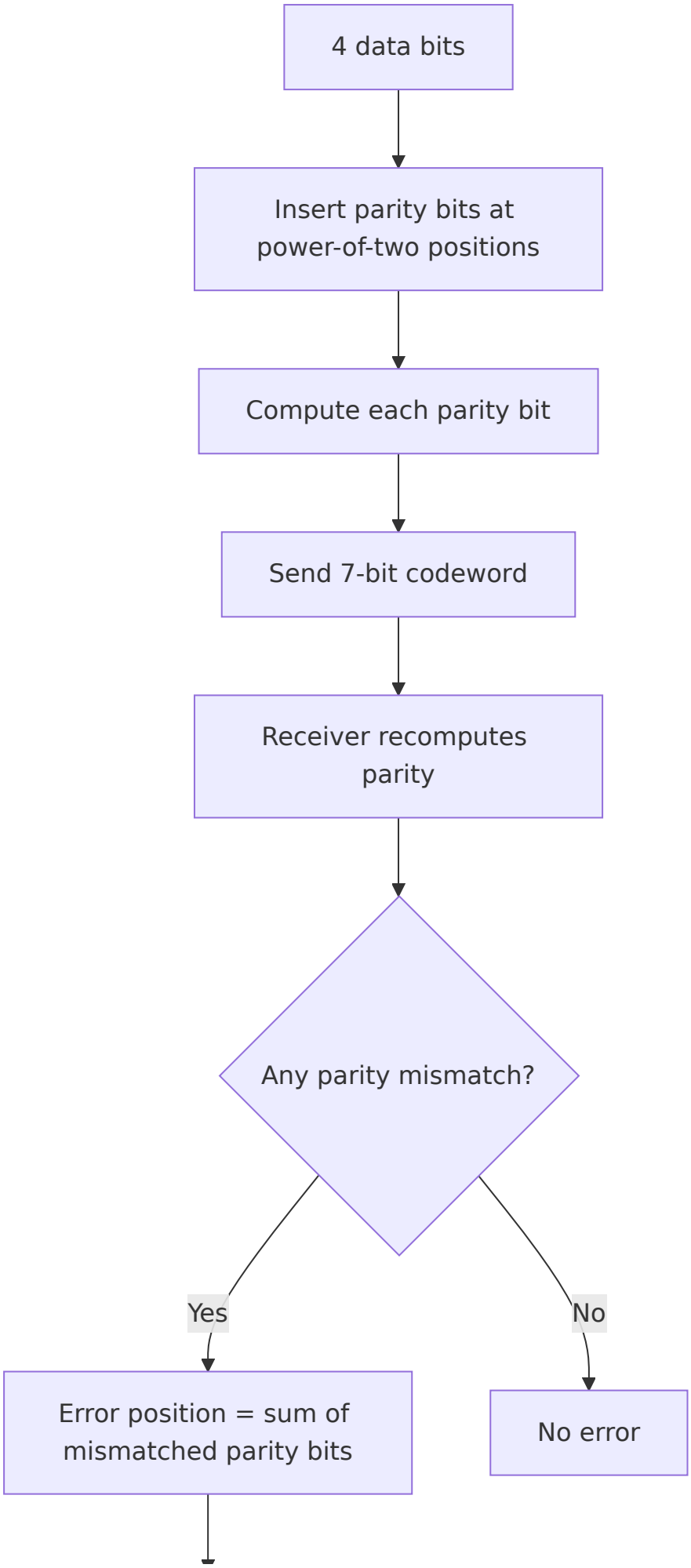
Hamming code can detect and **correct** single-bit errors. It adds parity bits at positions that are powers of two (1, 2, 4, 8, …). Each parity bit checks a specific set of data bits.

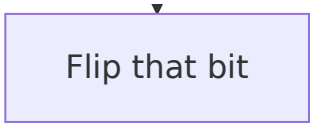
Example: Send 4-bit data `1011` using Hamming(7,4).
Positions: 1(p1),2(p2),3(d1),4(p3),5(d2),6(d3),7(d4) where d1=1, d2=0, d3=1, d4=1.
Compute parity:

- p1 covers positions 1,3,5,7 → 1,1,0,1 → even parity → p1=1
- p2 covers 2,3,6,7 → 1,1,1,1 → even → p2=1
- p3 covers 4,5,6,7 → 1,0,1,1 → odd → p3=0
  Final codeword: `1 1 1 0 0 1 1` (positions 1-7).

If bit 5 flips to 1, receiver recomputes parity and finds the error position by adding the failed parity bit positions. Then it flips the bit back.

Diagram:


4 data bits
Insert parity bits at power-of-two positions
Compute each parity bit
Send 7-bit codeword
Receiver recomputes parity
Any parity mismatch?
Yes
No
Error position = sum of mismatched parity bits
No error



4.3 Flow & Error Control Protocols

These protocols ensure that data is sent at the right speed and errors are handled.

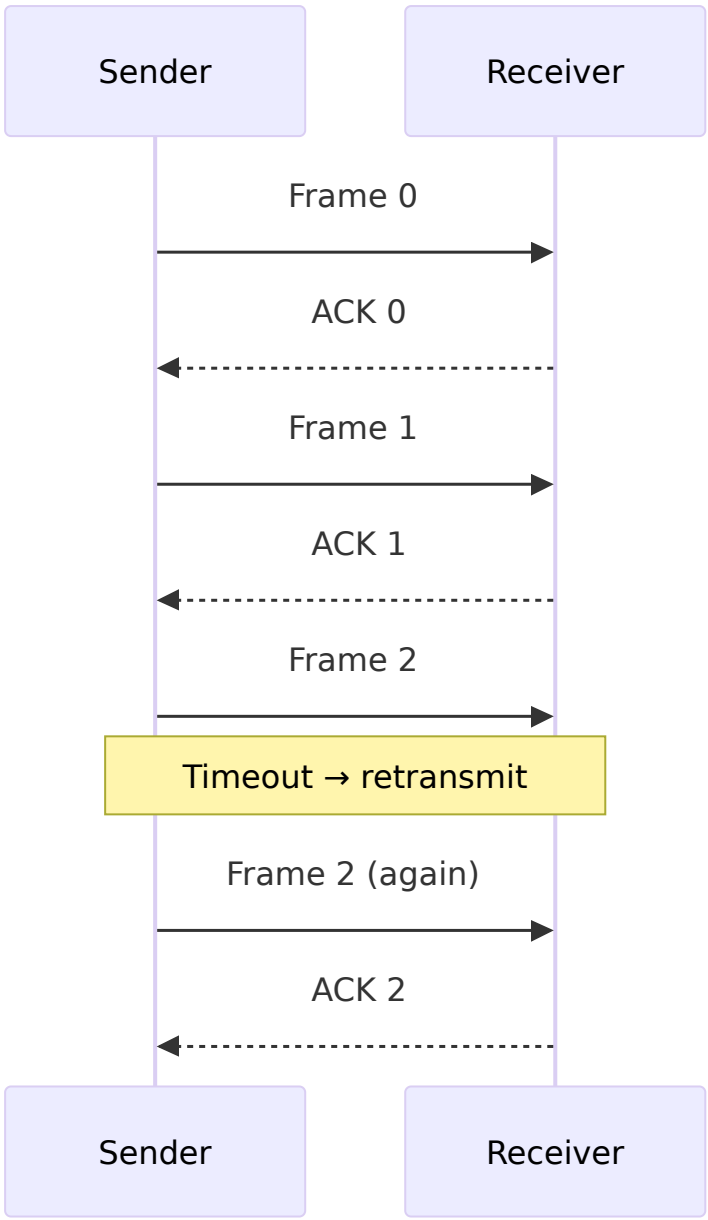
4.3.1 Stop-and-Wait

The sender sends one frame and then waits for an acknowledgment (ACK) from the receiver before sending the next frame. If the ACK does not arrive within a timeout, the sender retransmits.

Example: A sends frame 1 → B receives → B sends ACK → A sends frame 2 → …

Efficiency = (transmission time) / (transmission time + round-trip time). Poor for long-distance links.

Diagram:



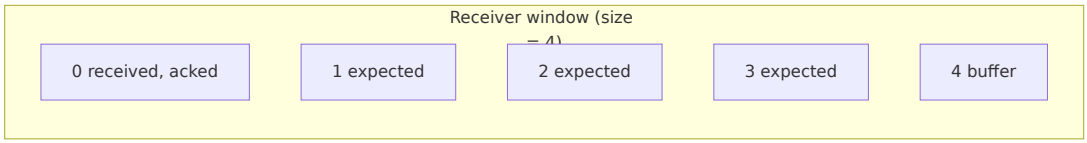
4.3.2 Sliding Window

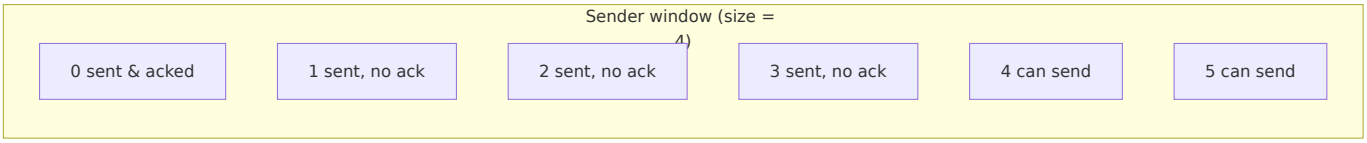
The sender can send multiple frames before waiting for an ACK. Both sides maintain a window of sequence numbers. The window size determines how many outstanding (unacknowledged) frames are allowed.

- **Sender window**: frames sent but not yet acknowledged.
- **Receiver window**: frames that can be accepted out of order.

As ACKs arrive, the window slides forward.

Diagram:





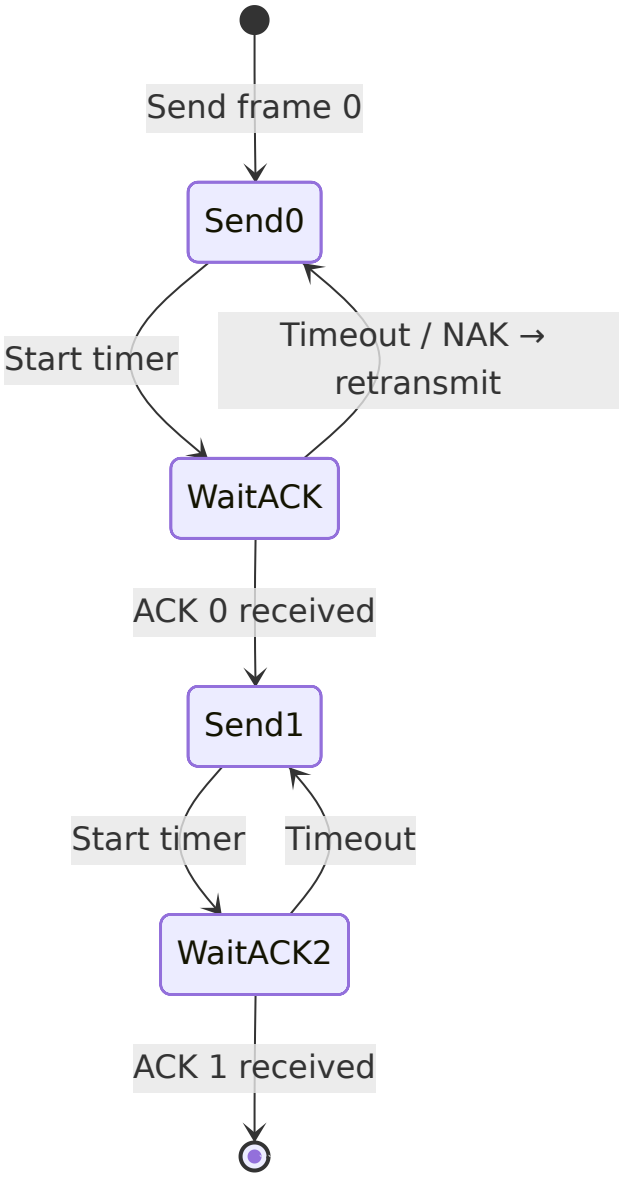
4.3.3 Automatic Repeat Request (ARQ) Protocols

ARQ combines error detection with retransmission.

Stop-and-Wait ARQ

Same as basic Stop-and-Wait, but adds sequence numbers (0 and 1) to distinguish retransmissions from new frames.

Diagram (simplified):

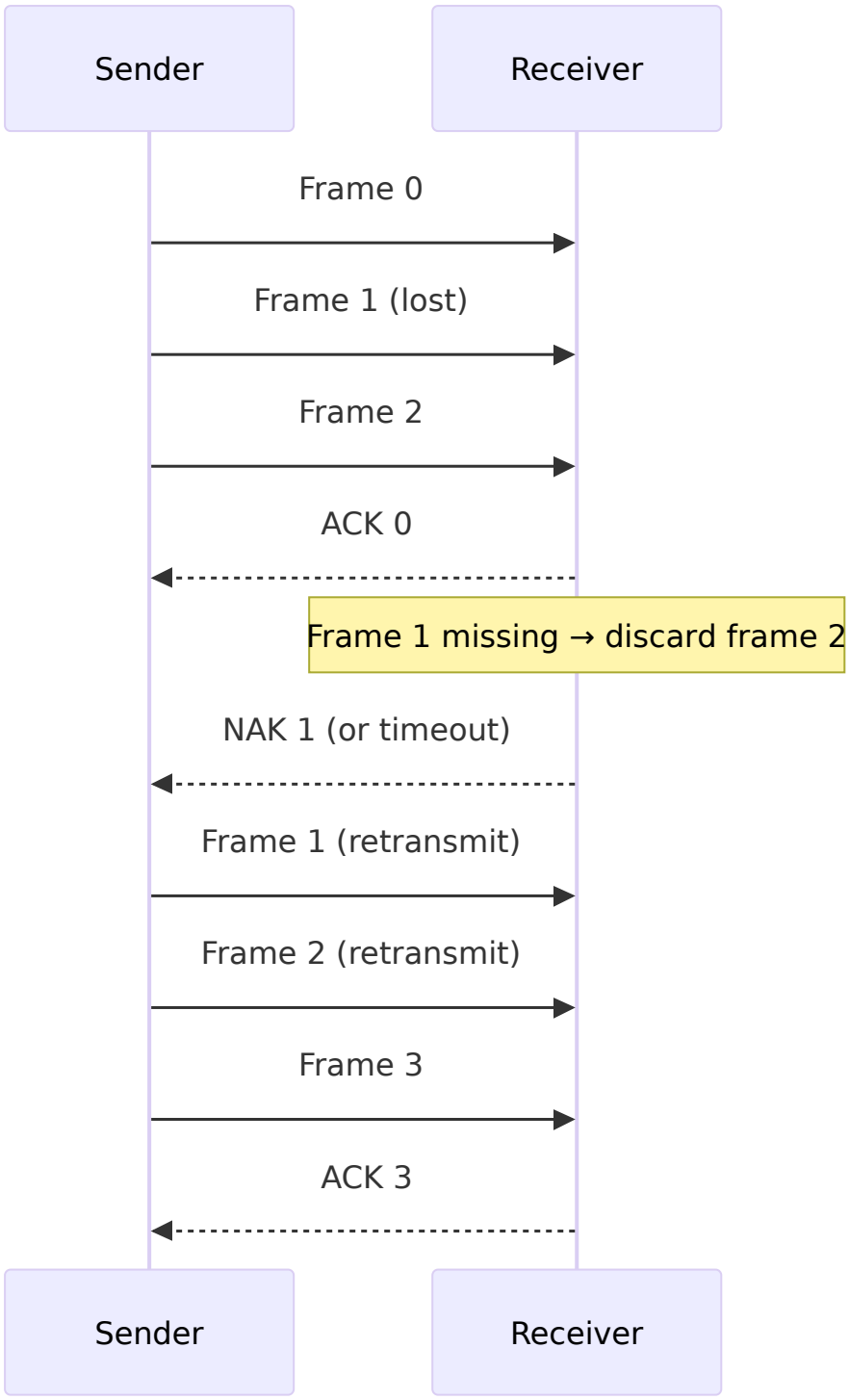


Go-Back-N ARQ

Sender can send up to N frames without ACK. If an error occurs, the receiver discards all subsequent frames, and the sender goes back to the lost frame and retransmits everything from that point.

Example: Window size = 4. Frames 0,1,2,3 sent. Frame 1 lost. Receiver gets frame 2 but discards it (out of order) and keeps asking for frame 1. Sender receives ACK for frame 0 only, then timeout → retransmits frames 1,2,3.

Diagram:

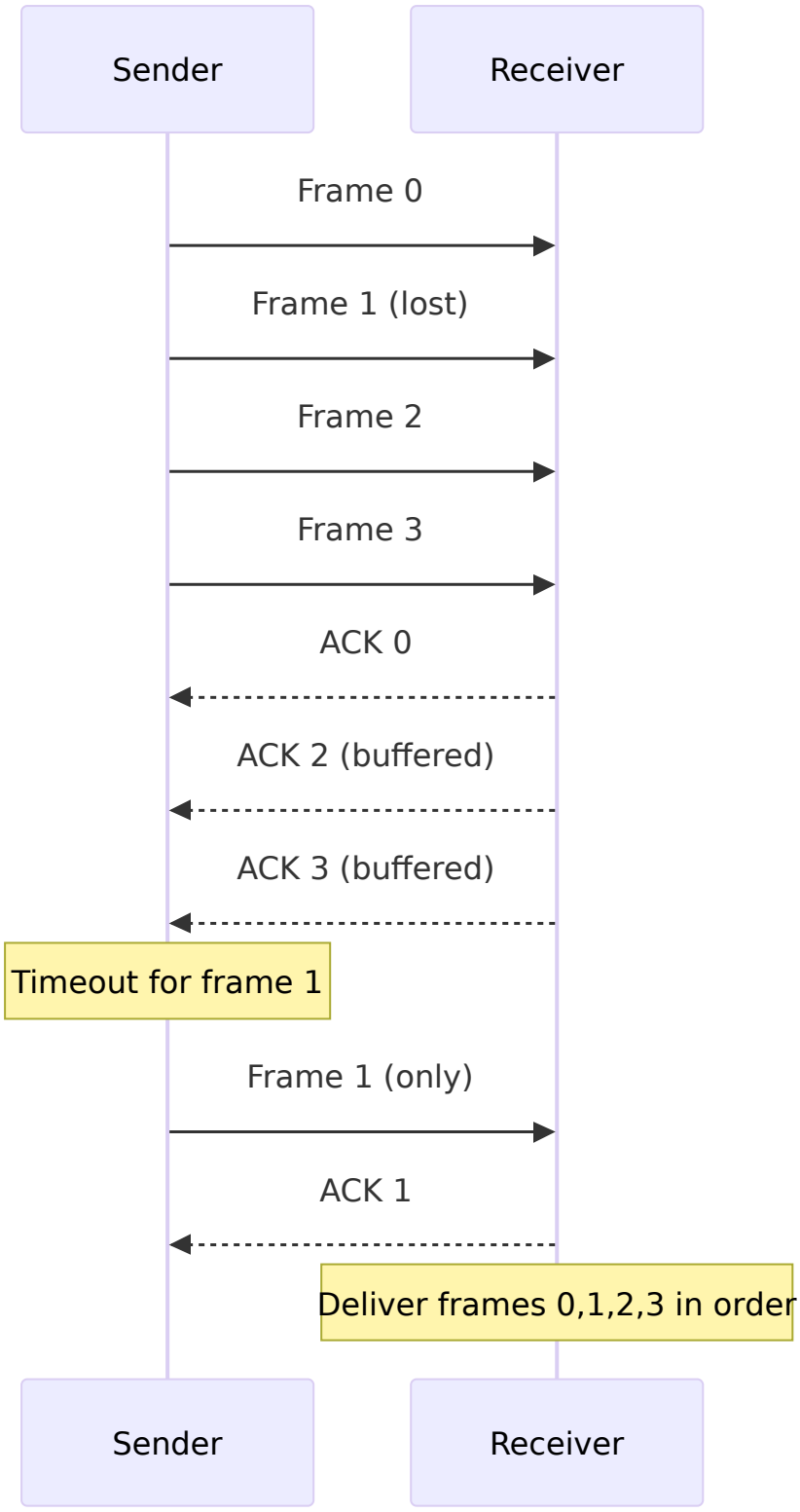


Selective Repeat ARQ

Only the lost frame is retransmitted. The receiver buffers out-of-order frames. Sender retransmits only the missing frame(s). Requires larger receiver buffer and more complex logic.

Example: Frames 0,1,2,3 sent. Frame 1 lost. Receiver buffers frame 2 and 3. Sender retransmits only frame 1. Then receiver delivers all frames in order.

Diagram:



4.4 MAC Sub-layer

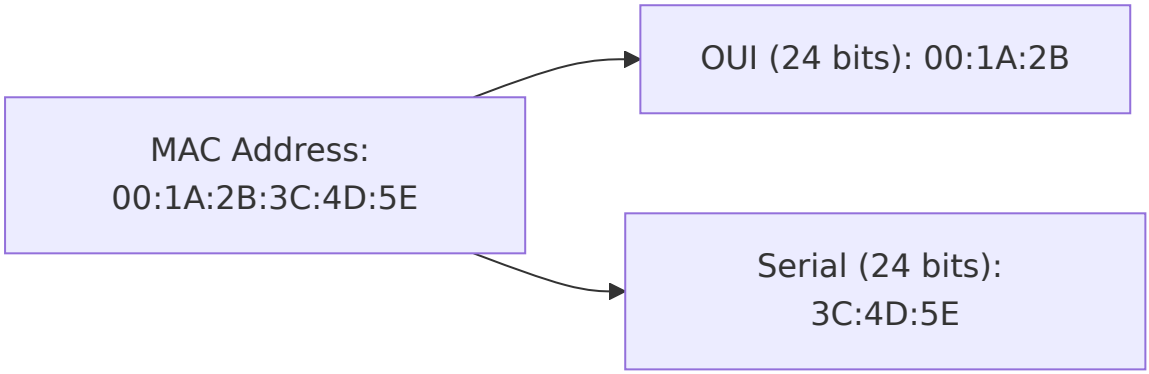
The Media Access Control (MAC) sub-layer is part of the Data Link Layer (the lower half). It handles addressing and channel access.

4.4.1 MAC Addressing

A MAC address is a unique 48-bit (or 64-bit) hardware address burned into network interfaces (NICs). It is written as six hexadecimal pairs, e.g., `00:1A:2B:3C:4D:5E`.

- **Purpose**: Identifies devices on the same local network. IP addresses change as you move; MAC addresses stay the same.
- **Structure**: First 24 bits = OUI (Organizationally Unique Identifier, assigned to manufacturer); last 24 bits = device serial.

Diagram:



4.4.2 Channel Access Protocols

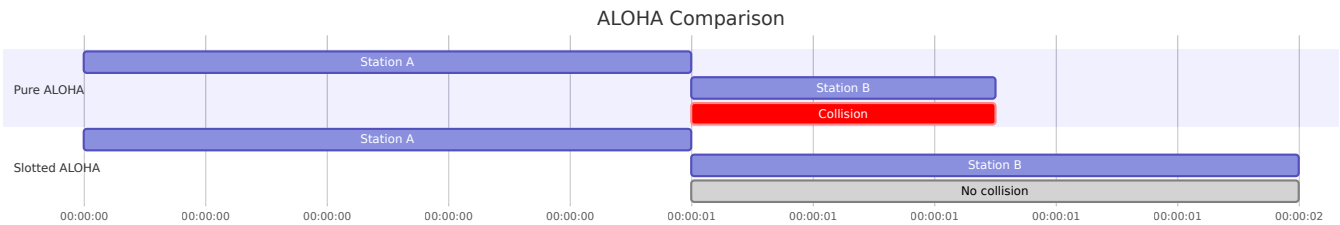
When many devices share the same medium (like a Wi-Fi channel or Ethernet cable), they need rules to avoid collisions.

ALOHA (Pure and Slotted)

Pure ALOHA: Devices transmit whenever they have data. If a collision happens, they wait a random time and retransmit. Maximum efficiency = 18.4%.

Slotted ALOHA: Time is divided into slots. Devices can only transmit at the start of a slot. This reduces collisions and doubles efficiency to 36.8%.

Diagram:

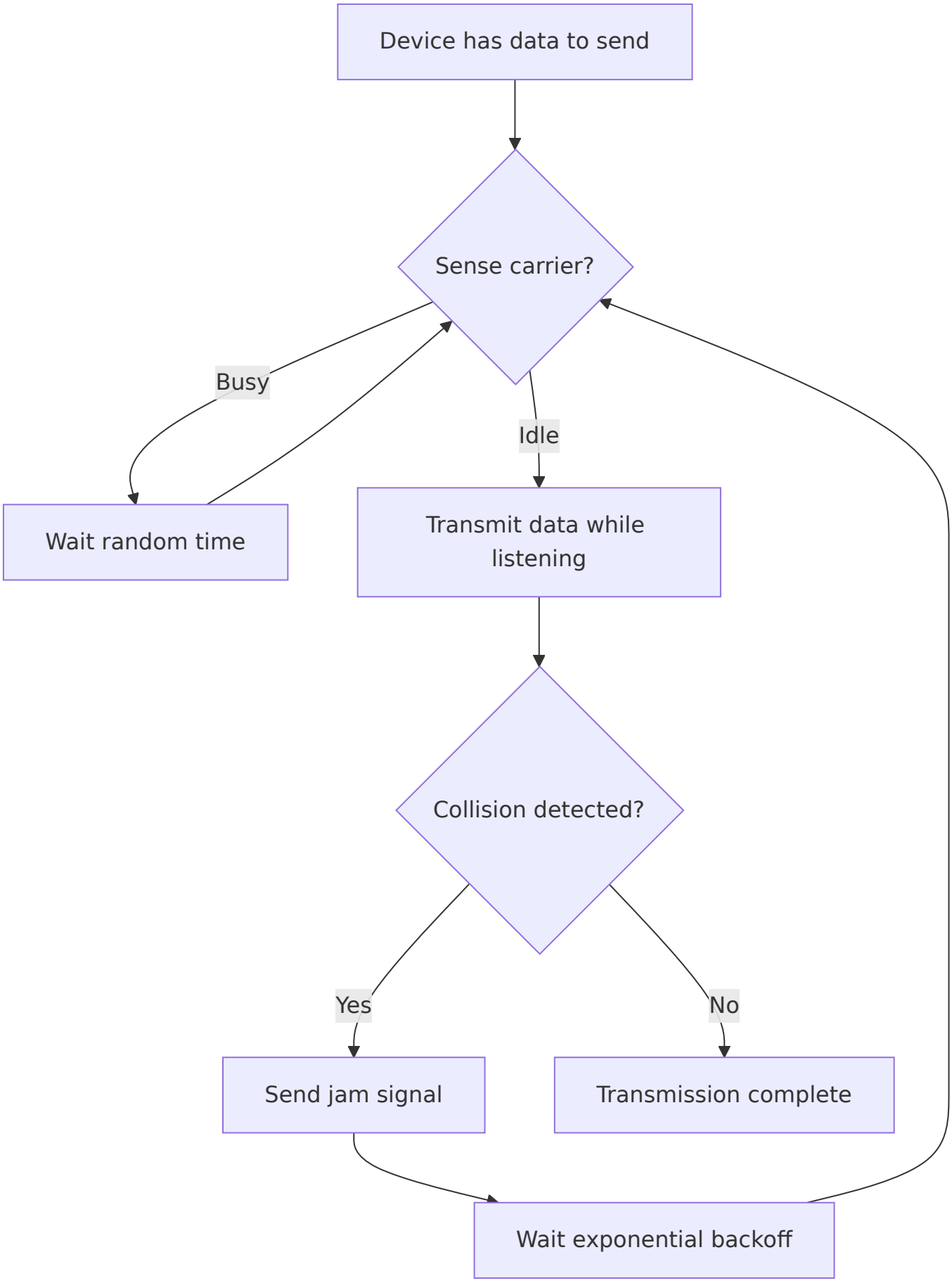


CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

Used in wired Ethernet. Devices **listen** before talking (Carrier Sense). If the channel is idle, they transmit. If a collision is detected (while transmitting), they stop, send a jam signal, and wait a random time before retransmitting.

Example: Two computers on the same Ethernet cable. Both sense idle and transmit at nearly the same time → collision detected → both stop and back off.

Diagram:



CSMA/CA (Collision Avoidance)

Used in wireless networks (Wi-Fi). Collision detection is impossible because a device cannot listen while transmitting (half-duplex radio). So CSMA/CA **avoids** collisions using:

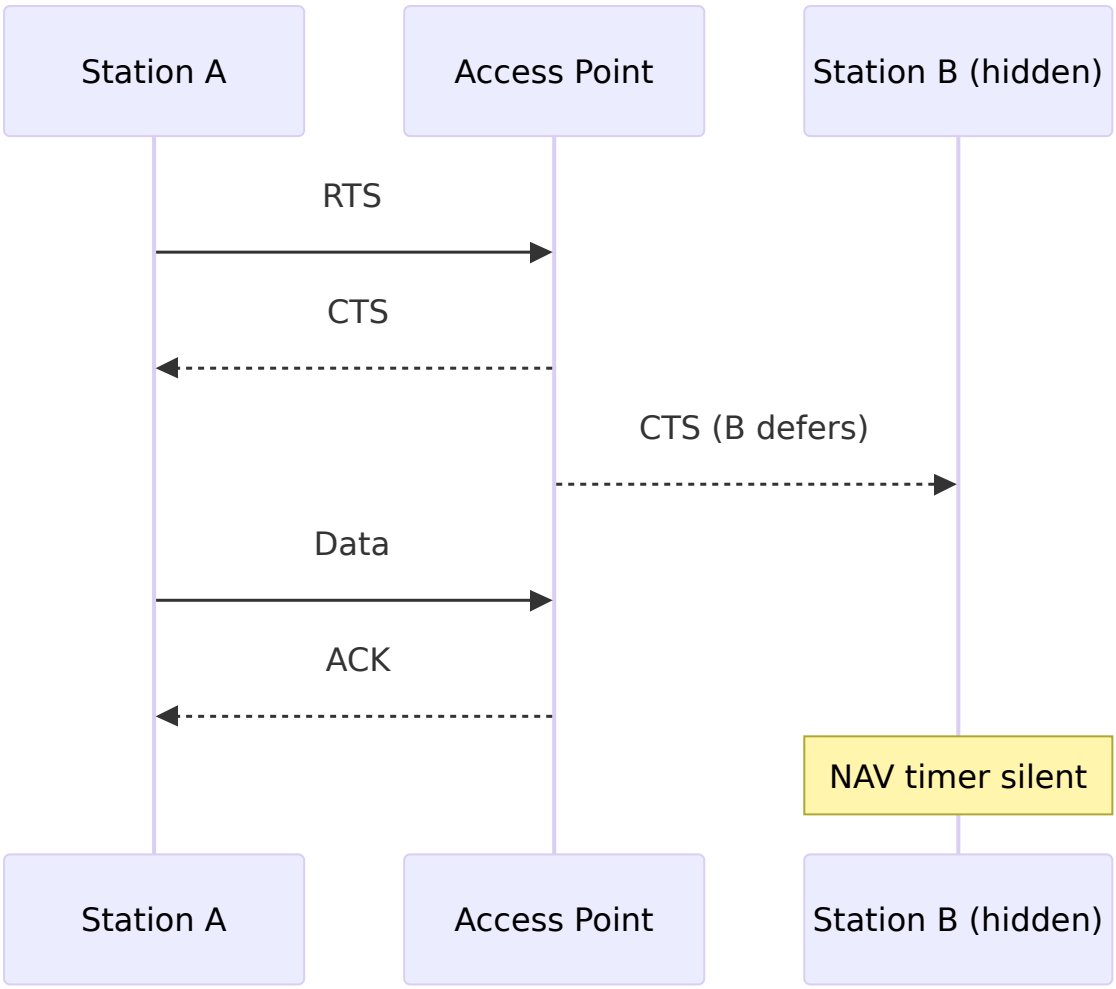
- **Request to Send (RTS) / Clear to Send (CTS)** handshake.
- **Network Allocation Vector (NAV)** – a timer that tells others how long the channel will be busy.

Process:

1. Sender sends RTS to access point (AP).
2. AP replies with CTS, which all nearby devices hear.
3. Other devices set their NAV and stay silent.
4. Sender transmits data.
5. Receiver sends ACK.

Example: Laptop A wants to send to Wi-Fi router. Laptop B is within range of A but not the router (hidden node problem). A sends RTS → router sends CTS (B hears CTS and defers) → A sends data without collision.

Diagram:



4.5 Summary Table

Function / Protocol	**Key Idea**	**Example Use**
Framing	Group bits into frames	Ethernet frame with header/trailer
Parity Check	One extra bit for even/odd count	Simple serial links
Checksum	Sum of segments, complement	TCP/UDP
CRC	Polynomial division	Ethernet, Wi-Fi
Hamming Code	Parity bits at power-of-two positions	Memory error correction
Stop-and-Wait	Send one frame, wait for ACK	Simple modems

Sliding Window	Multiple outstanding frames	TCP, HDLC
Go-Back-N	Retransmit from lost frame onward	Satellite links
Selective Repeat	Retransmit only lost frames	Reliable networks
CSMA/CD	Listen, then transmit; detect collision	Wired Ethernet
CSMA/CA	Avoid collision using RTS/CTS	Wi-Fi

This chapter gives you the foundation to understand how data reliably moves across a single link. The next step is to explore how these protocols work together in real networks like Ethernet and Wi-Fi.

Chapter 5: Network Layer

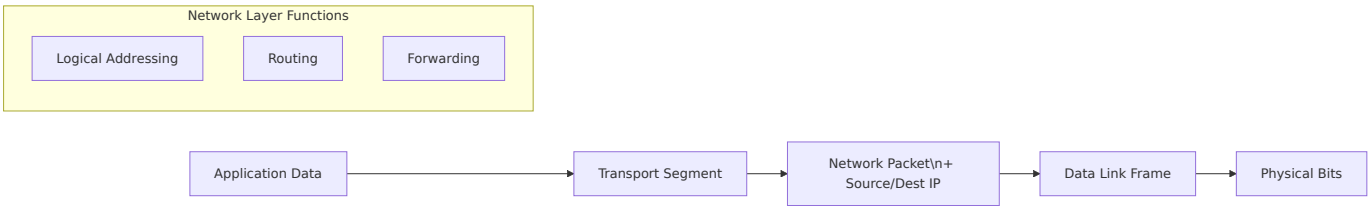
The network layer (Layer 3) is responsible for end-to-end communication across multiple networks. Its main functions are:

- **Logical addressing** – Assigning unique identifiers (IP addresses) to devices.
- **Routing** – Determining the best path from source to destination.
- **Packet forwarding** – Moving packets from an incoming interface to the correct outgoing interface.

Below we explore each topic in depth with practical examples and Mermaid diagrams.

---

1. Functions of the Network Layer



- **Logical addressing** – Every host gets an IP address (e.g., 192.168.1.10). This address is used to identify the source and destination.
- **Routing** – Routers exchange information about network topologies and build routing tables.
- **Forwarding** – When a packet arrives, the router looks up the destination IP in its table and sends it to the next hop.

---

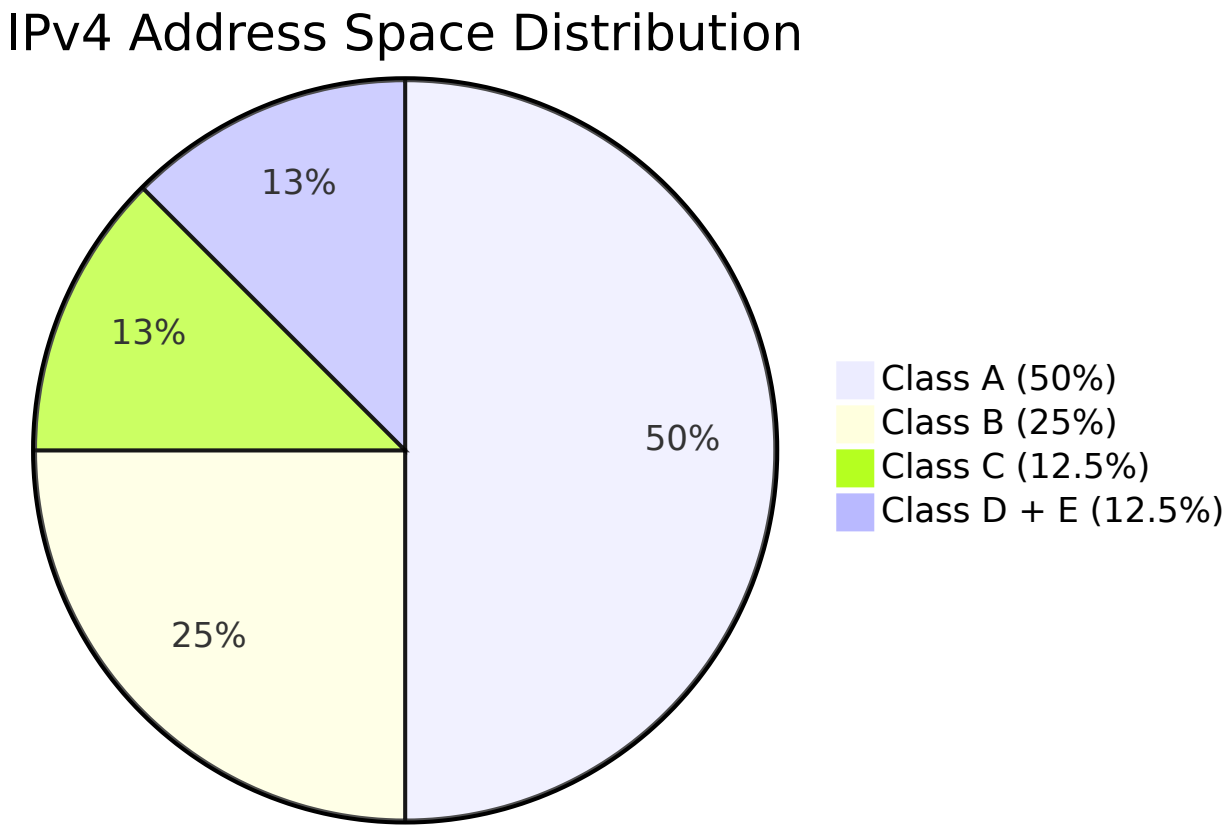
2. IP Addressing

2.1 IPv4 Address Classes

IPv4 addresses are 32 bits long, usually written as four decimal octets. Originally, addresses were divided into **classes**:

Class	Leading Bits	Range (First Octet)	Default Mask	Purpose
A	0	1 – 126	255.0.0.0	Large networks

B	10	128 – 191	255.255.0.0	Medium networks
C	110	192 – 223	255.255.255.0	Small networks
D	1110	224 – 239	None	Multicast
E	1111	240 – 255	None	Experimental



Example:

- 10.0.0.1 is a Class A address (network: 10.0.0.0, host: 0.0.0.1).
- 172.16.0.1 is Class B (network: 172.16.0.0).
- 192.168.1.1 is Class C (network: 192.168.1.0).

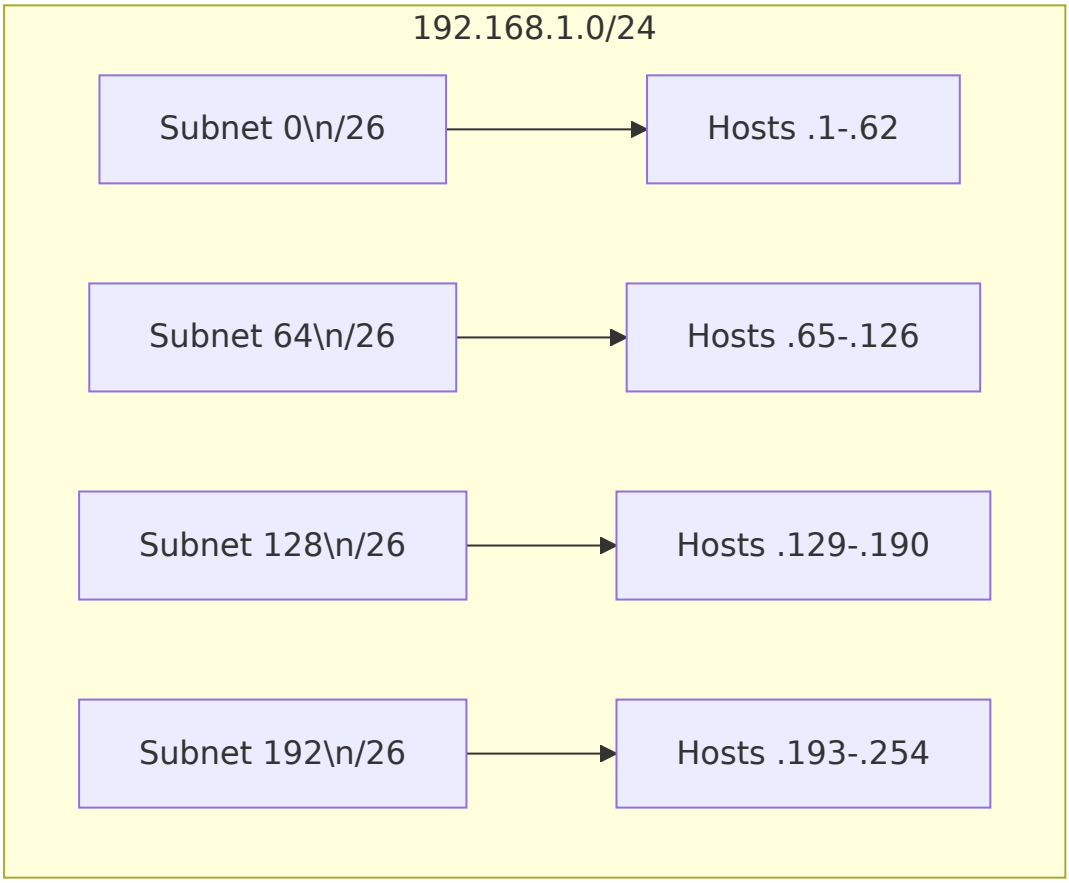
2.2 Subnetting

Subnetting divides a large network into smaller, manageable subnets. It borrows host bits for the network portion.

Example:
Given `192.168.1.0/24` (Class C, 256 addresses). We need 4 subnets with at least 50 hosts each.

- Borrow 2 bits from the host part → new subnet mask = `/26` (255.255.255.192).
- Number of subnets = $2^2 = 4$.
- Hosts per subnet = $2^6 - 2 = 62$.

Subnet	Network Address	Usable Range	Broadcast
1	192.168.1.0	192.168.1.1 – 62	192.168.1.63
2	192.168.1.64	192.168.1.65 – 126	192.168.1.127
3	192.168.1.128	192.168.1.129 – 190	192.168.1.191
4	192.168.1.192	192.168.1.193 – 254	192.168.1.255

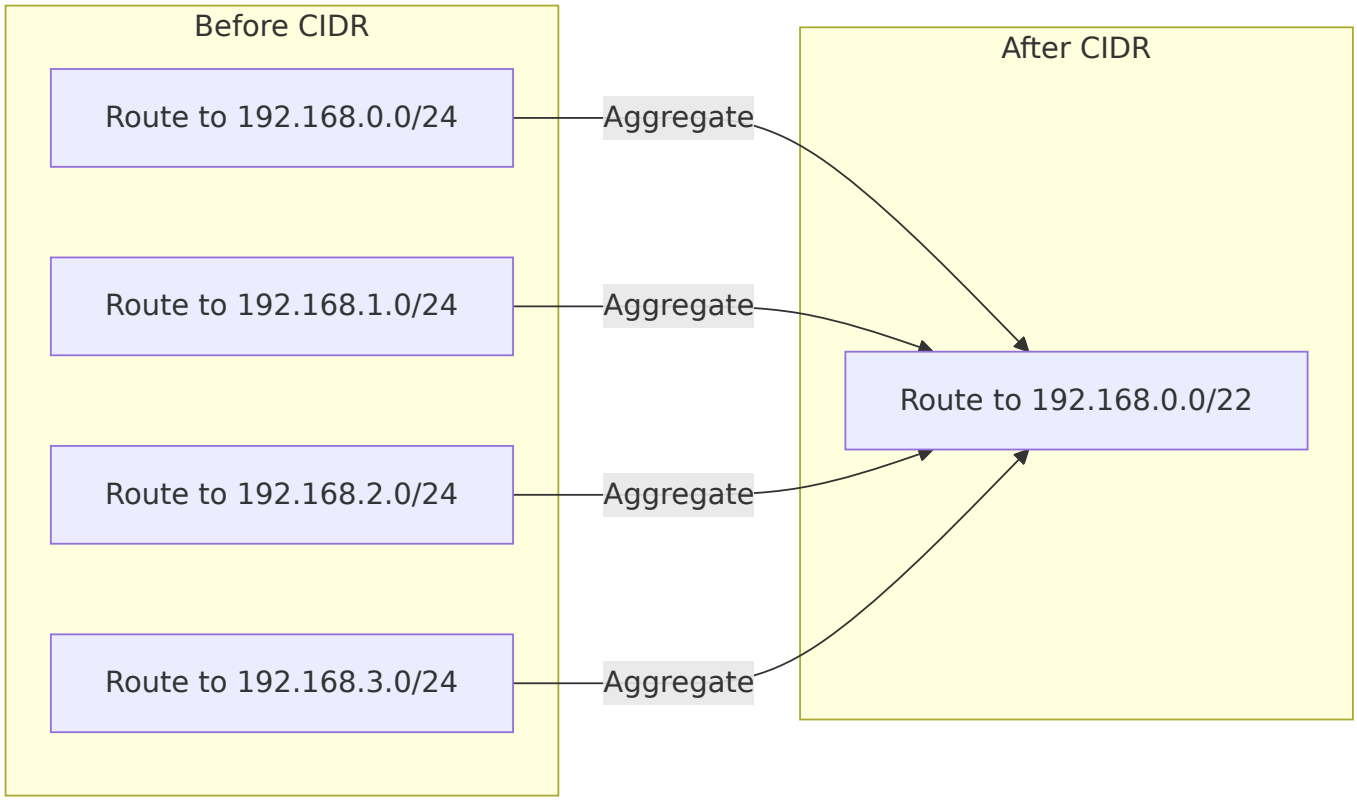


2.3 CIDR (Classless Inter-Domain Routing)

CIDR removes class boundaries. An IP address is written with a prefix length (e.g., `/20`). It allows **supernetting** – aggregating multiple networks into one route.

Example:
Combine `192.168.0.0/24`, `192.168.1.0/24`, `192.168.2.0/24`, `192.168.3.0/24` into `192.168.0.0/22` (because the first 22 bits are common).

- This reduces routing table entries.



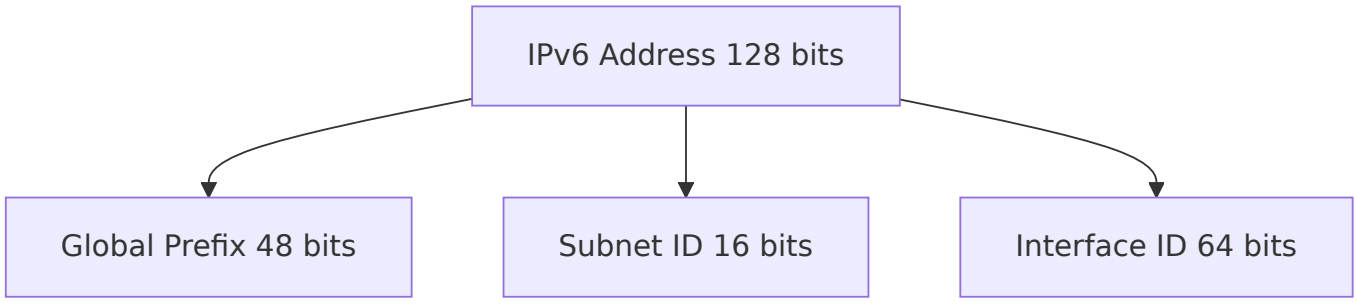
2.4 IPv6 Basics

IPv6 uses 128-bit addresses (written as 8 groups of 4 hex digits). Main features:

- **No broadcast** – uses multicast and anycast.
- **Auto-configuration** (SLAAC).
- **Simplified header** (no checksum, no fragmentation at routers).
- **Example**: `2001:0db8:85a3:0000:0000:8a2e:0370:7334`

IPv6 address types:

- **Unicast** (single interface)
- **Multicast** (one-to-many)
- **Anycast** (one-to-nearest)

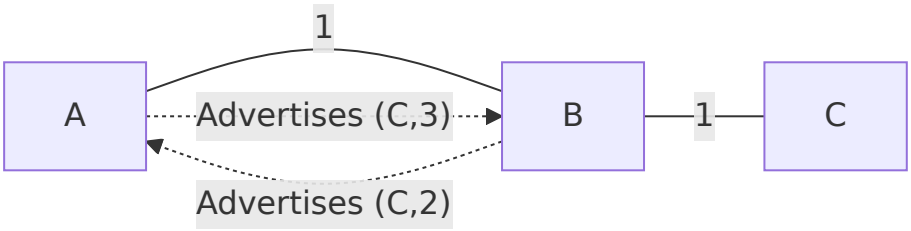


3. Routing Algorithms

3.1 Distance Vector (Bellman-Ford)

Each router shares its entire routing table with **neighbors** periodically. Metrics like hop count are used.
Problem: Slow convergence, count-to-infinity.

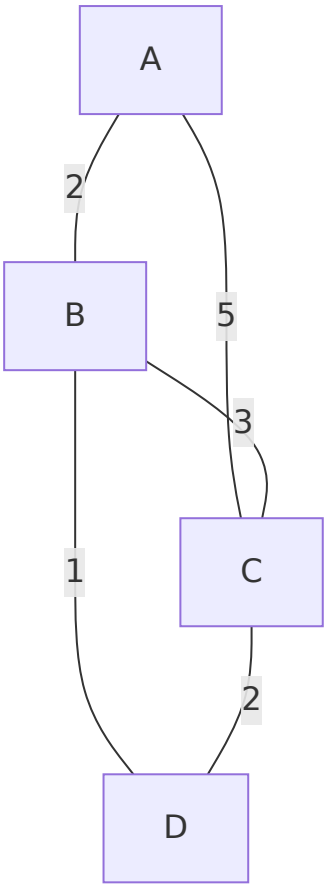
Example: Three routers A–B–C. Link B–C fails. A thinks it can reach C via B (cost 3), B thinks via A (cost 4) – loop until infinity (set to 16).



3.2 Link State (Dijkstra)

Every router learns the **full topology** and computes shortest paths using Dijkstra. Each router floods Link State Advertisements (LSAs) to all others.
Advantage: Fast convergence, no loops.

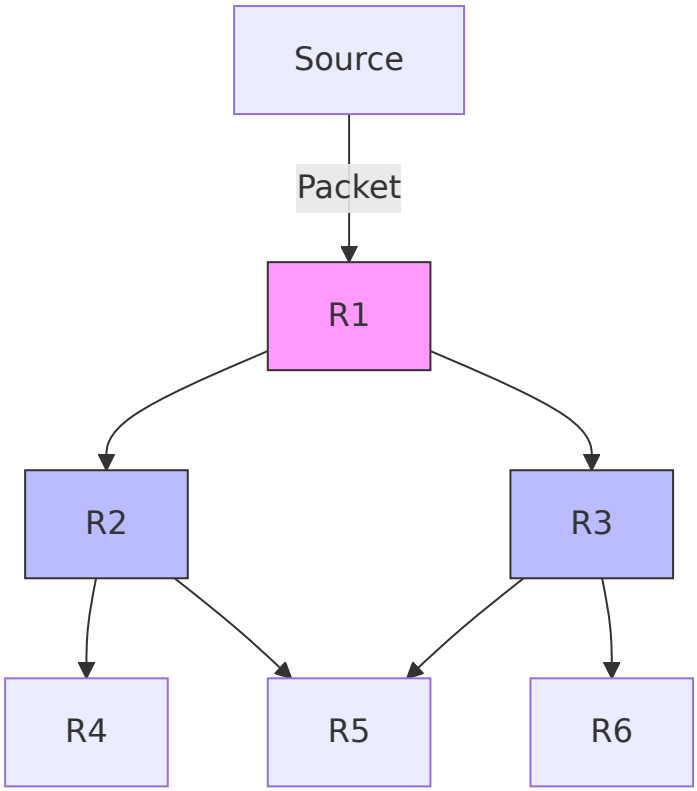
Example network:



From A, shortest path to D is A–B–D (cost 3). Dijkstra builds a shortest path tree.

3.3 Flooding

A router forwards an incoming packet to **every interface except the one it arrived on**. Used in military, OSPF initial LSA propagation, or for robustness.
Control mechanisms: hop count limit, sequence numbers.

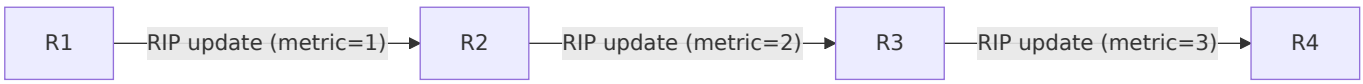


---

4. Routing Protocols

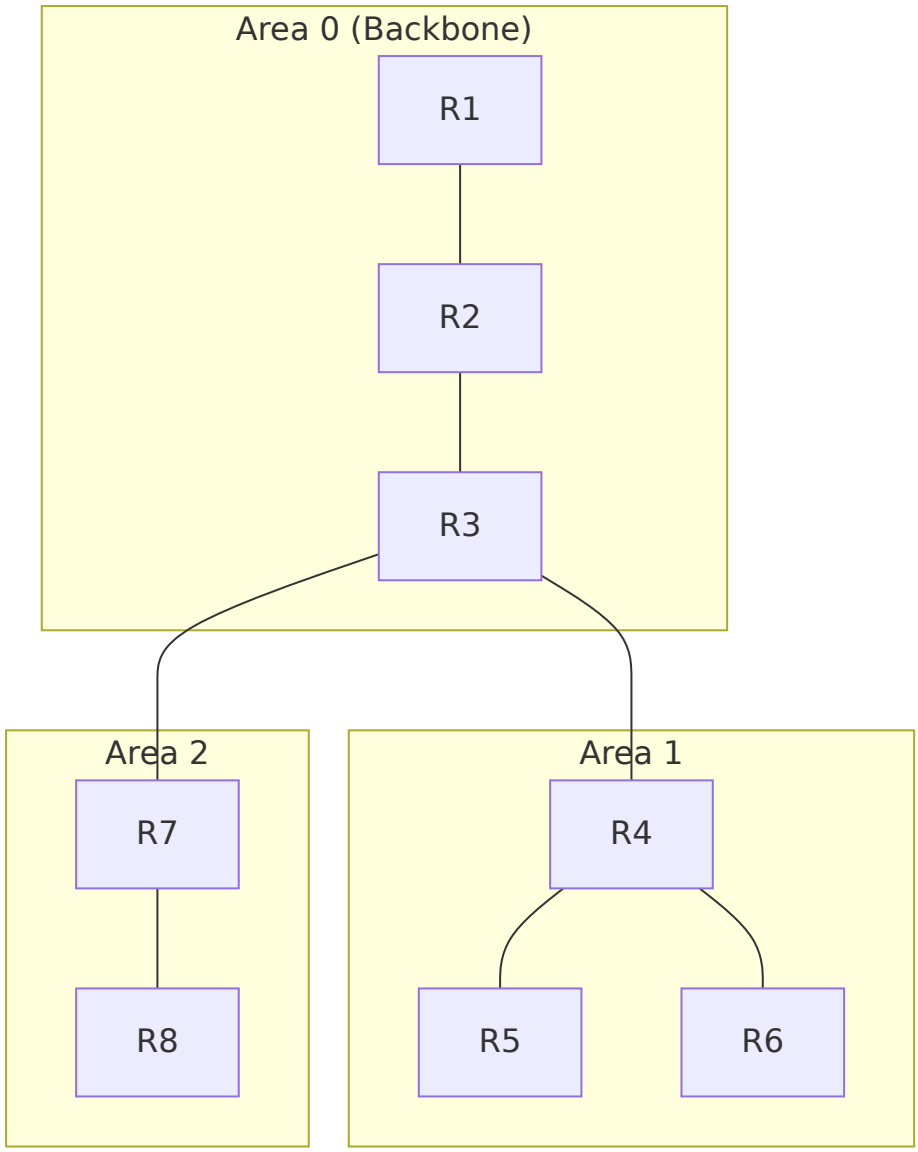
4.1 RIP (Routing Information Protocol)

- Distance-vector, uses **hop count** as metric (max 15 hops).
- Updates every 30 seconds.
- **Example**: RIP route entry: `192.168.5.0/24 via 10.0.0.2, metric 2`.



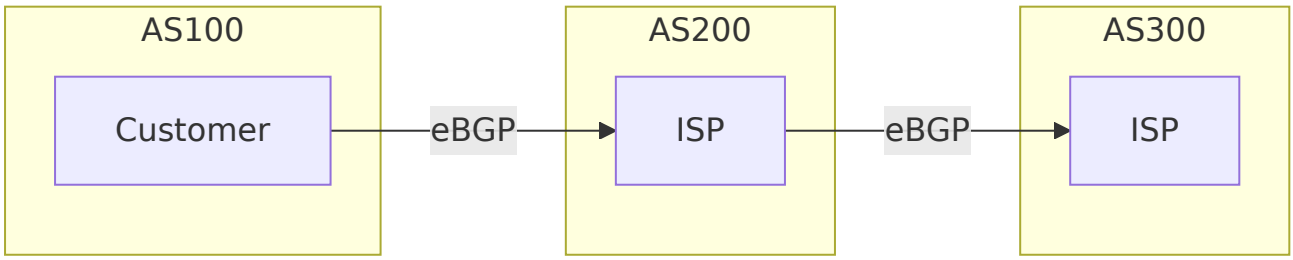
4.2 OSPF (Open Shortest Path First)

- Link-state protocol, uses **cost** (based on bandwidth).
- Hierarchical design: **areas** connected to a backbone (Area 0).
- Faster convergence, supports VLSM/CIDR.



4.3 BGP (Border Gateway Protocol) – Basic Idea

- **Path-vector** protocol used between Autonomous Systems (AS).
- Routes include **AS_PATH** attribute to prevent loops.
- Example: AS 100 advertises `200.1.0.0/16` with path `[100]`. AS 200 receives it, prepends its own AS: `[200, 100]`.



---

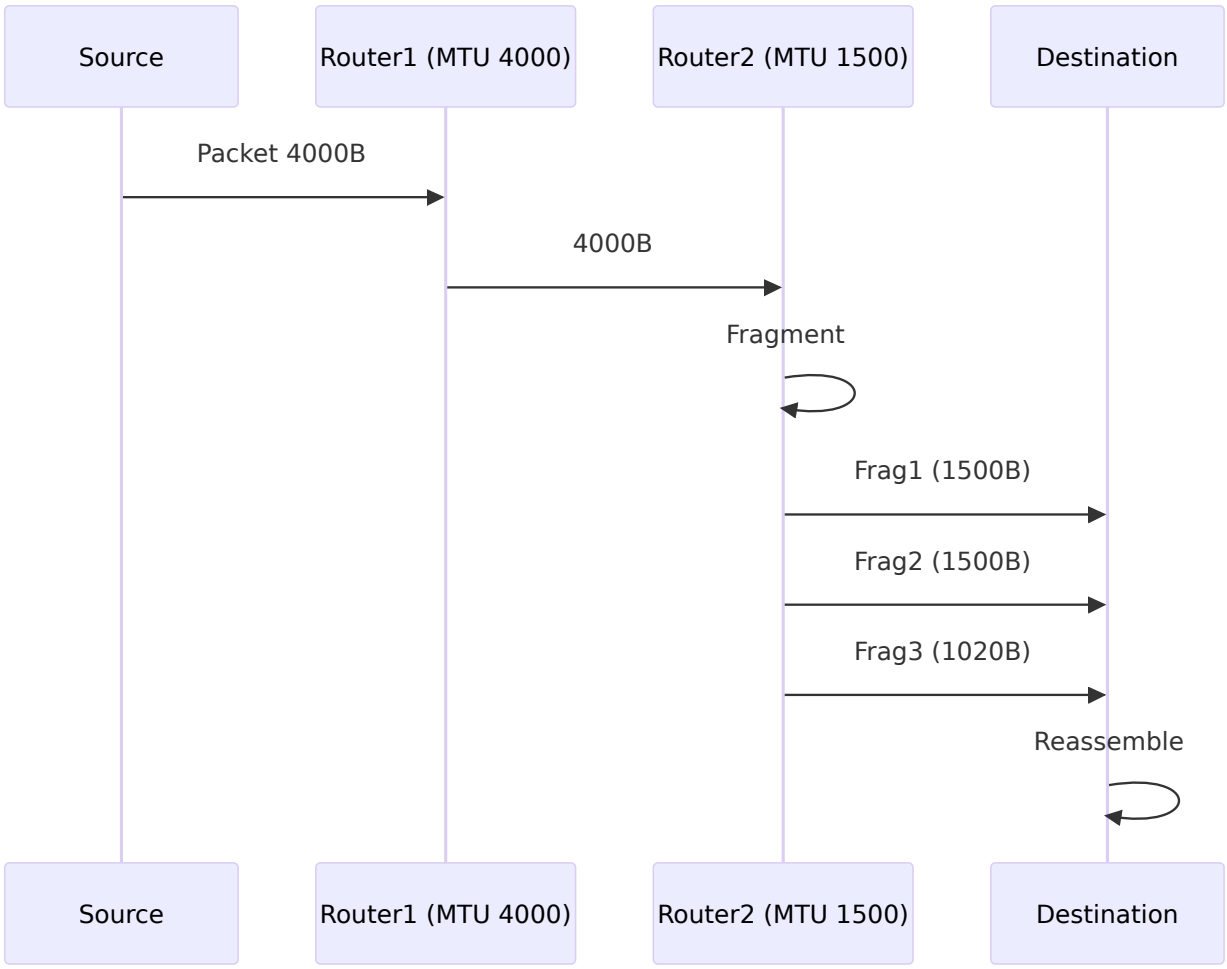
5. Additional Topics

5.1 Fragmentation

When a packet is larger than the MTU (Maximum Transmission Unit) of a link, the router fragments it. IPv4 fragments are reassembled at the **destination**; IPv6 does not allow in-path fragmentation.

Example:
MTU = 1500 bytes, original packet = 4000 bytes (20 header + 3980 data).
Three fragments:

- Fragment 1: offset 0, length 1500 (20+1480 data)
- Fragment 2: offset 1480, length 1500 (20+1480 data)
- Fragment 3: offset 2960, length 1020 (20+1000 data)

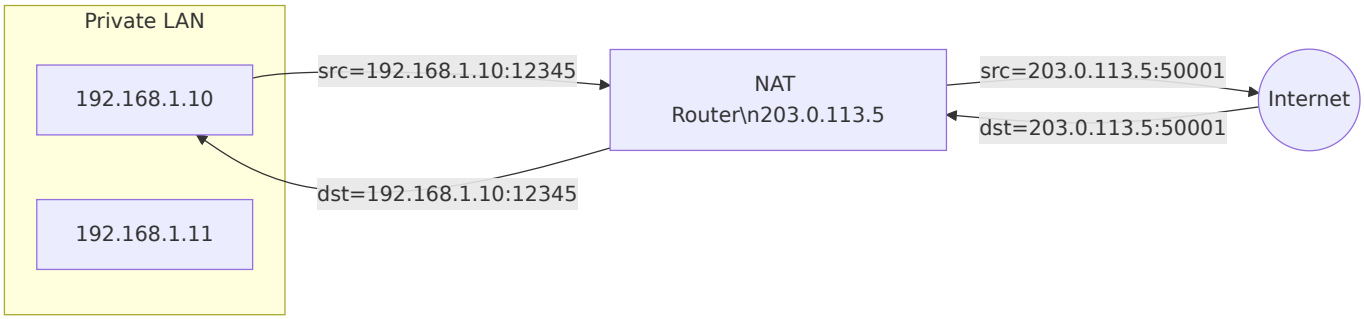


5.2 NAT (Network Address Translation)

NAT allows multiple private IP addresses (e.g., 192.168.1.x) to share a single public IP. Common types: **Source NAT** (SNAT) for outbound traffic, **Port Address Translation** (PAT).

Example:

Private IP:Port	Public IP:Port
192.168.1.10:12345	203.0.113.5:50001
192.168.1.11:80	203.0.113.5:50002



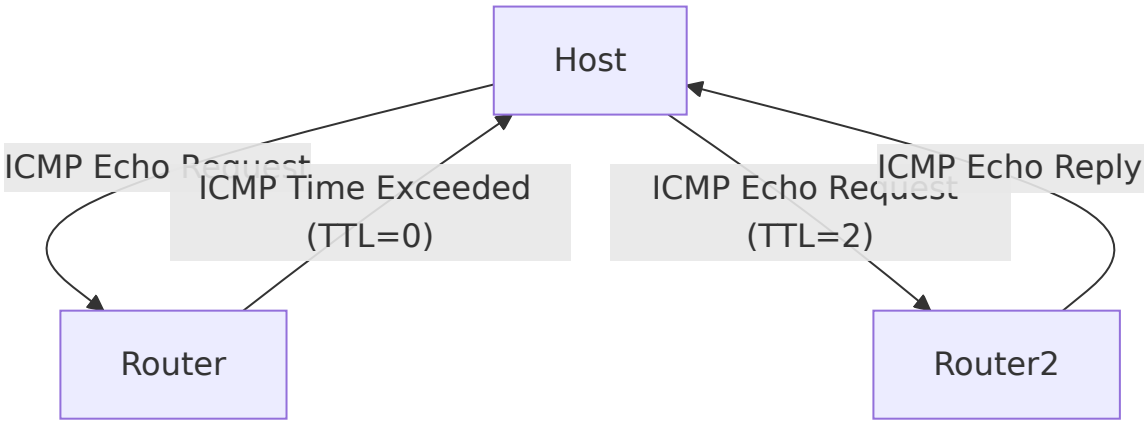
5.3 ICMP (Internet Control Message Protocol)

ICMP is used for error reporting and diagnostics. It is encapsulated inside IP.

Common message types:

- **Echo request / reply** (ping)
- **Destination unreachable** (e.g., port unreachable, network unreachable)
- **Time exceeded** (used by traceroute)
- **Redirect** (informs host of better gateway)

Example: `ping 8.8.8.8` sends an ICMP Echo Request; Google’ s server replies with Echo Reply.



Summary Table

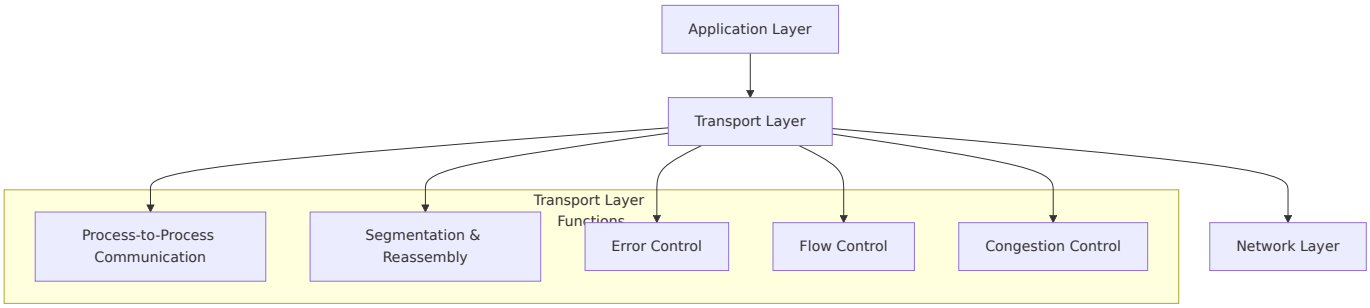
Topic	Key Characteristic	Example / Use
Subnetting	Borrow host bits for network	192.168.1.0/26 → 4 subnets of 62 hosts
CIDR	Aggregate prefixes	192.168.0.0/22 for four /24s
Distance Vector	Bellman-Ford, periodic updates	RIP
Link State	Dijkstra, full topology	OSPF
Flooding	Send out all ports except incoming	LSA distribution
NAT	Map private ↔ public IPs	Home router shares one public IP
ICMP	Error reporting & diagnostics	ping, traceroute

Chapter 6: Transport Layer

The Transport Layer is the fourth layer in the OSI model and the core of the TCP/IP stack. It provides logical communication between application processes running on different hosts.

Functions of the Transport Layer

The following diagram illustrates the key responsibilities of the transport layer:



- **Process-to-Process Communication**: Unlike the network layer which delivers packets to hosts, the transport layer delivers data to specific processes (applications) using ports.
- **Segmentation and Reassembly**: Divides application data into smaller segments for transmission and reassembles them at the destination.
- **Error Control**: Detects corrupted or lost segments and requests retransmission.
- **Flow Control**: Prevents a fast sender from overwhelming a slow receiver.
- **Congestion Control**: Prevents excessive traffic from causing network congestion.

Two main transport protocols dominate the Internet: **TCP** (Transmission Control Protocol) and **UDP** (User Datagram Protocol).

TCP - Transmission Control Protocol

TCP is a connection-oriented, reliable transport protocol that provides a stream-oriented service.

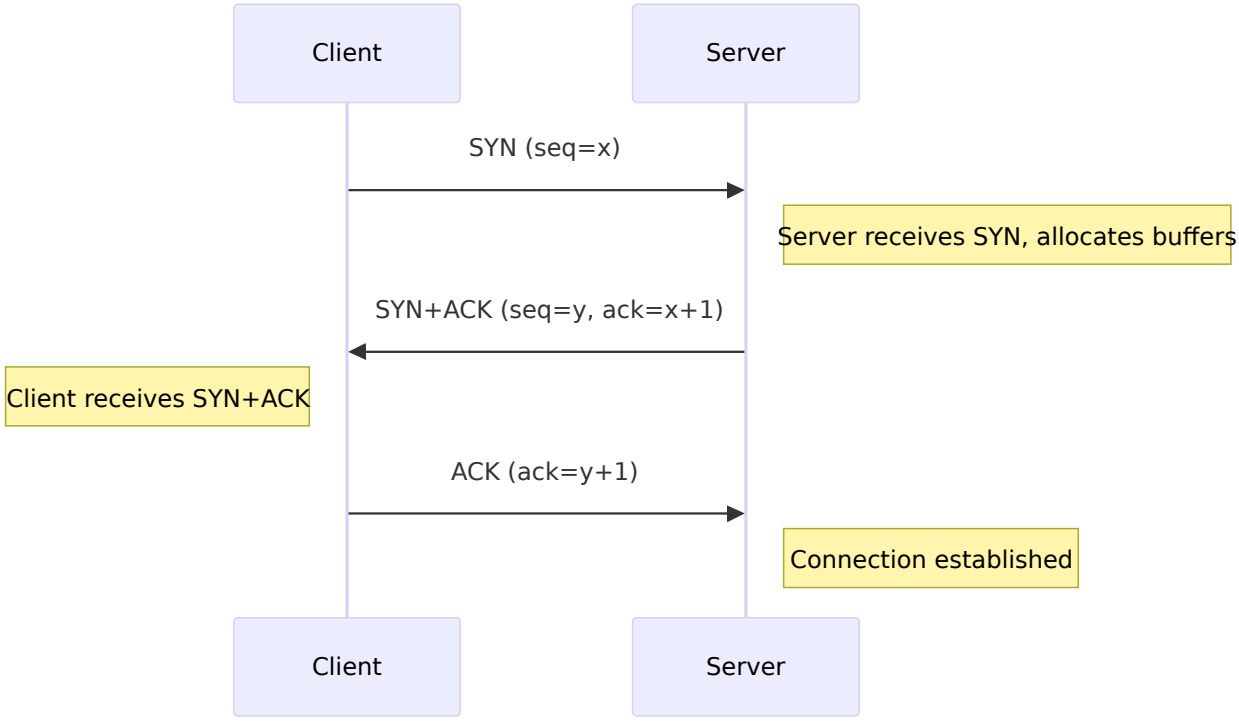
Key Characteristics

- **Connection-oriented**: A logical connection must be established before data exchange.
- **Reliable**: Uses acknowledgements (ACKs) and retransmissions to guarantee delivery.
- **In-order delivery**: Segments are reassembled in the correct order.

- **Full-duplex**: Simultaneous bidirectional data transfer.

Three-Way Handshake

TCP establishes a connection using a three-way handshake. This synchronizes sequence numbers and negotiates parameters.



- **SYN**: Synchronize sequence number
- **ACK**: Acknowledgment
- After this exchange, both sides can send data.

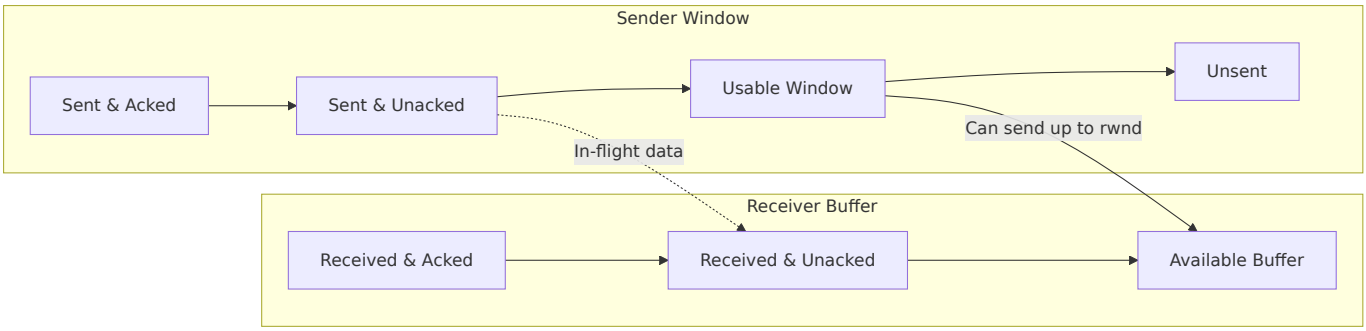
Reliable Communication

TCP uses:

- **Sequence numbers** to number each byte of data.
- **Acknowledgments** to confirm receipt.
- **Retransmission timers** to resend unacknowledged data after timeout.

Flow Control: Sliding Window

TCP uses a sliding window mechanism to control the flow of data. The receiver advertises a `rwnd` (receiver window) indicating available buffer space.



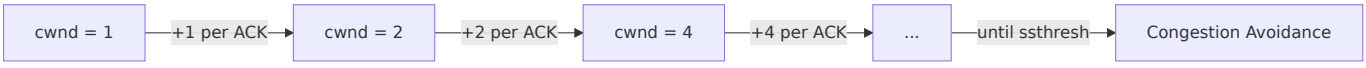
- The sender cannot exceed the receiver's advertised window.
- As the receiver processes data and sends ACKs, the window slides forward.

Congestion Control

TCP implements four algorithms to manage network congestion. The congestion window (`cwnd`) limits the amount of data a sender can inject into the network.

1. Slow Start

- Initially, `cwnd = 1 MSS` (Maximum Segment Size).
- For each ACK received, `cwnd` doubles (exponential growth) until a threshold (`ssthresh`) is reached.



2. Congestion Avoidance

- When `cwnd >= ssthresh` , growth becomes linear: `cwnd += 1/cwnd` per ACK.
- Continues until packet loss is detected.

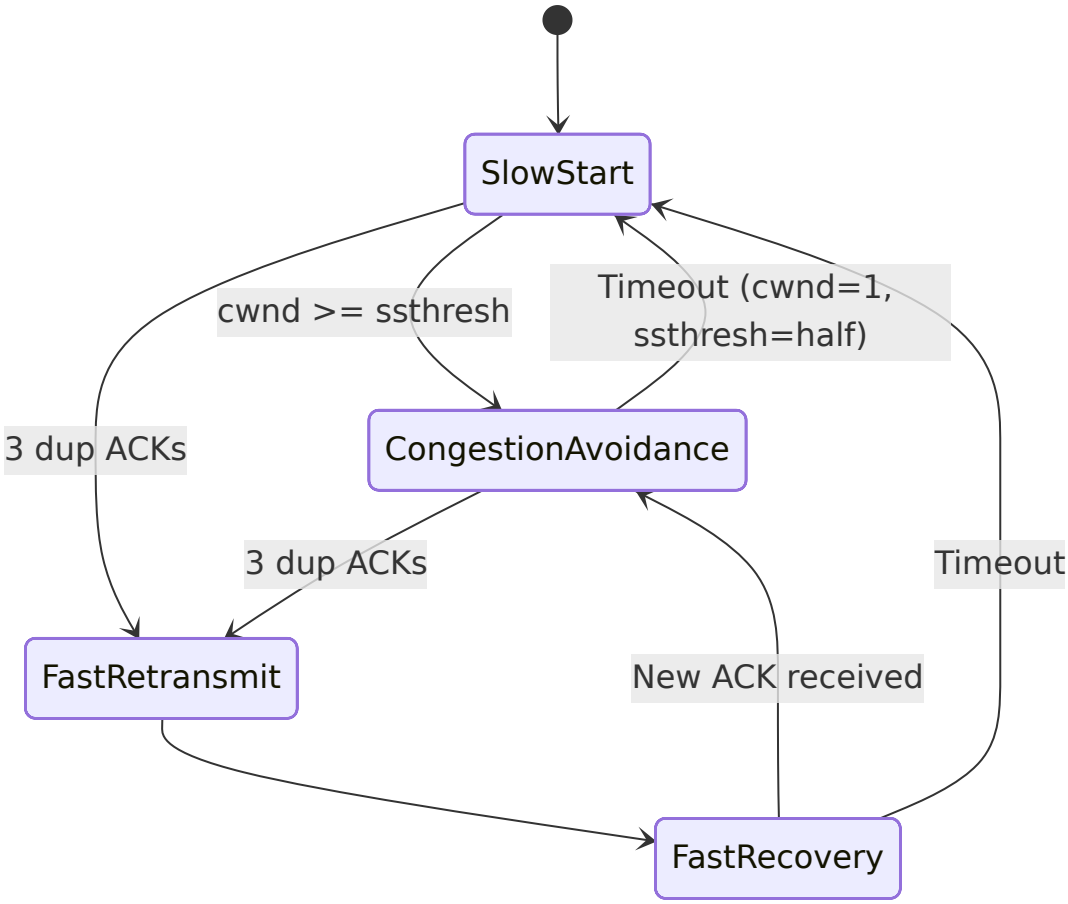
3. Fast Retransmit

- Upon receiving 3 duplicate ACKs, the sender retransmits the missing segment immediately (without waiting for timeout).

4. Fast Recovery

- After fast retransmit, `ssthresh = cwnd/2` , `cwnd = ssthresh + 3` .
- For each subsequent duplicate ACK, `cwnd += 1` .
- When a new ACK arrives, `cwnd = ssthresh` and enters congestion avoidance.

The following state diagram summarizes TCP congestion control behavior:



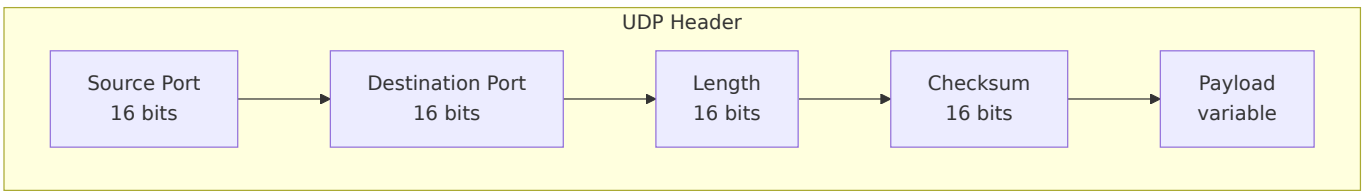
UDP - User Datagram Protocol

UDP is a simple, connectionless transport protocol that provides minimal service.

Key Characteristics

- **Connectionless**: No handshake; each datagram is independent.
- **Unreliable**: No guarantees of delivery, ordering, or duplicate protection.
- **No flow control**: Sender can transmit at any rate.
- **No congestion control**: Does not react to network congestion.
- **Low overhead**: 8-byte header (vs. 20-byte TCP header).

UDP Datagram Format

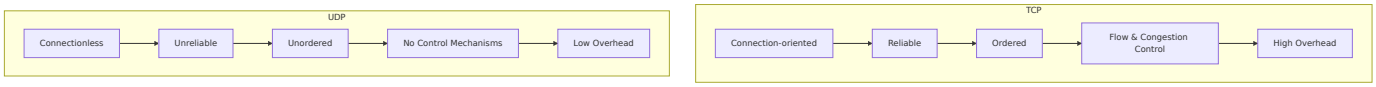


Use Cases

UDP is preferred when speed and low latency outweigh reliability:

Application	Why UDP?
Live streaming / VoIP	Dropped packets are tolerable; retransmission causes delay
DNS queries	Short request-response; retry mechanism at application layer
SNMP (network monitoring)	Low overhead, periodic polling
DHCP	Broadcast-based, no need for connection state
Online gaming	Fast updates; old data is useless

Comparison Diagram



Summary

- The transport layer enables **process-to-process communication** and provides segmentation, error control, flow control, and congestion control.
- **TCP** is connection-oriented and reliable, using three-way handshake, sliding window flow control, and sophisticated congestion control (slow start, congestion avoidance, fast retransmit, fast recovery).
- **UDP** is connectionless, fast, and lightweight, suitable for real-time applications and simple request-response protocols.

Choose TCP when data integrity and order matter; choose UDP when speed and low latency are critical.

Chapter 7: Application Layer

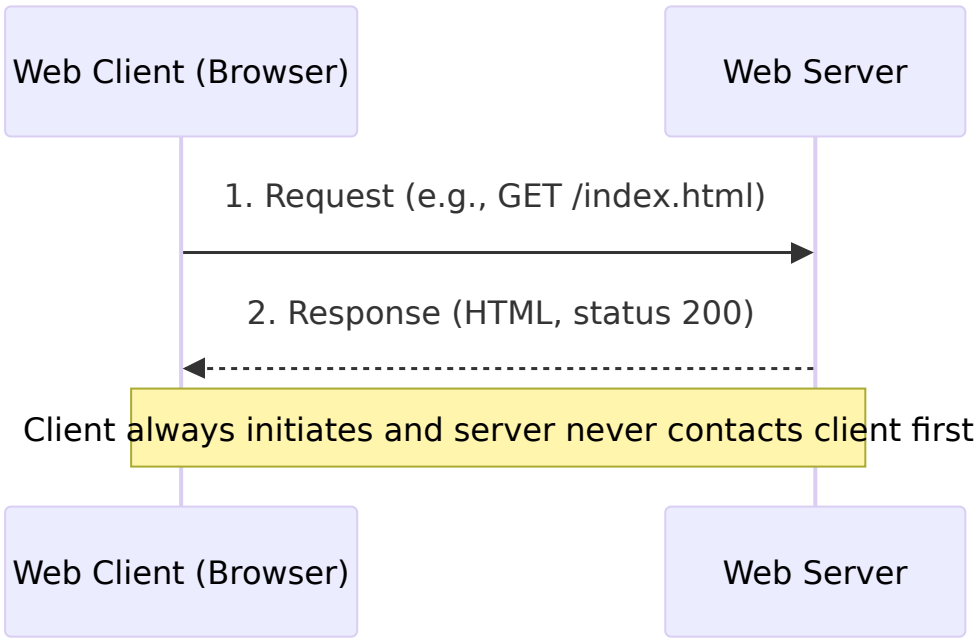
The **Application Layer** (Layer 7 of the OSI model) is the closest to the end user. It provides protocols and services that enable applications to communicate over a network. This chapter covers the essential protocols—HTTP/HTTPS, FTP, SMTP, POP3/IMAP, DNS, DHCP—and the core concepts of client-server architecture, URL structure, cookies, and sessions.

---

1. Client-Server Architecture

Definition: A distributed application structure that partitions tasks between *service providers* (servers) and *service requesters* (clients).

- **Client** – initiates contact, waits for replies, runs on user devices.
- **Server** – always on, listens for requests, provides responses, often serves many clients simultaneously.



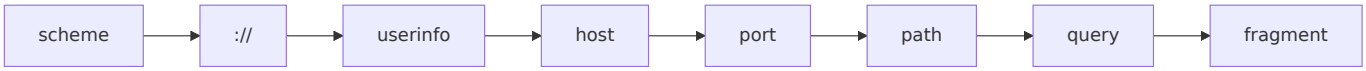
Example:

- **Client**: Your smartphone’s weather app.
- **Server**: A cloud-hosted weather API (e.g., `api.weather.com`). The app requests current temperature, the server replies with JSON data.

---

2. URL Structure (Uniform Resource Locator)

A URL is a specific string that identifies a resource and tells the client *how* to access it.



Components (with real example)
`https://alice:[email protected]:443/search?q=mermaid&lang=en#results`

Component	Value	Purpose
scheme	`https`	Protocol to use (HTTP over TLS)
user:password	`alice:secret`	Authentication (rarely used in browsers)
host	`docs.example.com`	Domain name or IP address
port	`443`	TCP port (default for HTTPS)
path	`/search`	Resource location on server
query	`q=mermaid&lang=en`	Key-value parameters (dynamic content)
fragment	`results`	Section inside the HTML page (client-side only)

> **Note**: In practice, credentials in URLs are deprecated for security reasons.

---

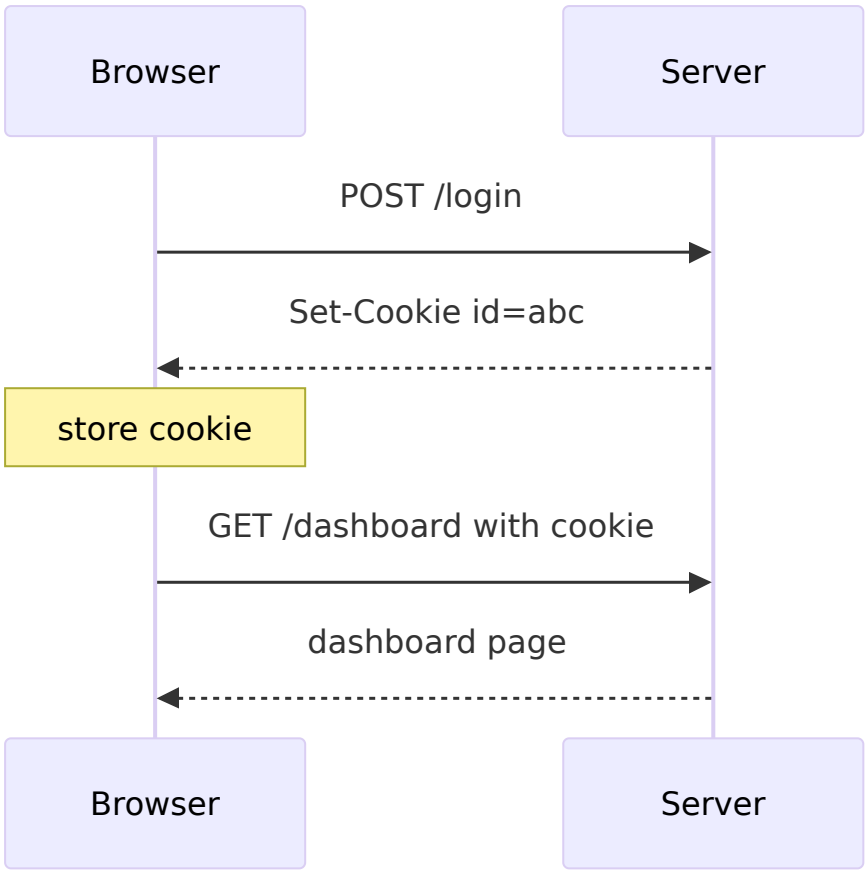
3. Cookies and Sessions

HTTP is **stateless** – each request is independent. Cookies and sessions add state.

3.1 Cookies

Small text files stored by the browser and sent automatically with every request to the same domain.

How it works:



Key attributes:

- `Expires` / `Max-Age` – lifetime.
- `Domain` / `Path` – scope.
- `Secure` – only send over HTTPS.
- `HttpOnly` – prevents JavaScript access (mitigates XSS).

3.2 Sessions

A **session** is server-side storage tied to a unique session ID (usually stored in a cookie). The server keeps user data (e.g., shopping cart, login state) and the client only holds the identifier.

Example:

- User adds items to cart → session ID `xyz` stores `{cart: ["laptop","mouse"]}` on server.
- Next request includes `Cookie: sessionId=xyz` → server retrieves the cart.

---

4. HTTP / HTTPS

HTTP (HyperText Transfer Protocol)

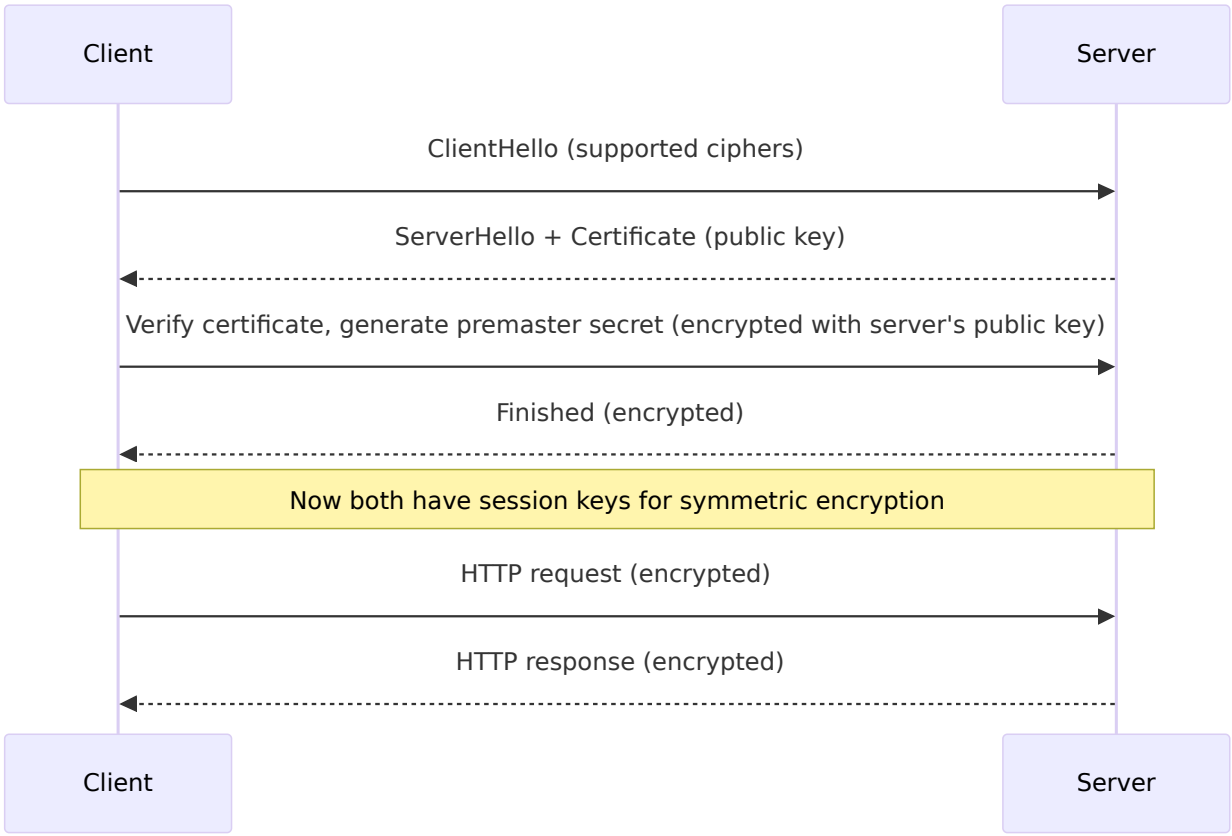
- **Port**: 80 (plain text).
- **Methods**: GET, POST, PUT, DELETE, etc.
- **Status codes**: 200 OK, 404 Not Found, 500 Internal Server Error.

Basic HTTP exchange:

HTTPS (HTTP Secure)

- **Port**: 443.
- Adds **TLS/SSL** encryption between client and server.
- Prevents eavesdropping, tampering, and impersonation.

Simplified TLS handshake:



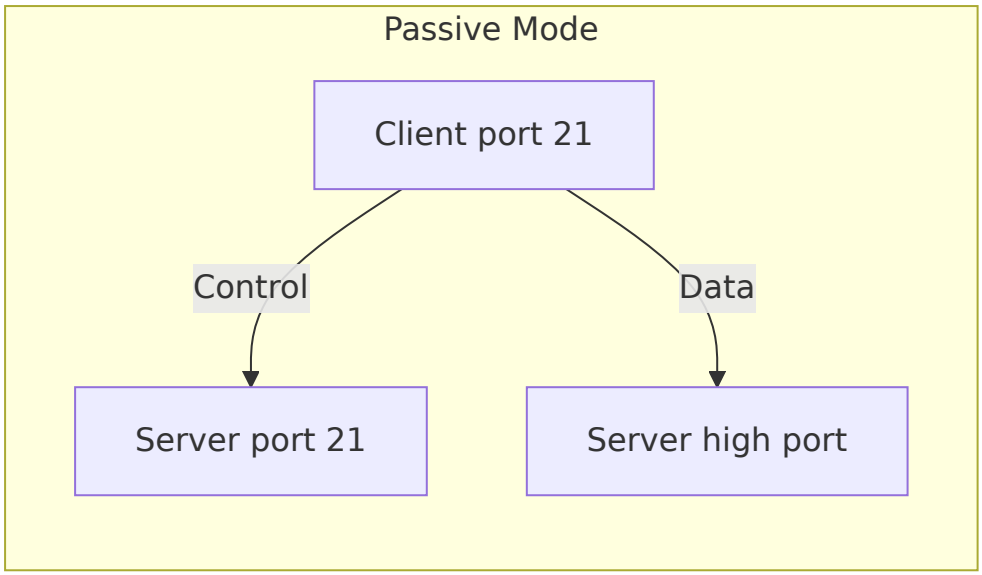
Real-world example: Banking websites, e-commerce (Amazon, eBay) – always use HTTPS to protect credentials and payment data.

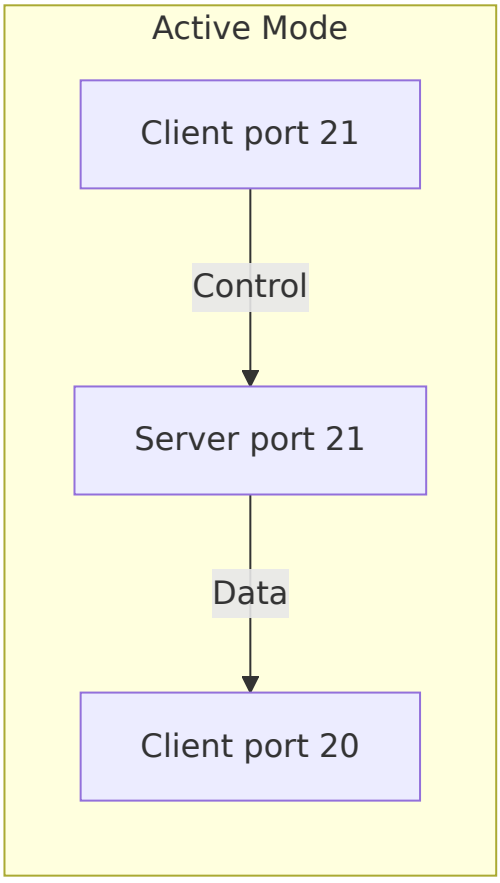
---

5. FTP (File Transfer Protocol)

- **Ports**: 20 (data) & 21 (control).
- **Purpose**: Transfer files between client and server.
- **Authentication**: Username/password (or anonymous).

FTP has two modes:





Common FTP commands:

- `USER`, `PASS` – login.
- `RETR filename` – download.
- `STOR filename` – upload.
- `LIST` – list directory.

Example: A web developer uploads website files to a hosting server using FileZilla (FTP client) to `ftp.example.com`.

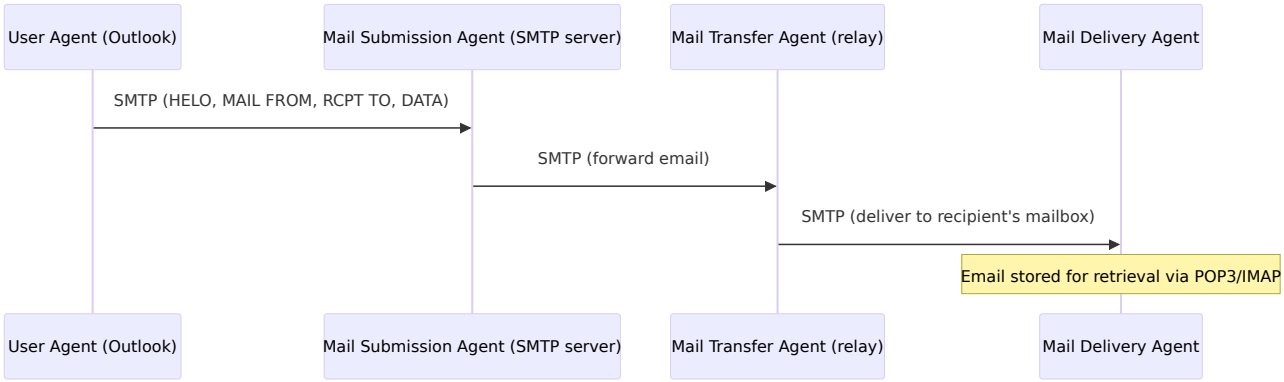
> **Note**: FTP is insecure (passwords sent in plain text). SFTP (SSH File Transfer Protocol) or FTPS (FTP over TLS) are preferred.

---

6. SMTP (Simple Mail Transfer Protocol)

- **Port**: 25 (standard), 587 (submission), 465 (SMTPS).
- **Purpose**: Send email from client to server, and between mail servers.
- **Direction**: Push protocol (client pushes email to server).

Email flow using SMTP:



Example SMTP session:

```
HELO mail.example.com
MAIL FROM: <[email protected]>
RCPT TO: <[email protected]>
DATA
Subject: Meeting

Tomorrow at 10 AM.
.
QUIT
```

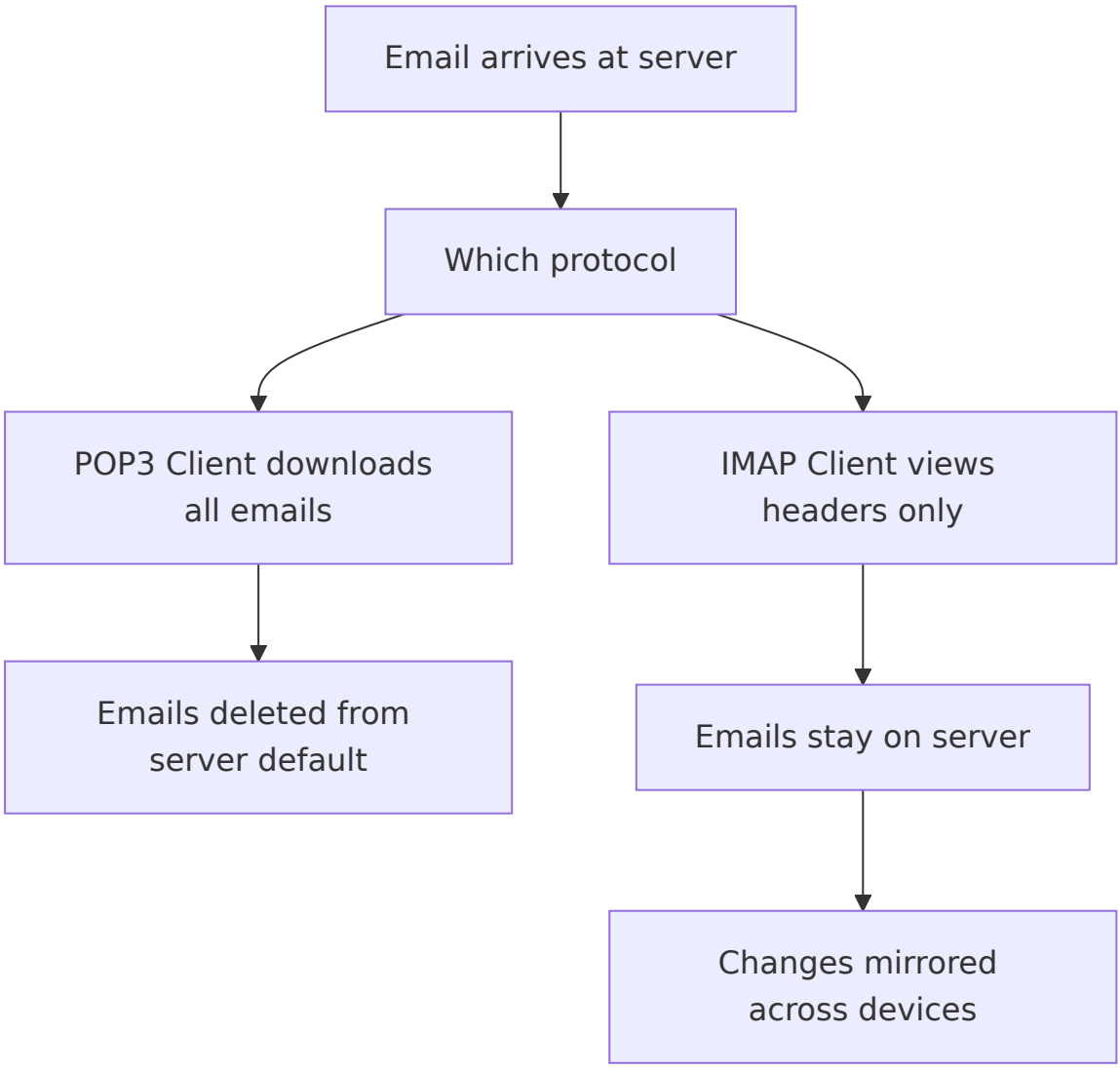
---

7. POP3 / IMAP (Email Retrieval)

Both are **pull protocols** – the client retrieves emails from the server.

Feature	POP3 (Post Office Protocol v3)	IMAP (Internet Message Access Protocol)
Port	110 (plain), 995 (SSL)	143 (plain), 993 (SSL)
Email storage	Downloads and usually deletes from server	Keeps emails on server, synchronises across devices
Folder support	No (only Inbox)	Yes (multiple folders)
Offline access	Yes (emails stored locally)	Limited (requires sync)

Typical workflow:



Example:

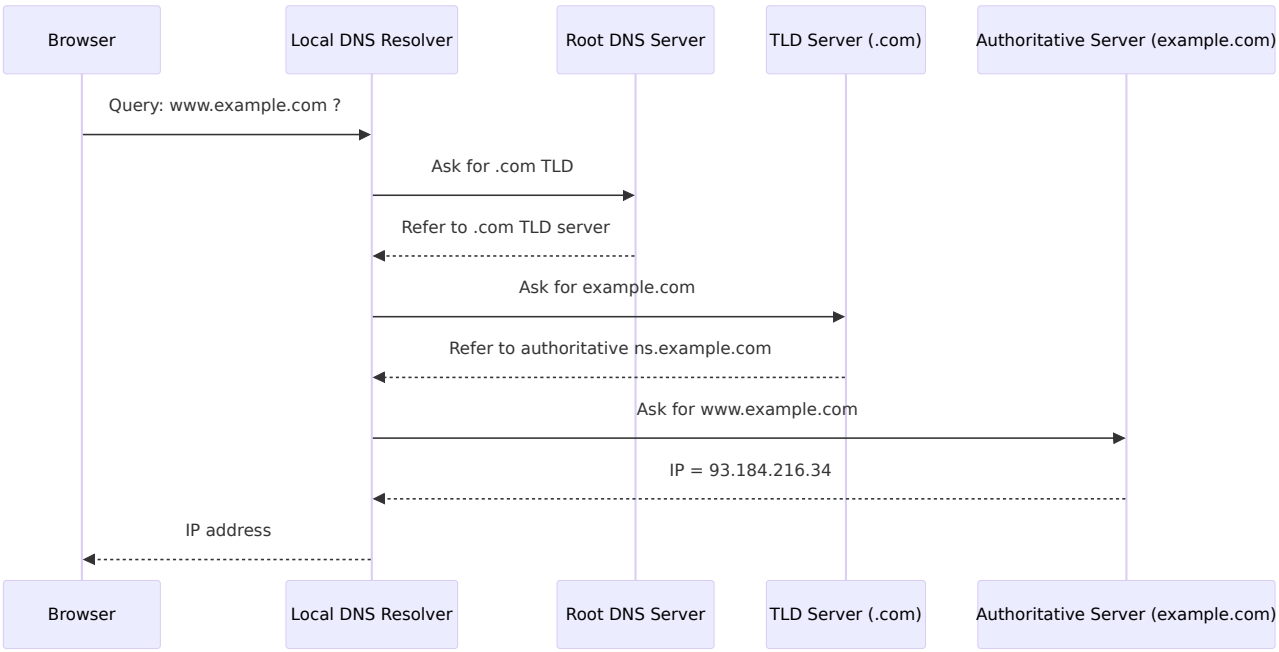
- **POP3** used in legacy setups where you read email only from one computer.
- **IMAP** used by Gmail, Outlook.com, and corporate environments (access from phone, laptop, webmail all stay in sync).

---

8. DNS (Domain Name System)

- **Port**: 53 (UDP for queries, TCP for zone transfers).
- **Purpose**: Translates human-readable domain names (e.g., `google.com`) to IP addresses (e.g., `142.250.190.46`).

DNS resolution steps:



Record types:

- **A** – IPv4 address.
- **AAAA** – IPv6 address.
- **CNAME** – alias (e.g., `www` → `@`).
- **MX** – mail exchange server.

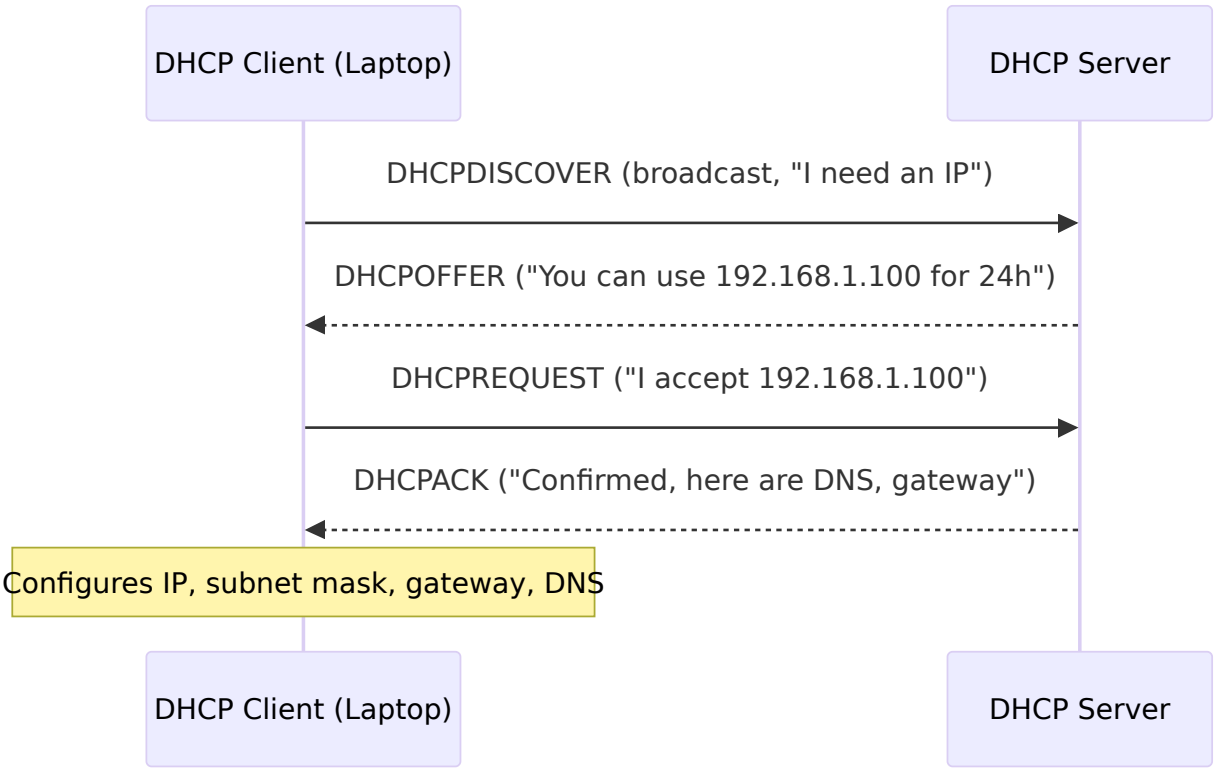
Example: When you type `https://github.com`, your computer asks DNS: "Where is github.com?" → receives `140.82.112.3`.

---

9. DHCP (Dynamic Host Configuration Protocol)

- **Port**: 67 (server), 68 (client).
- **Purpose**: Automatically assigns IP addresses, subnet mask, default gateway, and DNS servers to devices on a network.

DORA process (Discover, Offer, Request, Acknowledge):



Lease time: The IP is valid only for a limited time (e.g., 24 hours). Client renews before expiry.

Example: When you connect to a coffee shop Wi-Fi, your phone uses DHCP to obtain an IP address like `192.168.0.15`, along with the router's IP as gateway and the ISP's DNS servers.

Summary Table of Protocols

Protocol	Port(s)	Main Function	Transport
HTTP	80	Transfer web pages (plain)	TCP
HTTPS	443	Secure web transfer	TCP (TLS)

FTP	20,21	File upload/download	TCP
SMTP	25,587	Send email	TCP
POP3	110,995	Retrieve email (download & delete)	TCP
IMAP	143,993	Retrieve email (server-side sync)	TCP
DNS	53	Name → IP resolution	UDP (mostly)
DHCP	67,68	Auto-assign IP configuration	UDP

Final Notes

- The **Application Layer** hides the complexity of lower layers (transport, network, data link, physical) from end-user applications.
- **Client-server** is the dominant model, but peer-to-peer (P2P) also exists (e.g., BitTorrent).
- **Security** is increasingly embedded at this layer (HTTPS, SMTPS, IMAPS, POP3S, DNSSEC).

Understanding these protocols and concepts is essential for anyone building, using, or troubleshooting modern networks and the web.

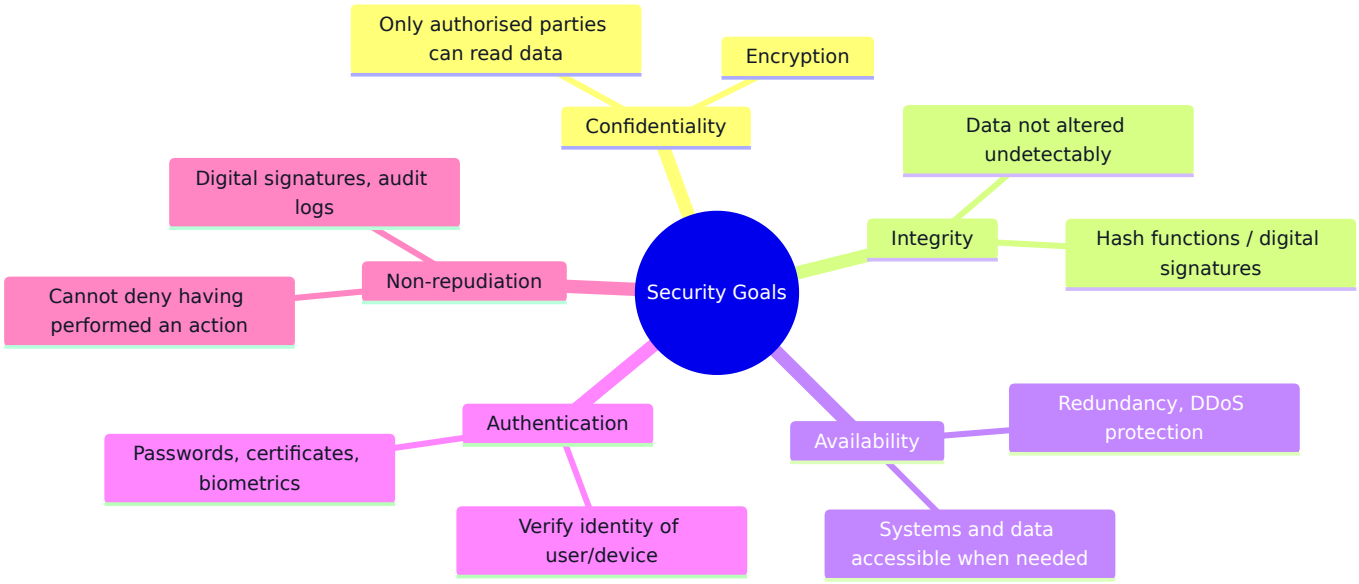
Chapter 8: Network Security

Network security is the practice of protecting data, devices, and communications from unauthorized access, misuse, or destruction. This chapter covers the fundamental goals of security, the cryptographic building blocks, key security protocols (SSL/TLS, HTTPS), common attacks (phishing, MitM, DoS/DDoS), and protection mechanisms (firewalls, intrusion detection systems).

---

1. Security Goals (The CIA Triad + Non-repudiation)

Security objectives are often summarised as **Confidentiality**, **Integrity**, **Availability** – plus **Authentication** and **Non-repudiation**.



Goal	Meaning	Real-world example
Confidentiality	Prevent unauthorised reading	Encrypting a WhatsApp message so only recipient can read it
Integrity	Prevent unauthorised modification	Using a checksum to verify a downloaded file wasn’ t tampered
Availability	Ensure service is accessible	Website remains online during a flash sale

Authentication	Prove identity	Logging into online banking with username + OTP
Non-repudiation	Prevent denial of an action	Digitally signed contract – sender cannot claim they didn’ t send it

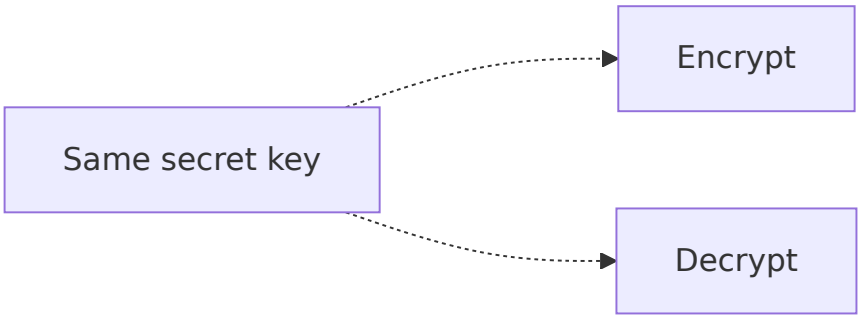
---

2. Cryptography

Cryptography is the art of securing communication by transforming data into an unreadable format (ciphertext) and back.

2.1 Symmetric Key Encryption

- **Same key** for encryption and decryption.
- Fast and efficient for bulk data.
- Problem: securely sharing the key between parties.



Common algorithms: AES (Advanced Encryption Standard), ChaCha20, DES (legacy).

Example:

- Encrypting a hard drive with BitLocker (uses AES).
- Wi-Fi WPA2 uses AES for data confidentiality.

2.2 Asymmetric Key Encryption (Public Key)

- **Key pair**: public key (share freely) and private key (kept secret).
- Data encrypted with **public key** can only be decrypted with the matching **private key**.

- Much slower than symmetric; often used to exchange a symmetric session key.

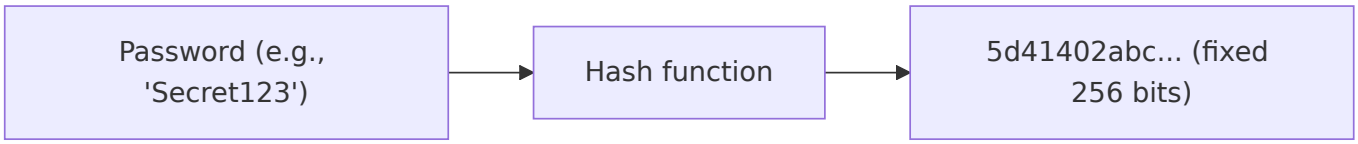
Common algorithms: RSA, ECC (Elliptic Curve Cryptography).

Real-world use:

- HTTPS: browser uses server’ s public key to send a secret symmetric key (TLS handshake).
- Email encryption (PGP).
- Digital signatures (sign with private key, verify with public key).

2.3 Hash Functions

- **One-way** function: input → fixed-length output (hash/digest).
- Cannot reverse a hash to find the original input.
- Same input always produces the same hash.
- Small change in input → completely different hash (avalanche effect).



Common algorithms: SHA-256, SHA-3, MD5 (broken, not secure).

Applications:

- **Password storage**: store hash, not plaintext. When user logs in, hash the entered password and compare.
- **Integrity checking**: download a file + its SHA-256 hash; recompute hash to verify no corruption.
- **Digital signatures**: sign the hash of a message (faster than signing whole message).

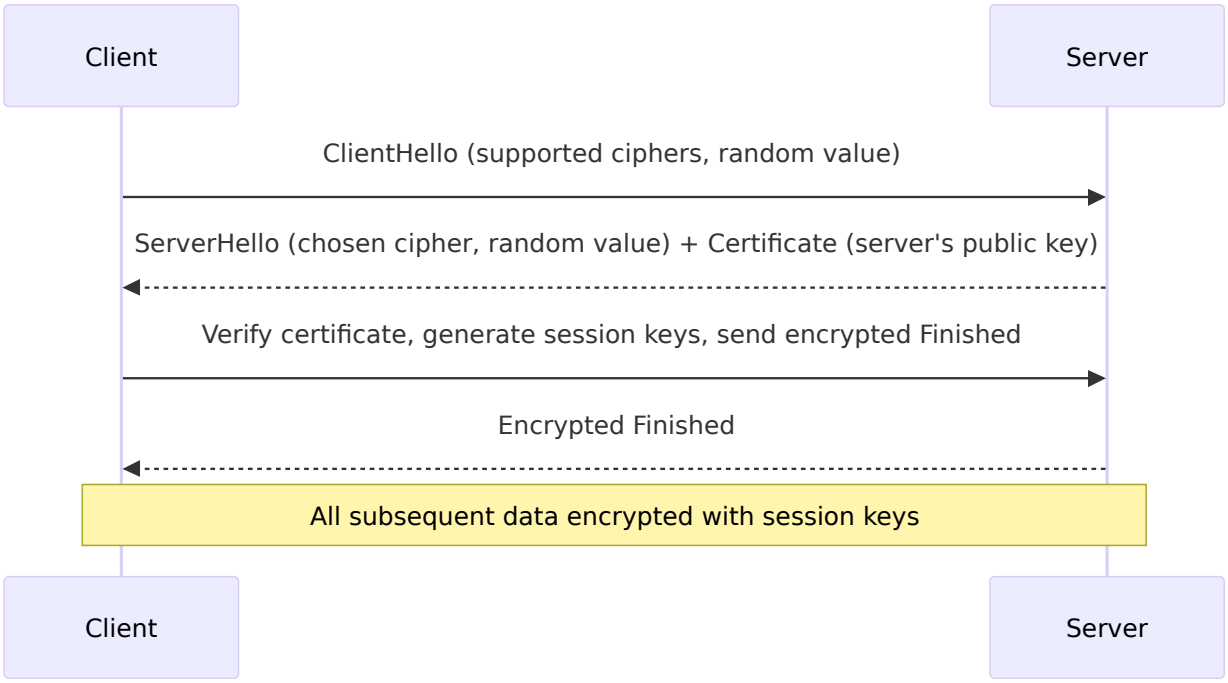
---

3. Security Protocols: SSL/TLS and HTTPS

3.1 SSL / TLS (Transport Layer Security)

- **SSL** (Secure Sockets Layer) is deprecated; **TLS** (Transport Layer Security) is the modern standard.
- Operates between the Application layer (HTTP) and Transport layer (TCP).
- Provides: confidentiality (encryption), integrity (message authentication codes), and authentication (certificates).

Simplified TLS 1.3 handshake:



Certificate: Binds a domain name to a public key, digitally signed by a Certificate Authority (CA) (e.g., Let’ s Encrypt, DigiCert).

3.2 HTTPS (HTTP over TLS)

- **Port 443**.
- HTTP messages are sent inside a TLS tunnel.
- Protects against eavesdropping (no one sees your search terms), tampering (no injected ads by ISP), and impersonation (you are talking to the real `paypal.com`).

How a browser verifies a certificate:

1. Checks that the certificate’ s domain matches the URL.
2. Checks that the certificate is not expired.
3. Checks that the signature is from a trusted CA (list built into OS/browser).
4. (Optional) checks revocation status via OCSP/CRL.

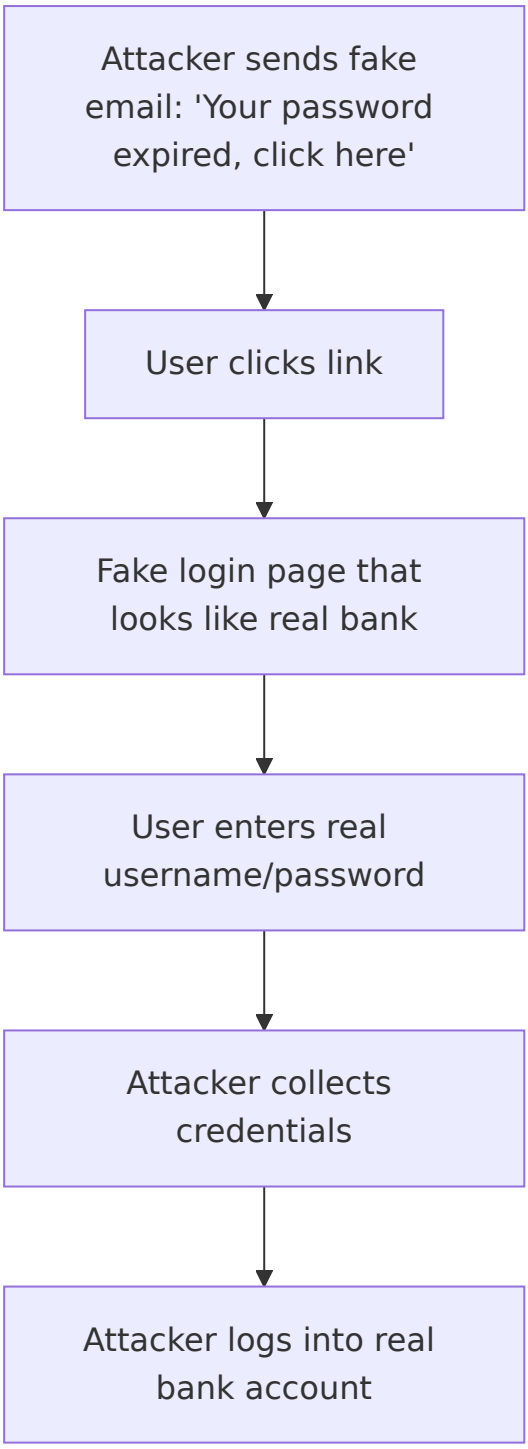
> **Example**: When you visit `https://github.com`, the browser performs a TLS handshake with GitHub's server, verifies its certificate, and then sends the HTTP request encrypted.

---

4. Common Attacks

4.1 Phishing

- Attacker impersonates a legitimate entity (bank, IT support) to trick the user into revealing credentials, credit card numbers, or clicking malicious links.
- Usually delivered via email, SMS (smishing), or fake websites.

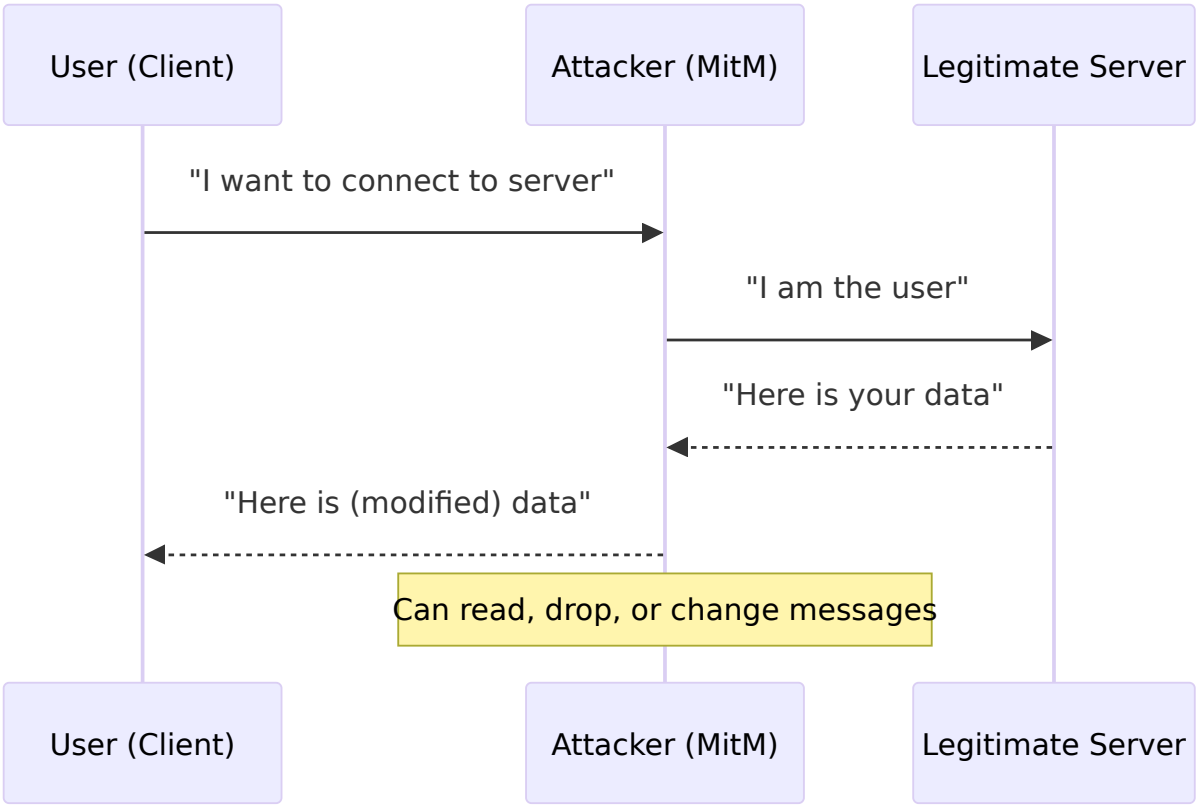


Prevention:

- Check sender address and URL carefully.
- Use multi-factor authentication (MFA).
- Browser warnings for known phishing sites.
- User education.

4.2 Man-in-the-Middle (MitM)

- Attacker secretly intercepts and possibly alters communication between two parties who believe they are talking directly.
- Can happen on unencrypted Wi-Fi (evil twin access point), ARP spoofing, or compromised routers.



Prevention:

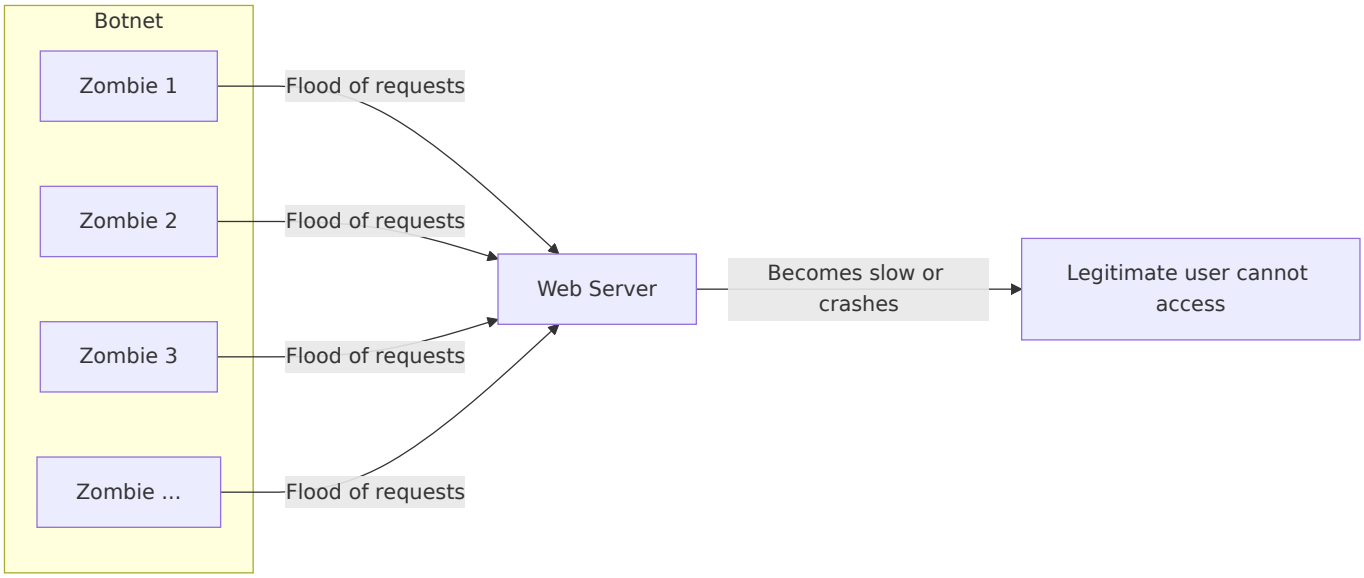
- HTTPS with proper certificate validation (browser will warn if certificate is spoofed).
- Avoid public Wi-Fi without VPN.
- Use mutual authentication (e.g., client certificates).

4.3 DoS / DDoS (Denial of Service)

- **DoS**: Single source floods a target with traffic, making it unavailable.
- **DDoS**: Multiple compromised machines (botnet) coordinate the attack.

Types:

- **Volumetric**: Flood bandwidth (UDP floods, ICMP floods).
- **Protocol**: Exploit weaknesses in protocols (SYN flood, Ping of Death).
- **Application layer**: Slow HTTP requests, exhausting server resources.



Real-world example: 2016 Dyn DNS attack – massive DDoS using Mirai botnet (IoT devices) took down Twitter, Netflix, and others.

Protection:

- Rate limiting, firewalls, intrusion prevention systems.
- Content Delivery Networks (CDNs) like Cloudflare absorb traffic.
- Over-provisioning bandwidth and using DDoS mitigation services.

---

5. Protection Mechanisms

5.1 Firewalls

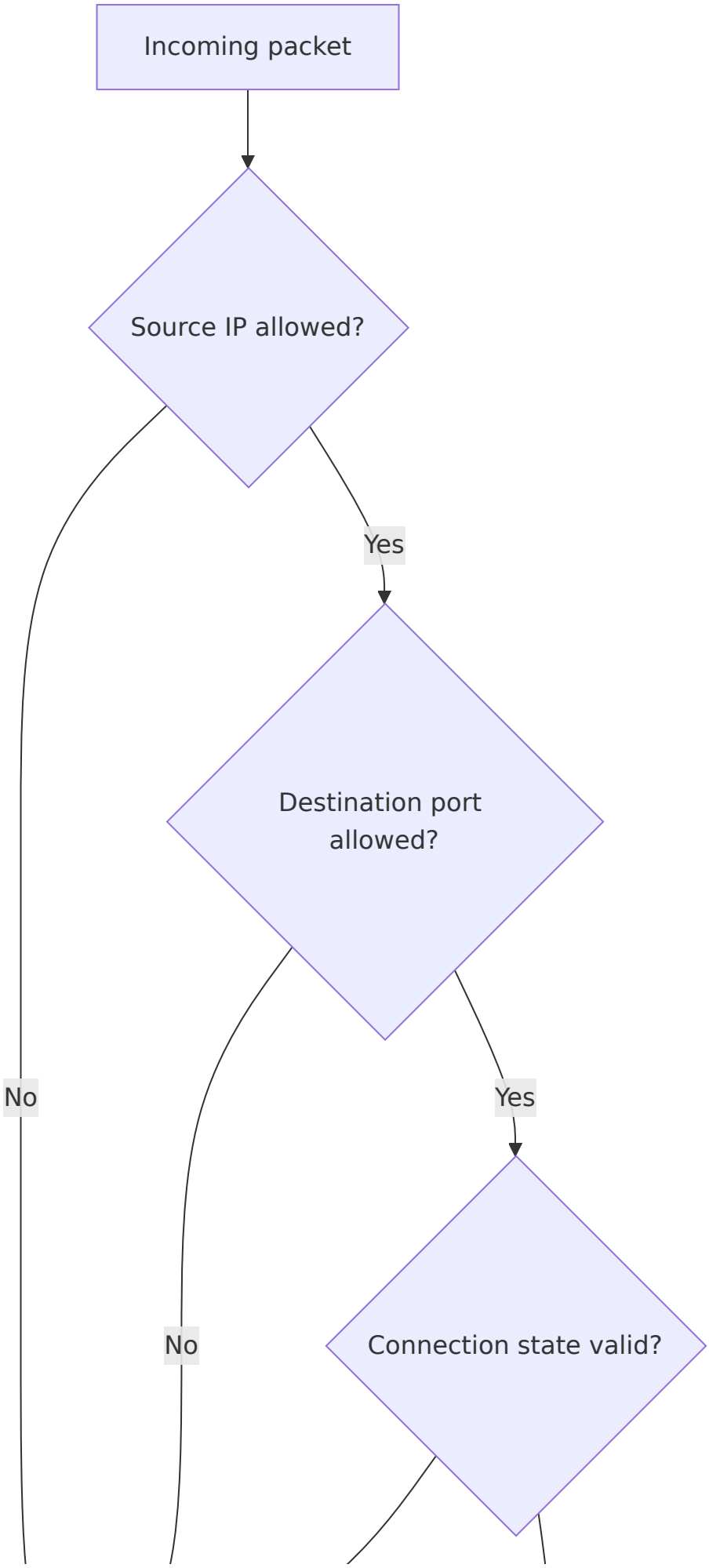
A firewall monitors and controls incoming/outgoing network traffic based on predefined rules.

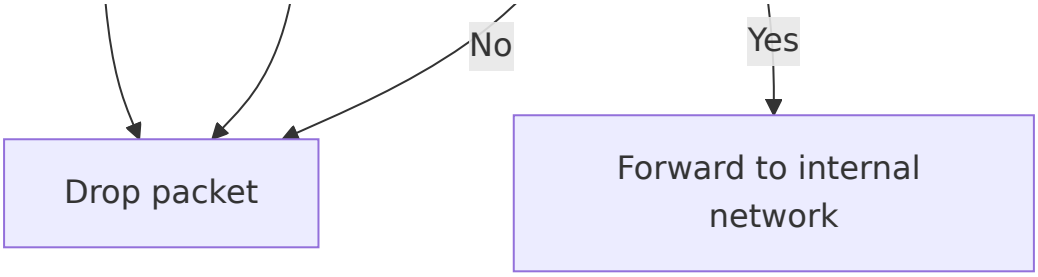
Types:

Type	How it works	Example
Packet filtering	Inspects headers (IP, port, protocol)	iptables, Windows Firewall
Stateful inspection	Tracks connection state (e.g., allows return traffic)	Most modern firewalls
Application layer (Next-Gen)	Deep packet inspection, understands HTTP, DNS, etc.	Palo Alto, Fortinet

Example rule (conceptual):

- Block all incoming traffic on port 23 (Telnet).
- Allow outgoing HTTPS (port 443) from any internal IP.
- Allow SSH (port 22) only from the IT department’ s IP range.


Incoming packet
Source IP allowed?
Yes
No
Destination port allowed?
Yes
No
Connection state valid?



5.2 Intrusion Detection Systems (IDS)

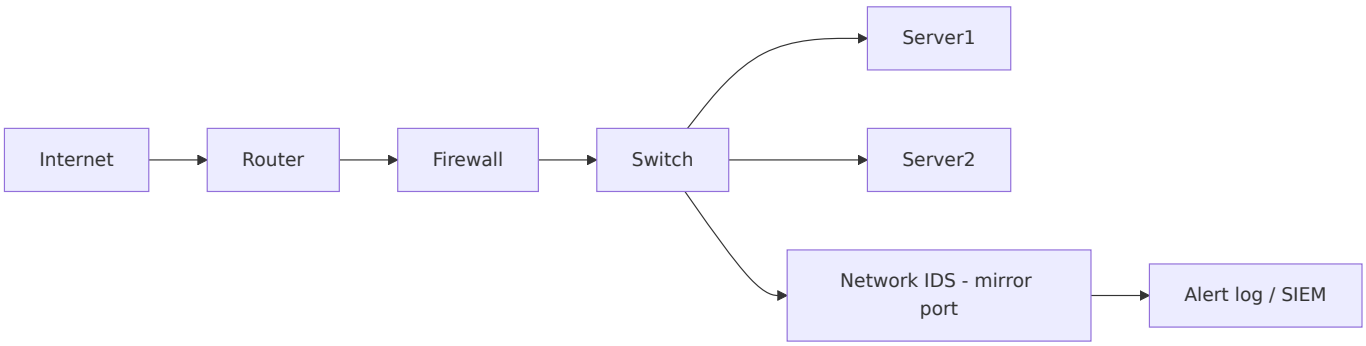
- **IDS** passively monitors traffic and alerts when suspicious patterns (signatures) or anomalies are detected.
- Does **not** block traffic (unlike Intrusion Prevention System – IPS).

Two main detection methods:

1. **Signature-based** – matches known attack patterns (e.g., a specific exploit string). Fast but cannot detect new attacks (zero-day).
2. **Anomaly-based** – learns normal behaviour and alerts on deviations. Can find novel attacks but may have false positives.

Deployment:

- **NIDS** (Network IDS): monitors a network segment (e.g., at a switch span port).
- **HIDS** (Host-based IDS): runs on a server, monitors logs, file integrity, system calls.



Example: Snort (open source NIDS) can alert when it sees a known SQL injection attempt like `' OR '1'='1` in a HTTP request.

Summary Table of Security Topics

Topic	Key concept	Real-world tool/protocol
Confidentiality	Encryption keeps data secret	AES, TLS

Integrity	Hash functions detect changes	SHA-256, HMAC
Authentication	Prove identity	Passwords, certificates, MFA
Non-repudiation	Digital signatures	RSA + SHA-256
Symmetric encryption	Same key for both sides	AES-256
Asymmetric encryption	Public / private key pair	RSA, ECC
Hash function	One-way fingerprint	SHA-256
TLS/SSL	Encrypts application data	HTTPS (port 443)
Phishing	Social engineering, fake login pages	MFA, user training
Man-in-the-middle	Intercepting communication	Certificate validation, VPN
DDoS	Overwhelming resources	Cloudflare, rate limiting
Firewall	Filters packets by rules	iptables, pfSense
IDS	Detects intrusions, alerts	Snort, Suricata

Final Notes

- **Defence in depth** – use multiple layers: firewall, IDS, encryption, strong authentication.
- **No single solution** guarantees security; people, processes, and technology must work together.
- **Threat landscape evolves** – keep software updated, monitor logs, and practice incident response.

Understanding these security fundamentals is essential for designing, operating, and auditing networks in any organisation.

Chapter 9: Congestion Control and Quality of Service (QoS)

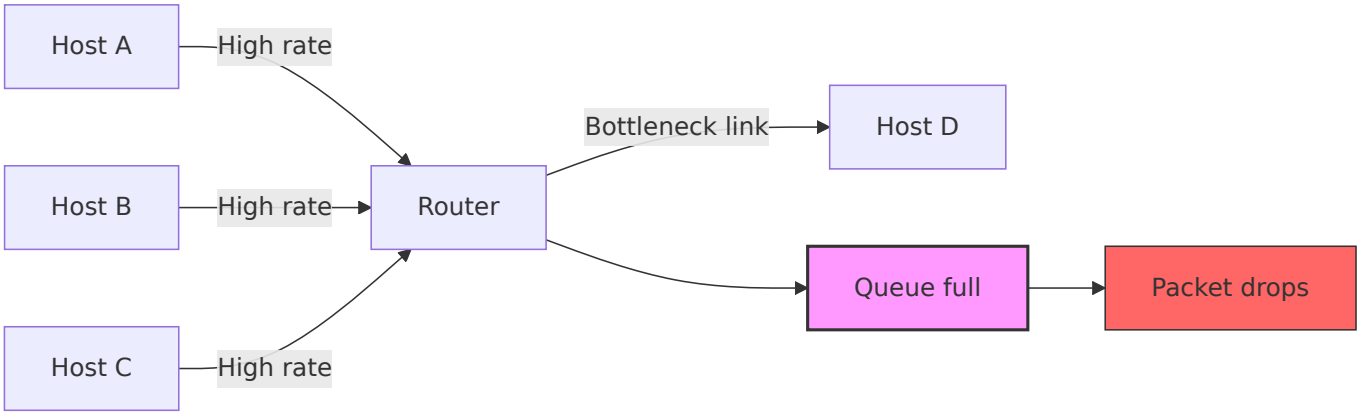
This document provides an overview of congestion control mechanisms, traffic shaping algorithms, and key Quality of Service (QoS) parameters. It includes Mermaid diagrams and practical examples to illustrate these networking concepts.

Causes of Congestion

Congestion occurs when the demand on network resources exceeds the available capacity. Common causes include:

- **High traffic bursts** – Sudden spikes in data from multiple sources.
- **Insufficient link bandwidth** – Slow links become bottlenecks.
- **Buffer overflow at routers** – Packets are dropped when queues fill up.
- **Slow receiver processing** – Receiver cannot keep up with incoming data.
- **Retransmissions** – Lost packets trigger more traffic, worsening the condition.

Congestion Scenario



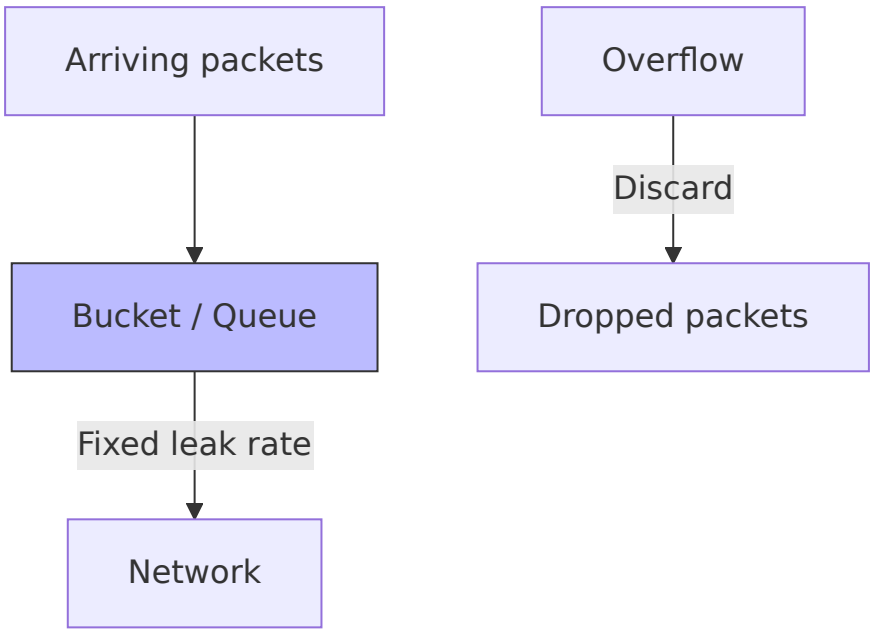
Traffic Shaping

Traffic shaping controls the rate and burstiness of data transmission to avoid congestion.

Leaky Bucket

The **Leaky Bucket** algorithm enforces a constant output rate regardless of input bursts. It works like a bucket with a small hole at the bottom.

- Packets arrive at arbitrary rates.
- They are placed into a bucket (queue).
- The bucket leaks (transmits) packets at a fixed rate.
- If the bucket overflows, packets are discarded.

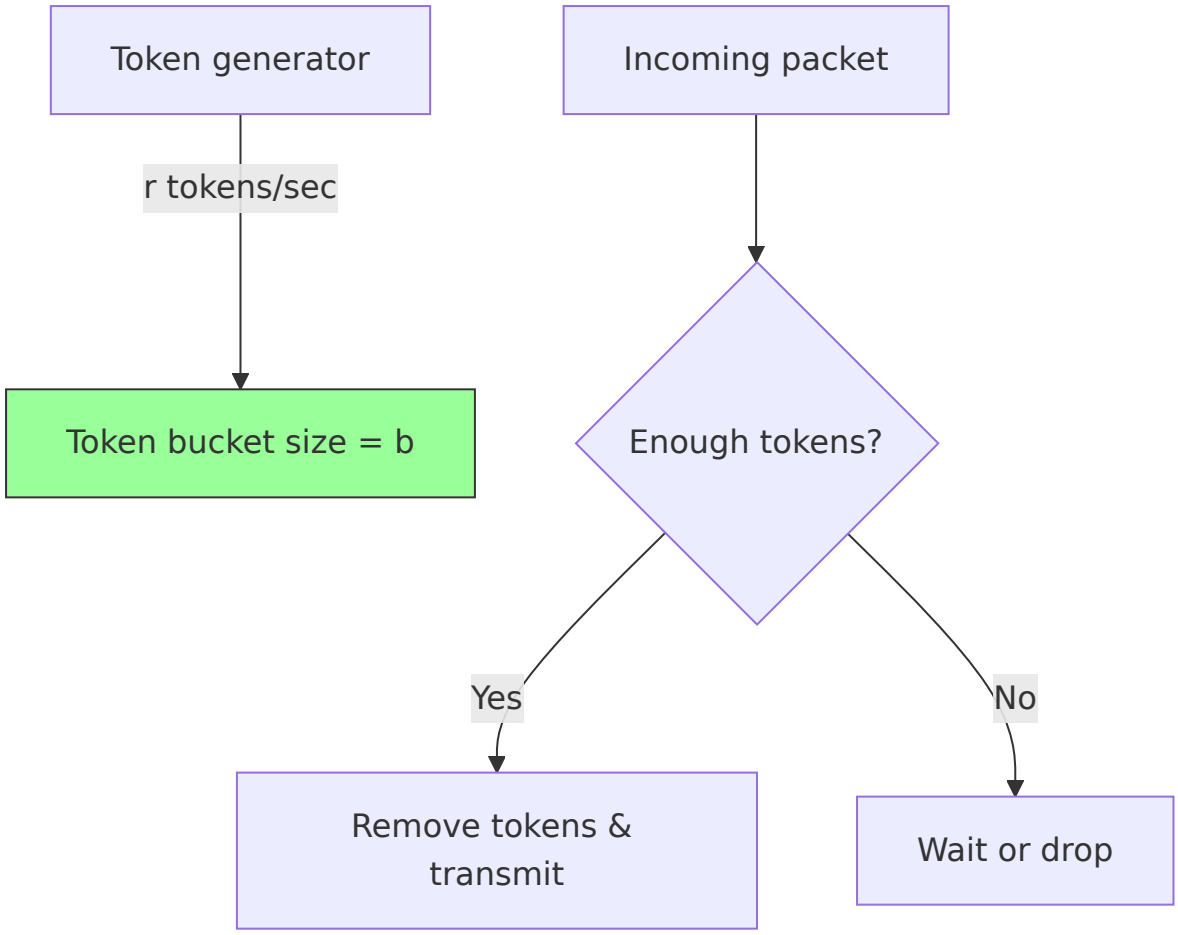


Example
A router is configured with a leaky bucket of size 5 packets and a leak rate of 1 packet/sec. If 10 packets arrive simultaneously, 5 are queued and transmitted one per second, while the other 5 are dropped.

Token Bucket

The **Token Bucket** allows bursts up to a certain limit while still shaping the average rate. Tokens are added to the bucket at a constant rate. A packet can be transmitted only if enough tokens are available.

- Bucket holds up to `b` tokens (burst size).
- Tokens arrive at rate `r` tokens/sec.
- Each packet consumes `t` tokens (usually 1 token per byte or per packet).
- If bucket is full, new tokens are discarded.



Example

A token bucket with rate `r = 1 Mbps` and bucket size `b = 1 MB`. A file transfer can send up to 1 MB instantly (using saved tokens), then continues at 1 Mbps. This allows bursts for interactive traffic while smoothing long-term flow.

Leaky Bucket vs. Token Bucket

Feature	Leaky Bucket	Token Bucket
Output rate	Strictly constant	Varies, up to peak burst rate
Burst handling	No bursts allowed	Allows bursts (up to bucket size)
Long-term average rate	Fixed leak rate	Token generation rate
Packet discard	On bucket overflow	No discard (packet waits or is dropped)

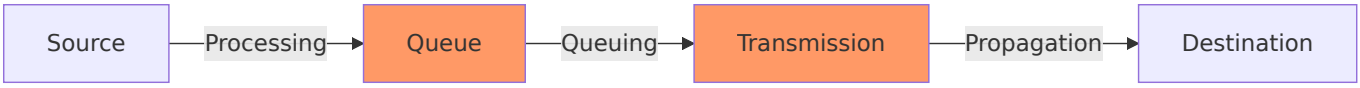
Quality of Service (QoS)

QoS refers to the ability to provide different priority levels to different traffic types, ensuring predictable performance.

Delay (Latency)

Delay is the time a packet takes to travel from source to destination. Components include:

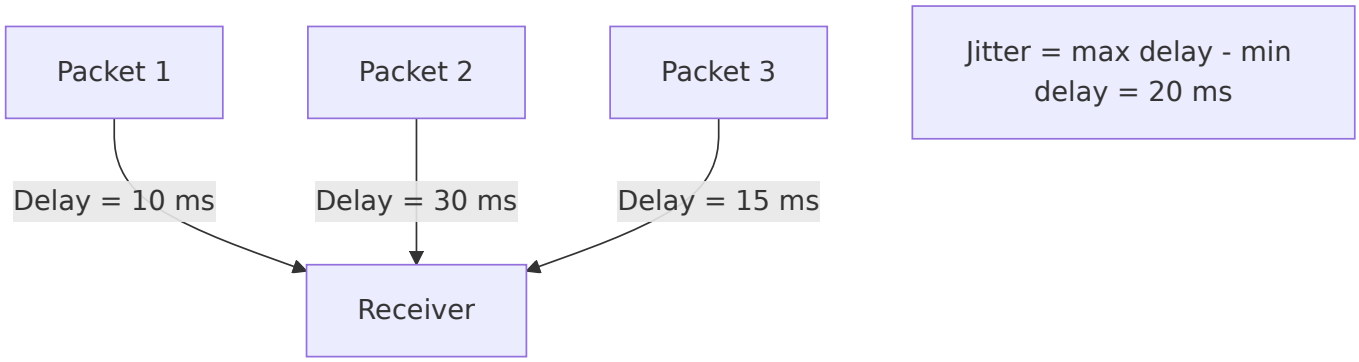
- **Processing delay** – Time to check bit errors and header.
- **Queuing delay** – Time waiting in output buffer.
- **Transmission delay** – Time to push bits onto the link.
- **Propagation delay** – Time for signal to travel the medium.



Example
In online gaming, a round-trip delay (ping) above 100 ms becomes noticeable; above 200 ms the game feels unresponsive.

Jitter

Jitter is the variation in packet arrival times. High jitter causes problems for real-time applications (VoIP, video conferencing) because packets are played out at uneven intervals.



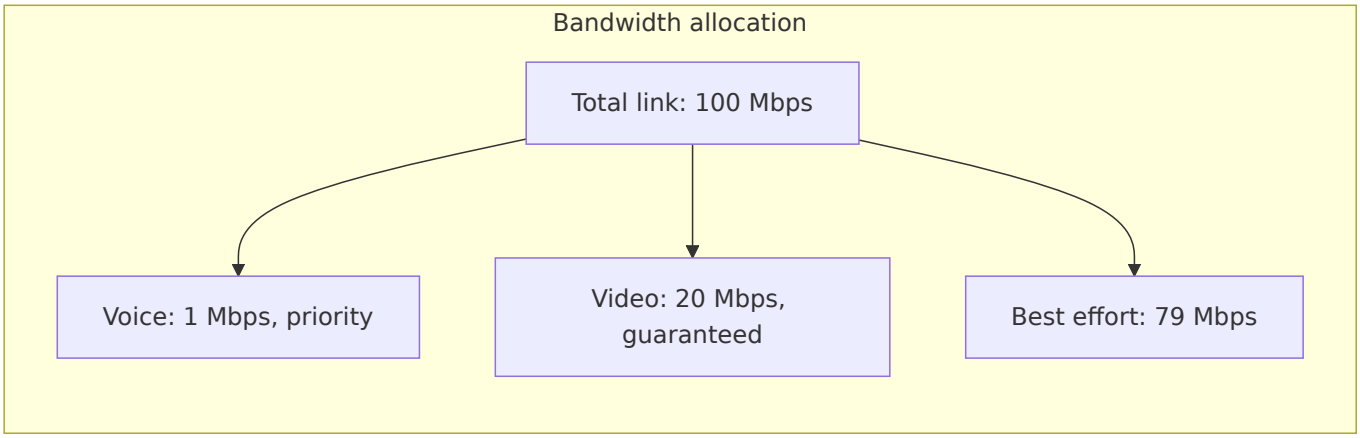
Example
A VoIP call with jitter of 50 ms may require a jitter buffer (adds delay) to smooth out playback; otherwise, audio becomes choppy.

Bandwidth

Bandwidth is the maximum data rate (throughput) of a network path. It is usually measured in bits per second (bps). Different applications have different bandwidth requirements.

Application	Required Bandwidth	Sensitivity
Email / Web	< 1 Mbps	Low (delay tolerant)

Video streaming	3–25 Mbps	Medium (needs stable throughput)
4K Video call	15–30 Mbps	High (jitter sensitive)
Large file download	100+ Mbps	Low (only speed matters)



Summary

- **Congestion** arises from oversubscription of resources; shaping and QoS help mitigate it.
- **Leaky bucket** enforces a constant output rate, **token bucket** allows controlled bursts.
- **QoS** parameters (delay, jitter, bandwidth) define performance expectations. Real-time apps are especially sensitive to jitter and delay.

These mechanisms are implemented in modern networks (e.g., DiffServ, traffic policing, queue management like RED) to ensure fair and predictable communication.

Chapter 10: Wireless and Mobile Networks

This chapter covers the fundamentals of wireless communication, cellular networks, Wi-Fi (IEEE 802.11) including its architecture and medium access control (CSMA/CA), and a basic overview of Mobile IP. Diagrams using Mermaid and practical examples are included.

Basics of Wireless Communication

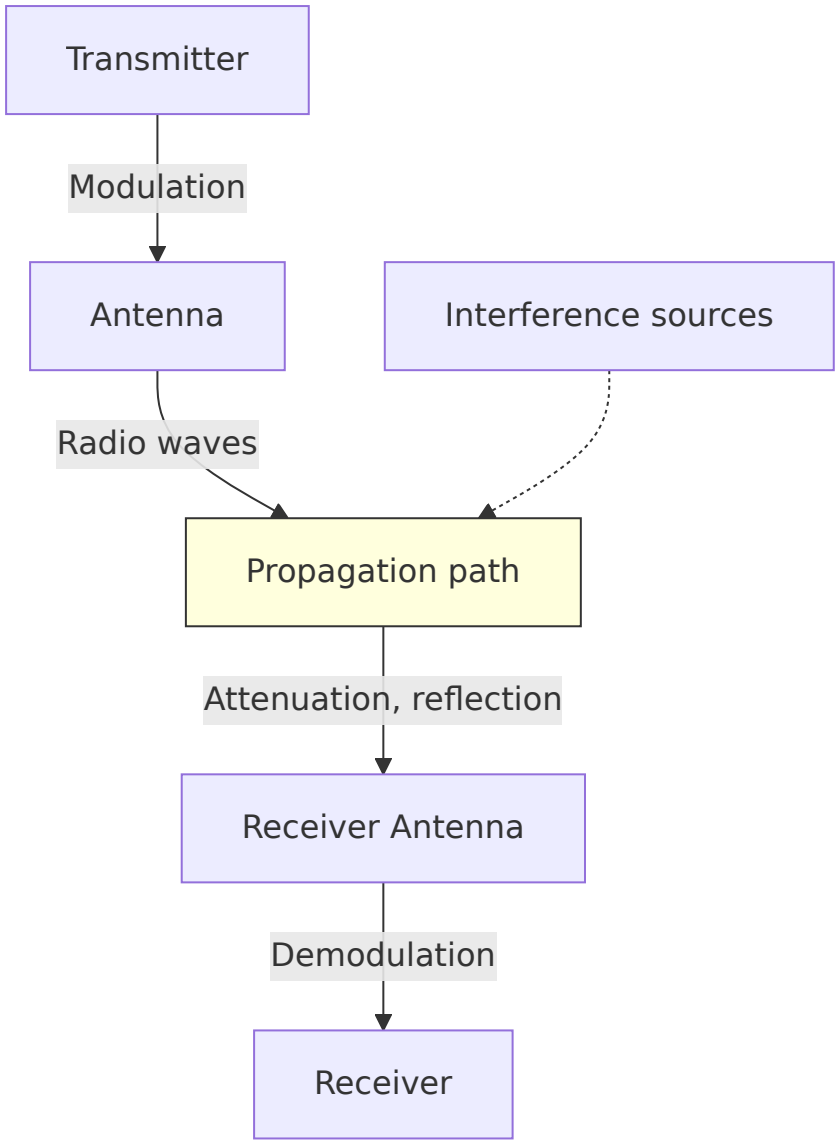
Wireless communication transmits data over the air using electromagnetic waves (radio frequencies, microwaves, infrared). Key challenges and concepts:

- **Signal attenuation** – Signal weakens with distance and obstacles.
- **Interference** – Other devices operating on same frequency cause collisions.
- **Multipath propagation** – Signals bounce off objects, arriving at different times.
- **Hidden terminal problem** – Two stations out of range of each other but both in range of an access point may collide.
- **SNR (Signal-to-Noise Ratio)** – Higher SNR gives better data rates.

Wireless vs. Wired

Feature	Wired (Ethernet)	Wireless (Wi-Fi)
Medium	Copper/fiber	Air (RF)
Collision detection	CSMA/CD (easy)	Hard to detect while transmitting
Interference	Low	High (other networks, devices)
Mobility	Limited by cable	Full mobility within coverage

Basic Wireless Communication Diagram



Cellular Networks

Cellular networks divide geographical area into **cells**, each served by a base station (cell tower). Cells reuse frequencies to maximise capacity.

Key Components

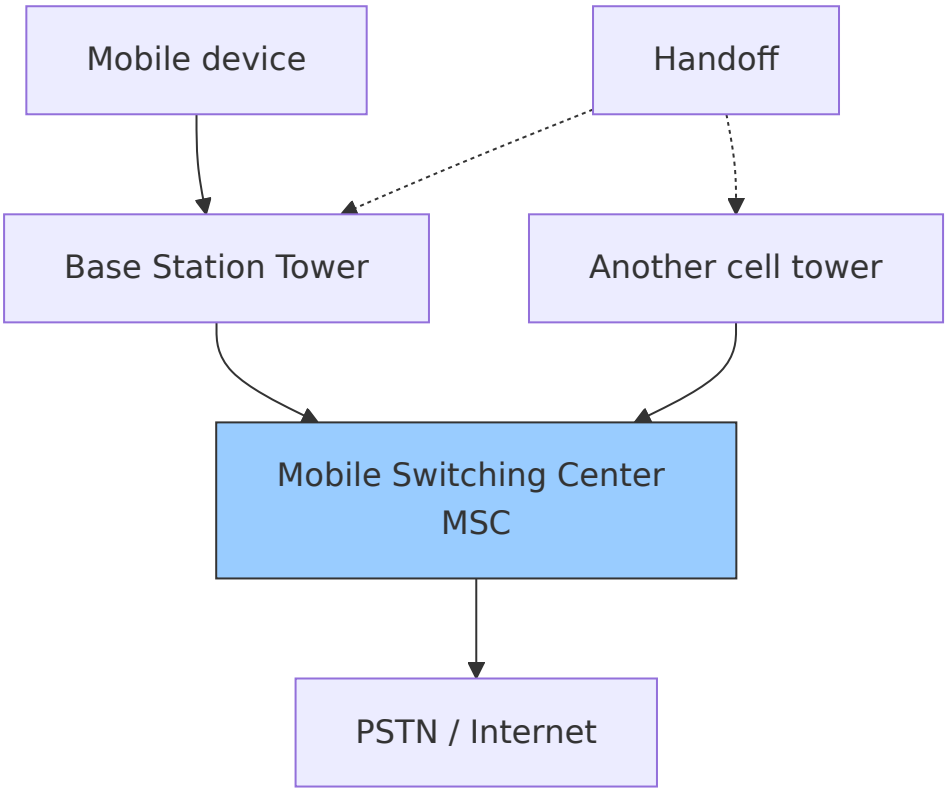
- **Mobile Station (MS)** – User device (phone, modem).
- **Base Station (BS)** – Tower with transceiver.
- **Mobile Switching Center (MSC)** – Connects calls, manages handoff, links to PSTN.
- **Handoff** – Transferring an active call from one cell to another as the user moves.

Generations Overview

Generation	Key Features	Data Rate

2G (GSM)	Digital voice, SMS	~100 kbps
3G (UMTS)	Mobile data, video calls	384 kbps – 2 Mbps
4G (LTE)	All-IP, low latency, streaming	10–100 Mbps
5G	Ultra-low latency, massive IoT, mmWave	1–10 Gbps

Cellular Architecture



Example
While driving and on a 4G LTE call, your phone measures signal strength from neighbouring towers. When the current tower’ s signal drops below a threshold, the MSC coordinates a **hard handoff** (or soft in CDMA) to the stronger tower without dropping the call.

Wi-Fi (IEEE 802.11)

Wi-Fi is a family of wireless local area network (WLAN) standards operating in 2.4 GHz, 5 GHz, and 6 GHz bands. Common versions: 802.11a/b/g/n/ac/ax (Wi-Fi 6).

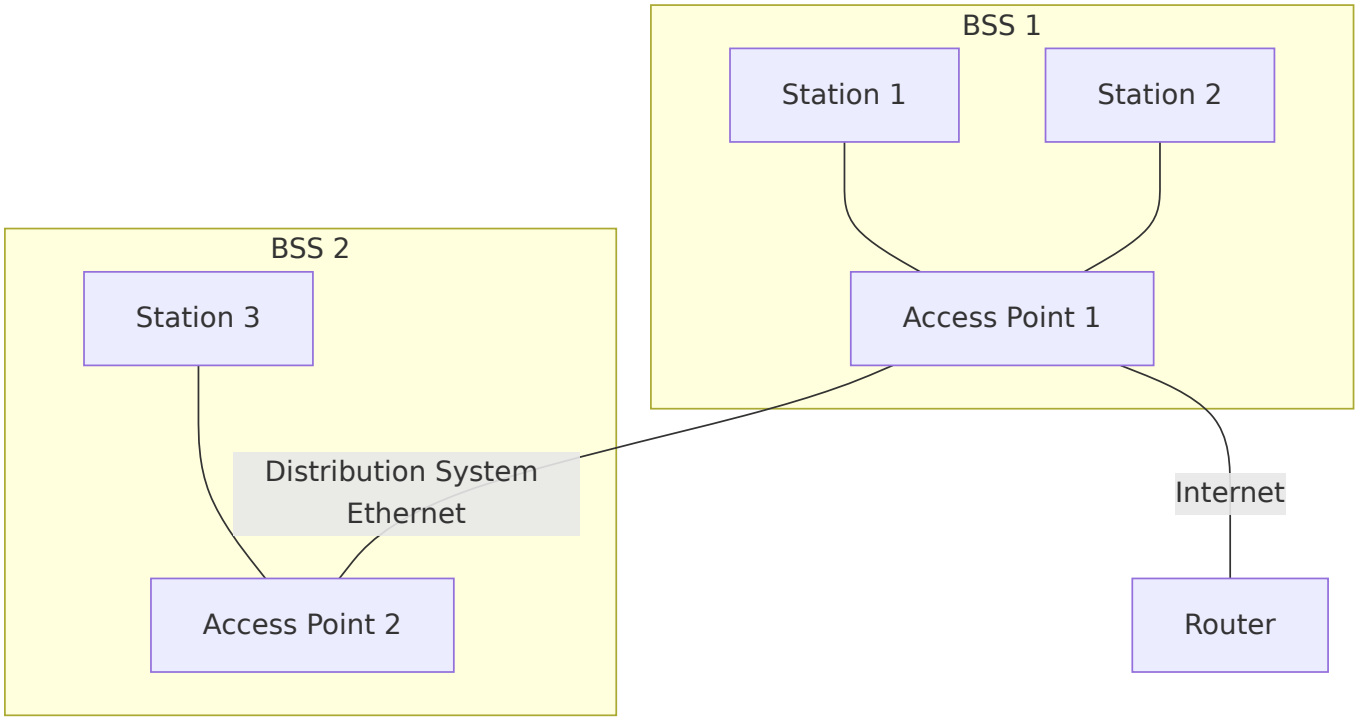
Architecture

Wi-Fi networks use two main modes:

1. **Infrastructure mode** – Devices (stations, STA) communicate via an **Access Point (AP)**. The AP connects to a wired network (e.g., Ethernet, DSL).
2. **Ad-hoc mode** – Devices communicate directly (IBSS – Independent Basic Service Set). Less common today.

Basic Service Set (BSS) – One AP plus associated stations.
Extended Service Set (ESS) – Multiple APs with the same SSID, allowing roaming.

Wi-Fi Architecture Diagram



CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

Wireless cannot use CSMA/CD (Collision Detection) because:

- A station cannot listen while transmitting (half-duplex radio).
- Hidden terminals make collisions hard to detect.

CSMA/CA uses **collision avoidance** via:

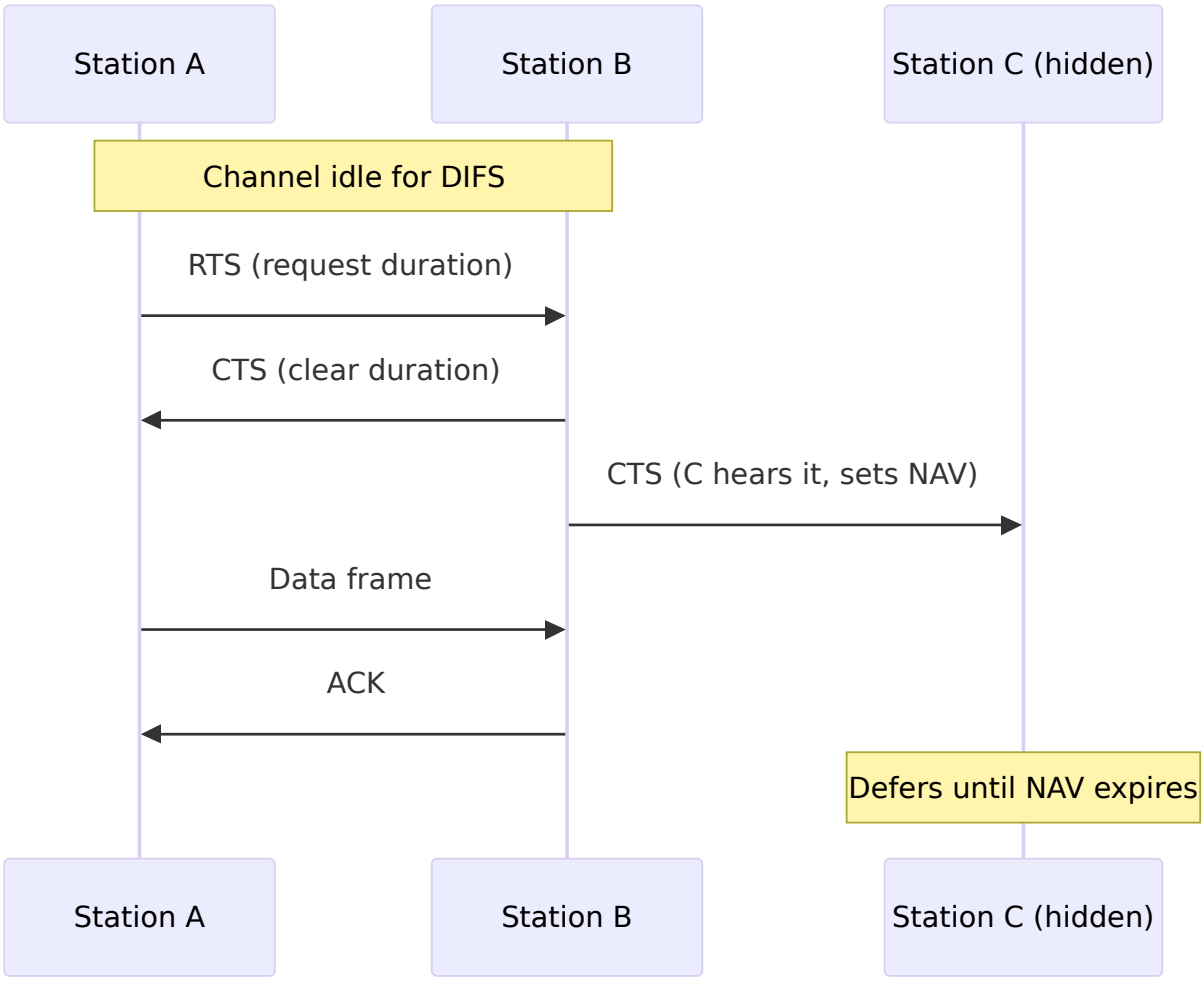
- **Physical carrier sensing** – Listen before talk. If channel idle for DIFS (DCF Interframe Space), transmit.
- **Virtual carrier sensing** – Use **RTS/CTS** (Request to Send / Clear to Send) frames to reserve the channel.

- **Random backoff** – After a busy channel becomes idle, wait random time to reduce simultaneous transmissions.

RTS/CTS Mechanism

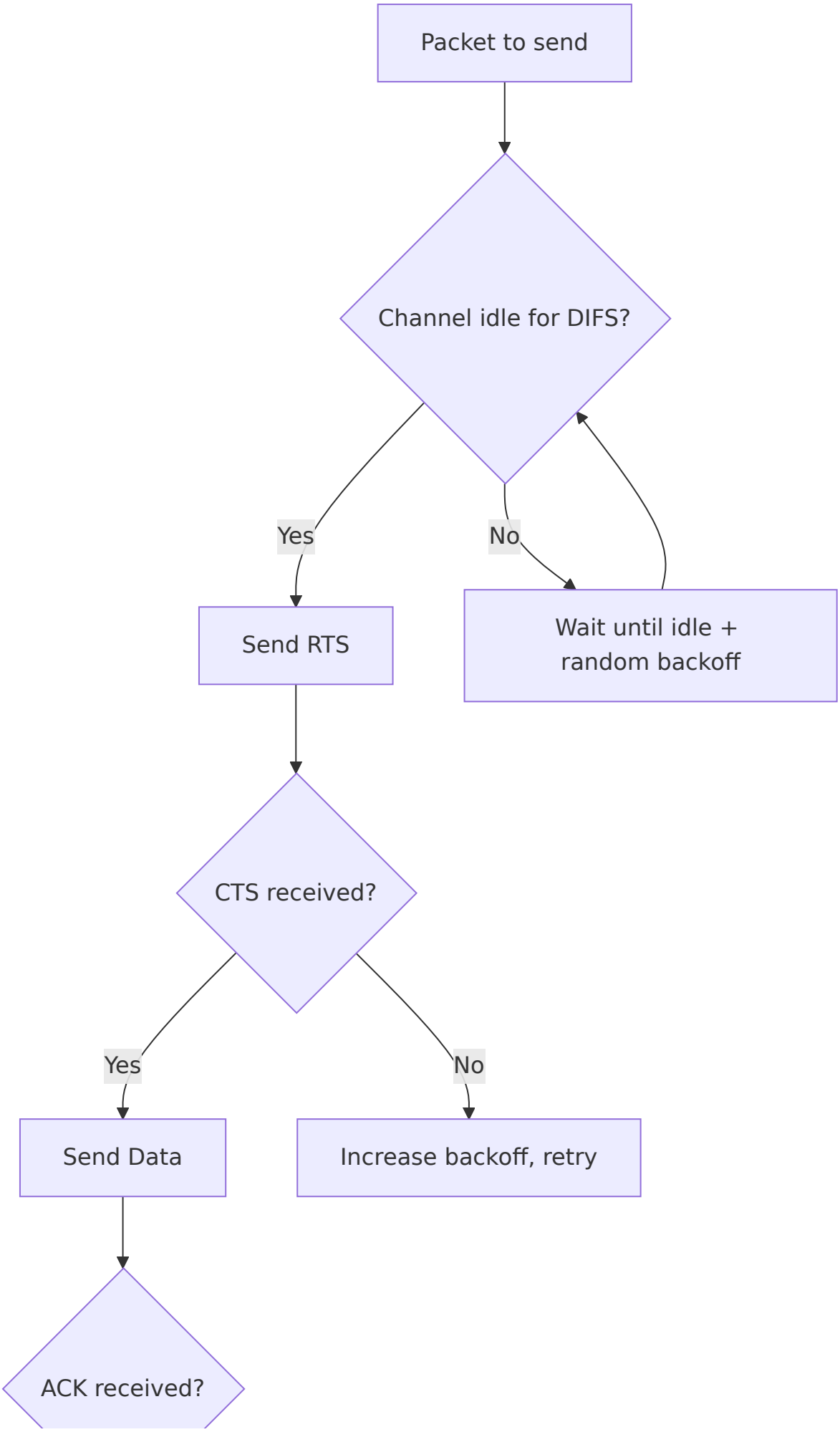
1. Sender sends **RTS** (contains duration).
2. Receiver replies with **CTS** (also contains duration).
3. All other stations hear either RTS or CTS and set their **NAV** (Network Allocation Vector) – a timer to defer transmission.
4. Sender transmits data.
5. Receiver sends **ACK**.

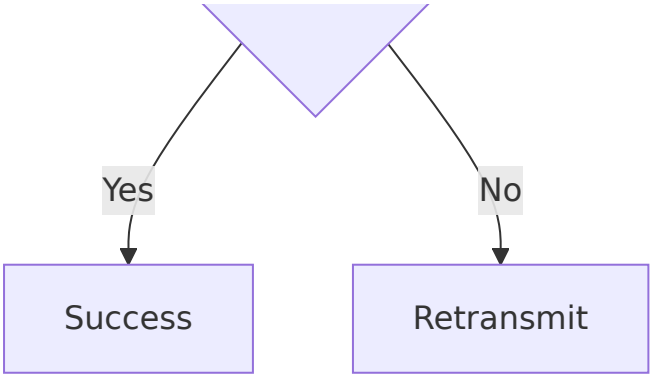
This solves hidden terminal problem.



Example
In a home Wi-Fi network, two laptops near the router but out of each other’ s range (hidden terminals) both try to upload a file. RTS/CTS prevents collisions – one laptop’ s RTS reaches the router, the router sends CTS that both laptops hear, so the other waits.

CSMA/CA Flowchart


Packet to send
Channel idle for DIFS?
Yes
No
Send RTS
Wait until idle +
random backoff
CTS received?
Yes
No
Send Data
Increase backoff, retry
ACK received?



Mobile IP (Basic Overview)

Mobile IP allows a mobile device (mobile node) to change its point of attachment to the Internet without changing its IP address, maintaining ongoing connections (e.g., TCP sessions).

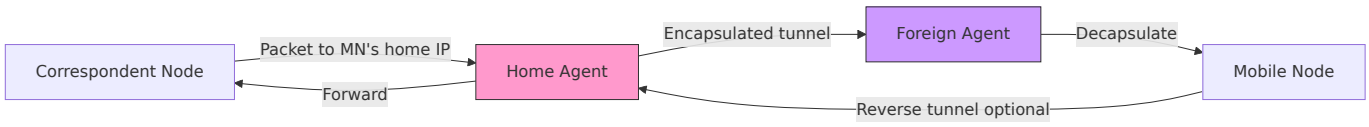
Key Components

- **Mobile Node (MN)** – The roaming device.
- **Home Agent (HA)** – A router on the mobile node’ s home network. Tracks the mobile node’ s current location.
- **Foreign Agent (FA)** – A router on the visited network. Provides care-of address and assists delivery.
- **Care-of Address (CoA)** – The temporary IP address of the mobile node while away from home.

Operation (Three Phases)

1. **Agent Discovery** – HA and FA advertise their presence via ICMP router discovery messages. MN detects when it is away.
2. **Registration** – MN registers its CoA with HA (via FA if needed). HA creates a mobility binding.
3. **Tunneling** – Packets sent to MN’ s home IP are intercepted by HA, encapsulated (IP-in-IP) and forwarded to CoA. FA decapsulates and delivers to MN.

Mobile IP Tunneling Diagram



Example

A travelling salesperson with a laptop (home IP 192.168.1.10) connects to a hotel Wi-Fi. The

hotel’ s router acts as FA. The laptop registers its care-of address (10.0.0.5) with its home agent back at the office. The home agent tunnels any incoming traffic (e.g., ongoing VoIP call) to 10.0.0.5, so the call continues uninterrupted.

Limitations of Mobile IP

- **Triangle routing** – Traffic from CN to MN goes via HA, increasing latency. Route optimisation exists but adds complexity.
- **Security** – Registration and tunnelling need authentication (IPsec).
- **Ingress filtering** – Some networks drop packets with non-topological source addresses.

Summary Table

Topic	Key Points
Wireless basics	Attenuation, interference, hidden terminal, SNR
Cellular networks	Cells, handoff, MSC, evolution 2G→5G
Wi-Fi architecture	BSS, ESS, infrastructure mode, AP, stations
CSMA/CA	Physical + virtual carrier sensing, RTS/CTS, NAV, random backoff
Mobile IP	Home/Foreign agent, care-of address, tunnelling, triangle routing

These concepts are essential for understanding how modern wireless and mobile networks provide connectivity, mobility, and acceptable performance despite inherent physical limitations.

Chapter 11: Numerical Topics in Computer Networks

This document provides a structured overview of essential numerical problems in computer networking. It is intended for students, educators, and practitioners who wish to develop proficiency through example problems and solutions. The following five topics are covered in detail.

Topics Covered

1. **Subnetting** – IPv4 addressing, subnet masks, CIDR, VLSM, network and broadcast address calculation.
2. **Transmission Delay and Propagation Delay** – Packet transmission time, signal propagation time, and total latency.
3. **Bandwidth–Delay Product** – Link capacity multiplied by round-trip time, buffering requirements, and window size optimisation.
4. **CRC (Cyclic Redundancy Check)** – Polynomial arithmetic, modulo-2 division, error detection, and generator polynomials.
5. **Sliding Window Protocol** – Go-Back-N, Selective Repeat, window size, sequence number space, and throughput computation.

Each section includes:

- Key formulas with clear notation.
- Step-by-step worked examples.
- Practice problems (solutions provided in the `/solutions` directory).
- Python scripts for automation (located in `/scripts`).

---

1. Subnetting

Key Formulas

- Number of subnets = 2^s, where `s` is the number of borrowed bits.
- Hosts per subnet = $2^h - 2$, where `h` is the number of host bits remaining.
- Subnet mask = network prefix with `s` additional 1s.

- Block size (address increment) = 2^h.

Example Problem

Given: 192.168.1.0/24, need 4 subnets.
Solution:

- Borrow `s = log2(4) = 2` bits.
- New subnet mask = /26 (255.255.255.192).
- Host bits remaining `h = 6` → hosts per subnet = $2^6 - 2 = 62$.
- Subnet addresses: 192.168.1.0, 64, 128, 192.

---

2. Transmission Delay and Propagation Delay

Key Formulas

- Transmission delay = `L / R`
  (L = packet length in bits, R = link transmission rate in bps)
- Propagation delay = `d / s`
  (d = physical distance, s = signal propagation speed, typically 2×10^8 m/s for copper/fibre)
- Total latency = Transmission delay + Propagation delay + queuing + processing
  (queuing/processing often omitted in basic problems)

Example Problem

Given: Packet length 1500 bytes, link speed 10 Mbps, distance 2000 km, propagation speed 2×10^8 m/s.
Solution:

- Transmission delay = $(1500 \times 8) / (10 \times 10^6) = 0.0012$ s = 1.2 ms.
- Propagation delay = $2000 \times 10^3 / (2 \times 10^8) = 0.01$ s = 10 ms.
- Total latency = 1.2 + 10 = 11.2 ms.

---

3. Bandwidth–Delay Product

Key Formula

- Bandwidth–delay product (BDP) = Bandwidth (bps) × Round-Trip Time (RTT in seconds)
  Units: bits (or bytes when divided by 8).

Significance

BDP represents the amount of data "in flight" on the link before the sender receives an acknowledgment. It determines the minimum window size required for full link utilisation.

Example Problem

Given: Bandwidth = 100 Mbps, RTT = 50 ms.
Solution:
BDP = $100 \times 10^6 \times 0.05 = 5 \times 10^6$ bits = 625,000 bytes.
To saturate the link, the sender's window should be at least 625,000 bytes (or approximately 417 packets of 1500 bytes).

---

4. Cyclic Redundancy Check (CRC)

Key Concepts

- CRC uses polynomial division (modulo-2) to generate a checksum.
- Sender appends `n` check bits (where `n` = degree of generator polynomial).
- Receiver divides received data by the same generator; remainder zero indicates no error.

Procedure

1. Append `n` zero bits to the data word (where `n` = degree of generator polynomial).
2. Divide the augmented data by the generator polynomial using XOR (modulo-2 division).
3. The remainder (of length `n`) is the CRC checksum.
4. Transmit original data followed by the checksum.
5. Receiver performs division; if remainder ≠ 0, error detected.

Example Problem

Given: Data = `110101`, Generator = `1011` (degree 3).
Solution (steps):

- Append 3 zeros: `110101000`.

- Divide by `1011` using XOR:
  (Detailed division yields remainder `011`).
- Transmitted frame: `110101011`.
- At receiver, division by `1011` gives remainder `000` → no error.

---

5. Sliding Window Protocol

Key Formulas

- Maximum window size (Go-Back-N) = $2^n - 1$, where `n` = number of bits in sequence number field.
- Maximum window size (Selective Repeat) = 2^{n-1}.
- Throughput = (Window size × Packet size) / RTT, provided window ⩽ BDP.

Example Problem (Go-Back-N)

Given: 3-bit sequence number (0–7), RTT = 200 ms, packet size = 1000 bytes, bandwidth = 2 Mbps.
Solution:

- Max window size = $2^3 - 1 = 7$ packets.
- BDP = $2 \times 10^6 \times 0.2 = 400{,}000$ bits = 50,000 bytes = 50 packets (of 1000 bytes).
- Window (7 packets) is smaller than BDP (50 packets), so link is not fully utilised.
- Throughput = $(7 \times 1000 \times 8) / 0.2 = 280{,}000$ bps = 0.28 Mbps.

Usage

1. Study the formulas and worked examples in each section.
2. Attempt the practice problems located in `/problems`.
3. Verify your answers using the solution guides in `/solutions`.
4. Use the Python scripts in `/scripts` to automate calculations and test edge cases.

Contribution

Contributions are welcome. Please ensure that all additions include clear derivations, maintain a professional tone, and include test cases where applicable.

License

Chapter 12: Important Interview Topics

This chapter covers fundamental networking and protocol concepts frequently discussed in technical interviews. Each topic is presented with concise explanations, key differences, and practical insights.

1. TCP vs UDP

Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are core transport layer protocols.

Feature	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Acknowledgment, retransmission, sequencing	No guarantees
Flow Control	Yes (sliding window)	No
Congestion Control	Yes	No
Ordering	Preserves packet order	No ordering
Overhead	Higher (20 byte header)	Lower (8 byte header)
Speed	Slower	Faster
Usage	HTTP, HTTPS, FTP, SSH, email	DNS, VoIP, streaming, gaming, DHCP

Key takeaway: Use TCP when data integrity and order matter; use UDP when speed and low latency are critical, and occasional loss is acceptable.

2. OSI vs TCP/IP Models

Both models describe network protocol stacks but differ in layer granularity and historical origin.

OSI Model (7 layers):

1. Physical
2. Data Link
3. Network
4. Transport
5. Session
6. Presentation
7. Application

TCP/IP Model (4 layers):

1. Network Interface (combines OSI 1-2)
2. Internet (OSI 3)
3. Transport (OSI 4)
4. Application (OSI 5-7)

OSI Layer	TCP/IP Layer	Example Protocols
Application	Application	HTTP, FTP, DNS, SMTP
Presentation	Application	TLS/SSL (partly), JPEG
Session	Application	NetBIOS, RPC
Transport	Transport	TCP, UDP
Network	Internet	IP, ICMP, ARP
Data Link	Network Interface	Ethernet, Wi-Fi, PPP
Physical	Network Interface	RJ45, fiber, radio

Key takeaway: OSI is a theoretical reference model; TCP/IP is the practical model of the internet.

3. HTTP vs HTTPS

Feature	HTTP	HTTPS
Full form	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Encryption	None (plain text)	TLS/SSL encryption

Port	80	443
Security	Vulnerable to eavesdropping, MITM	Confidentiality, integrity, authentication
Performance	Slightly faster (no crypto overhead)	Slightly slower due to handshake + encryption
SEO impact	Negative (browsers mark as unsafe)	Positive (ranking boost, trust indicators)
Certificate	Not required	Requires CA-signed certificate

How HTTPS works:

- Server presents a digital certificate.
- Client verifies certificate against trusted CAs.
- A symmetric session key is negotiated via asymmetric encryption (TLS handshake).
- All subsequent data is encrypted with that session key.

Key takeaway: Always use HTTPS for production websites to protect user data and build trust.

4. DNS Working (Domain Name System)

DNS translates human-readable domain names (e.g., `example.com`) to IP addresses.

Resolution steps (recursive query):

1. Browser checks local cache (OS + browser).
2. If not found, query sent to **Recursive Resolver** (usually ISP or public DNS like 8.8.8.8).
3. Resolver queries a **Root nameserver** → returns TLD nameserver address.
4. Resolver queries **TLD nameserver** (e.g., for .com) → returns authoritative nameserver for domain.
5. Resolver queries **Authoritative nameserver** → receives the IP address (A or AAAA record).
6. Resolver returns IP to client; client caches result (TTL governs lifetime).

Record types:

- A : IPv4 address
- AAAA : IPv6 address

- CNAME : Canonical name (alias)
- MX : Mail exchange
- TXT : Text (e.g., SPF, DKIM)
- NS : Nameserver

Key takeaway: DNS is a distributed, hierarchical database with caching at every level to reduce latency.

5. 3-Way Handshake & 4-Way Termination

These are the connection establishment and teardown procedures in TCP.

3-Way Handshake (SYN, SYN-ACK, ACK):

1. Client → Server: `SYN` (seq=x)
   - Client sends a TCP segment with SYN flag set, choosing an initial sequence number.
2. Server → Client: `SYN-ACK` (seq=y, ack=x+1)
   - Server acknowledges the SYN and sends its own SYN with a different sequence number.
3. Client → Server: `ACK` (ack=y+1)
   - Client acknowledges the server's SYN. Connection is now established (ESTABLISHED state).

State transitions:

- Client: CLOSED → SYN_SENT → ESTABLISHED
- Server: LISTEN → SYN_RCVD → ESTABLISHED

4-Way Termination (FIN, ACK, FIN, ACK):

Either side can initiate termination. Assume client initiates:

1. Client → Server: `FIN` (seq=u)
   - Client says "I have no more data to send."
2. Server → Client: `ACK` (ack=u+1)
   - Server acknowledges the FIN. Server may continue sending data (half-closed state).
3. Server → Client: `FIN` (seq=v, ack=u+1)
   - Server finishes sending its data and sends its own FIN.

4. Client → Server: `ACK` (ack=v+1)
    - Client acknowledges the FIN. After a wait (TIME_WAIT), client closes.

Key takeaway: The three-way handshake establishes reliable, ordered connections; the four-way termination ensures both sides agree to close without data loss.

6. NAT Working (Network Address Translation)

NAT allows multiple devices on a private network to share a single public IPv4 address.

Types:

- **Static NAT** : One-to-one mapping (private IP ↔ public IP). Used for servers.
- **Dynamic NAT** : Pool of public IPs assigned to private IPs as needed.
- **PAT (NAPT)** : Many-to-one mapping using port numbers. Most common in home routers.

How PAT works:

- Private device (192.168.1.10:12345) sends packet to destination.
- Router replaces source IP with its public IP and assigns a unique source port (e.g., 55001).
- Router maintains a **NAT table** mapping `(private IP:port) ↔ (public IP:port, dest IP:port)`.
- Return packets are matched based on destination port and translated back.

Key benefits:

- Conserves IPv4 addresses.
- Hides internal network topology (basic firewall).

Drawbacks:

- Breaks end-to-end connectivity (some protocols like IPsec or FTP need ALGs).
- Adds latency and processing overhead.

NAT traversal techniques: STUN, TURN, ICE (used in VoIP, gaming, WebRTC).

7. Difference Between Hub, Switch, and Router

These are network devices operating at different layers of the OSI model.

Feature	Hub (Layer 1)	Switch (Layer 2)	Router (Layer 3)

OSI Layer	Physical	Data Link	Network
Intelligence	None (dumb repeater)	Learns MAC addresses	Routes between networks
Traffic handling	Broadcasts to all ports	Forwards only to destination port	Uses IP routing table
Collision domain	Single (all ports share)	Each port separate	Each interface separate
Broadcast domain	Single	Single (by default, unless VLAN)	Boundaries broadcast
Address type	None (electrical signals)	MAC addresses	IP addresses
Performance	Low (collisions degrade)	High (full duplex)	Varies (depends on routing)
Typical use	Obsolete; lab/legacy only	Connecting devices inside a LAN	Connecting LAN to WAN/Internet

Illustrative traffic flow:

- Hub: Packet from A to B goes to all ports (C, D, E see it).
- Switch: After learning, A→B goes only to B's port.
- Router: Forwards between different IP subnets (e.g., LAN to Internet).

Key takeaway: Use switches for internal LAN efficiency, routers for inter-network connectivity; hubs are deprecated.